Week 7 · Four Tilting Thrusters, and the Open-Loop Check

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-11-03

ℹ Recordings of this semester's own classes

Every session of this course is recorded as it is given and posted to the playlist below. The written material here is the reference; the recording is the lecture, at the pace it was actually delivered. Use it to review a derivation, or to catch up on a missed week before the laboratory.

	Where
[충남대] 2026 센서신호처리 — this course, recorded live, 2026	playlist

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	**Control Engineering** (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	**Control System Design** (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	**Advanced Control Engineering** (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	**Sensor Signal Processing and Fusion** (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	**Capstone Design** (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

✎ Getting the course files, and keeping them current (Windows)

The notes, models and scripts are kept in one Git repository that is updated through the semester. Clone it once; before every class, pull. A pull downloads only what has changed since the last one.

When	Where to run it	Command
once	PowerShell — installs Git for Windows	`winget install --id Git.Git -e`
once	the folder that will hold the course, e.g. `Documents`	`git clone https://github.com/wkyouncnu/Sensor-Signal-Processing-and-Fusion.git`
before every class	inside the cloned folder `Sensor-Signal-Processing-and-Fusion`	`git pull`

- `git pull` prints `Already up to date.` when nothing has changed, and otherwise lists the files it updated.
- The repository is private. When Git asks, sign in with the GitHub account the instructor has given access.
- Experiment on copies, not on the cloned files: copy a week's `WXX_simulink` folder elsewhere first, and a pull can never collide with local edits. If it already has, `git stash`, then `git pull`, then `git stash pop` sets the edits aside, updates, and puts them back.
- The MSS toolbox is not part of the repository. The weeks that simulate the Otter need it at `Tools\MSS` inside the cloned folder, and `_tools\mss_path.m` finds it there in either release layout — `Tools\MSS\VESSELS\otter.m` in the older releases, or `Tools\MSS\CRAFT\USV\models\otter.m` in the current master.
- **The two releases are not the same vessel.** The figures printed in these notes were measured with the older layout. Terminal surge speed agrees at 1.0286 m/s, but the current master turns 25.5 % faster under differential thrust, 0.1276 rad/s against 0.1017 rad/s. `mss_path` therefore prefers the older layout when both are present, and warns when only the newer one is available, so a reading that cannot reproduce is announced rather than silent.
- **A 118 MB folder disappears on the first pull after 2026-10-27.** `lectures\MSS-master` held a copy of MSS until then. It sat outside `Tools\MSS`, so no course script could reach it, and it only made cloning slower. Its removal is expected and is not a fault. Any file kept inside that folder must be moved elsewhere before pulling.

- **Course**: USV Guidance, Navigation and Control (Graduate)
- **Department**: Autonomous Vehicle System Engineering, Chungnam National University
- **This week**: ① a tilting thruster occupies **two** columns of B, not one ② the four places `otter.m` must change to carry four of them ③ eight numbers given by hand, and the three asymmetries that make the symmetric guess wrong

★ Before starting

- Week 3's column rule must be available from memory: a thrust applied at a point contributes one column to B, and the rule that builds that column is the only definition of B this course uses.
- Week 6's change to the motion model must be understood as a change to the **model**, not an addition to its input. `otter_bow.m` carries the bow thrust inside τ; this week does the same thing for four tilting units.
- Verify both in one command: `cd lectures/W06_simulink; W06_0_setup` must print the three-thruster configuration without error, and `verify_otter_bow` must print `PASS`.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. State the column rule for a thrust applied at (x_i, y_i), and show from it that a fixed thruster occupies one column of B while a tilting thruster occupies two.
2. Write the generalised force of four tilting thrusters as $\boldsymbol{\tau} = B\mathbf{f}$ with $B \in \mathbb{R}^{3\times 8}$, state its rank and the dimension of its null space, and confirm that $\delta_i = 0$ recovers the two-thruster expression of `otter.m` exactly.
3. Identify the four places in `otter.m` that must change to carry four tilting units, and state why the hull hydrodynamics are not among them.
4. Predict the surge force left over when a shaft speed $+n$ is opposed by $-n$, compute the astern speed that cancels it, and confirm both against the measured table.
5. Derive $N = \bar{x}\,Y$ with $\bar{x} = (l_f - l_a)/2$ for four equally tilted thrusters, predict the residual yaw moment of a symmetric sway command to three decimal places, and derive the thrust ratio that removes it.
6. Distinguish a check that the equations are right from a check that the model is wired to those equations, and state why re-integrating the logged inputs outside Simulink is the second of these while a single-instant acceleration comparison is neither.
7. State why the three measured asymmetries make an allocation necessary rather than optional, and name the input, the output and the fixed data of the allocation stage.
8. Derive the minimum-norm allocation $\mathbf{f} = \mathbf{B}^{\top}(\mathbf{B}\mathbf{B}^{\top})^{-1}\boldsymbol{\tau}_d$ from a Lagrange multiplier, invert the thrust curve to n_i and δ_i, and say why the thrust cap needs one scalar after the solve while the servo limit does not.
9. State what a weight matrix changes and what it cannot change on this geometry, explain why only the ratios of its entries matter, and tune it to move load between units rather than to reduce tilt.
10. Turn $|\delta_i| \le \alpha_{\max}$ into two linear inequalities once the thrust sign is fixed, say why four units give sixteen convex subsets and what taking only the sign combination of the closed form risks, and read a measured solve time against the $20\,000$ µs budget of a 50 Hz loop.
11. Show that the shaft-speed limit $|n_i| \le n_{\max}$ is the thrust cap $|T_i| \le k_i n_{\max}^2$ seen through the thrust curve, replace the resulting circular arc by inscribed chords, and state why the polygon can never violate the cap.
12. Measure the force the motion model actually produced rather than the force the allocator reported, and explain why a law that clips inside the allocator and a law that leaves the clipping to the hardware deliver the same force.
13. State what this week deliberately leaves out, and why a controller is not needed to study allocation.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b or later, with Simulink
MSS toolbox	reachable at `Tools\MSS`; `_tools\mss_path.m` adds it
Optimization Toolbox	version 24.2 or later, for `lsqlin`, `quadprog` and `fmincon` in §7-10 and §7-11. Verify with `ver('optim')`
Weeks assumed	Week 3 (the column rule and $\boldsymbol{B}$), Week 5 (saturation and limits), Week 6 (changing the motion model itself, and the square allocation of §6-2 with its clipping), Appendix A1 (the rank of $\boldsymbol{B}$, the extended thrust vector, and the attainable control set)
Plant	`_tools\otter_tilt4.m` — `otter.m` with four tilting units, same hull
Configuration	`otter_config('quad_tilt')`
Model	`W07_openloop.slx`, written by `W07_1_build_openloop.m`
Solver	fixed-step `ode4`, $h = 0.02$ s
Allocation scripts	`W07_alloc.m` (eight laws behind one flag), `W07_hw.m` (thrust-curve inverse) — no model required
Scope of this week	§7-0 to §7-6 are **open loop only** — eight numbers given by hand. §7-7 to §7-12 add the **allocation** that produces those eight numbers from three. **No controller and no guidance in either half**

Part 1 · Principle and experiment, section by section

Every section states what is observed, says what follows from it, and then runs **the experiment that measures it**, on its own model. The numbers quoted in the text are printed by the run a few lines below it.

7-0. Setting up (10 min)

```
cd lectures/W07_simulink
W07_1_build_openloop          % once — writes W07_openloop.slx and img/*.png
W07_0_setup                   % every session — defines every constant
```

`W07_1_build_openloop` prints one line, and both counts must be zero:

```
  W07_openloop   overlaps 0   diagonals 0
```

`W07_0_setup` prints the configuration this week works in:

```
Week 7  four tilting thrusters, open loop
  thruster stations   x = [+1.50 +1.50 -1.00 -1.00] m
                      y = [-0.395 +0.395 -0.395 +0.395] m
  servo limit         +45 deg,  rate 90 deg/s
  shaft limits        -101.737 .. +103.931 rad/s
  thrust at +-60      +39.888 N ahead, -23.202 N astern   (k_neg/k_pos = 0.5817)
  cancelling astern    -78.670 rad/s, not -60
  B                   3 x 8, rank 3, null space 5
  command             n = [60 60 60 60] rad/s
                  delta = [45 45 45 45] deg
```

Four lines of that block are the week in miniature. The shaft limits are not symmetric, the two thrust coefficients are not equal, B has five more columns than rows, and the astern speed that cancels $+60$ rad/s is not -60.

> **⚠ The shaft command is in rad/s, not rpm**
>
> `n_cmd = [60 60 60 60]` is 60 radians per second, about 573 rpm. `otter.m` has always used rad/s and `otter_tilt4.m` keeps that, because the thrust coefficients k_{pos} and k_{neg} are quoted against it. Entering 600 expecting rpm asks for 5730 rpm, and the model saturates it to 103.931 rad/s.

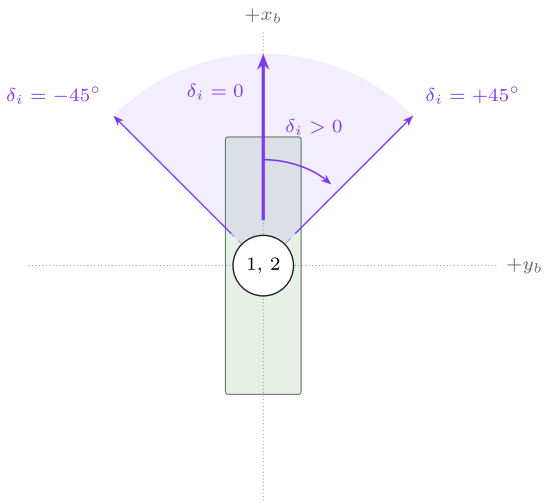
7-1. Four thrusters, two columns each, and two different zeros

What is observed. Experiment 7-1 reports that the forward pair at $\delta = 0$ reproduces `otter.m` exactly, that the aft mounting sign flips $\boldsymbol{\tau}$ to 3.55×10^{-15}, that $\boldsymbol{\tau}$ agrees with $B\mathbf{f}$ to 7.19×10^{-14} over 200 random states, and that B is 3×8 with rank 3.

- four thrust stations: two at $x_b = +1.50$ m, two at $x_b = -1.00$ m, each 0.395 m off centreline
- each unit turns through $\pm 45°$ at up to $90°$/s
- the two pairs are mounted facing **opposite** ways, so $\delta_i = 0$ drives the hull ahead at the bow and astern at the stern
- eight numbers are commanded; three components of force result

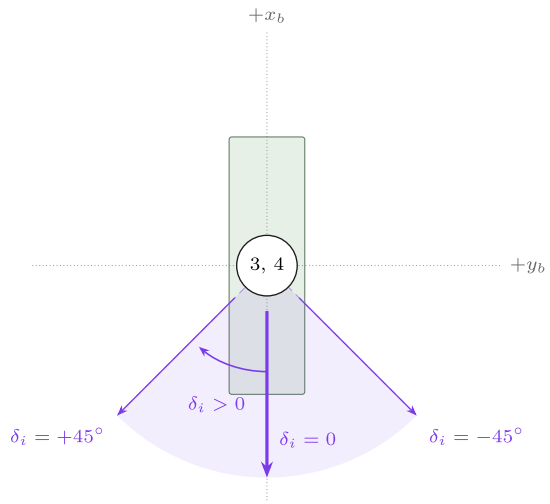
The mounting comes first, because every column of B carries it.

units 1 and 2 · the bow pair · $s_i = +1$



zero pushes the hull **ahead**, so $n_i > 0$ drives it forward

units 3 and 4 · the stern pair · $s_i = -1$



zero pushes the hull **astern**, so $n_i > 0$ drives it backward

The bow is up in both cells. Each pair measures δ_i from its *own* zero towards y_b, so a positive δ_i turns clockwise here in both. Only the zero differs, and that difference is the sign s_i carried by the columns of B.

The result. Write $s_i = +1$ when unit i's zero points ahead and $s_i = -1$ when it points astern. The thrust direction and the force of unit i are then

$$\mathbf{e}_i = s_i \begin{bmatrix} \cos\delta_i \\ \sin\delta_i \end{bmatrix}, \qquad \mathbf{f}_i = T_i \begin{bmatrix} \cos\delta_i \\ \sin\delta_i \end{bmatrix}, \qquad s = [+1 \quad +1 \quad -1 \quad -1]$$

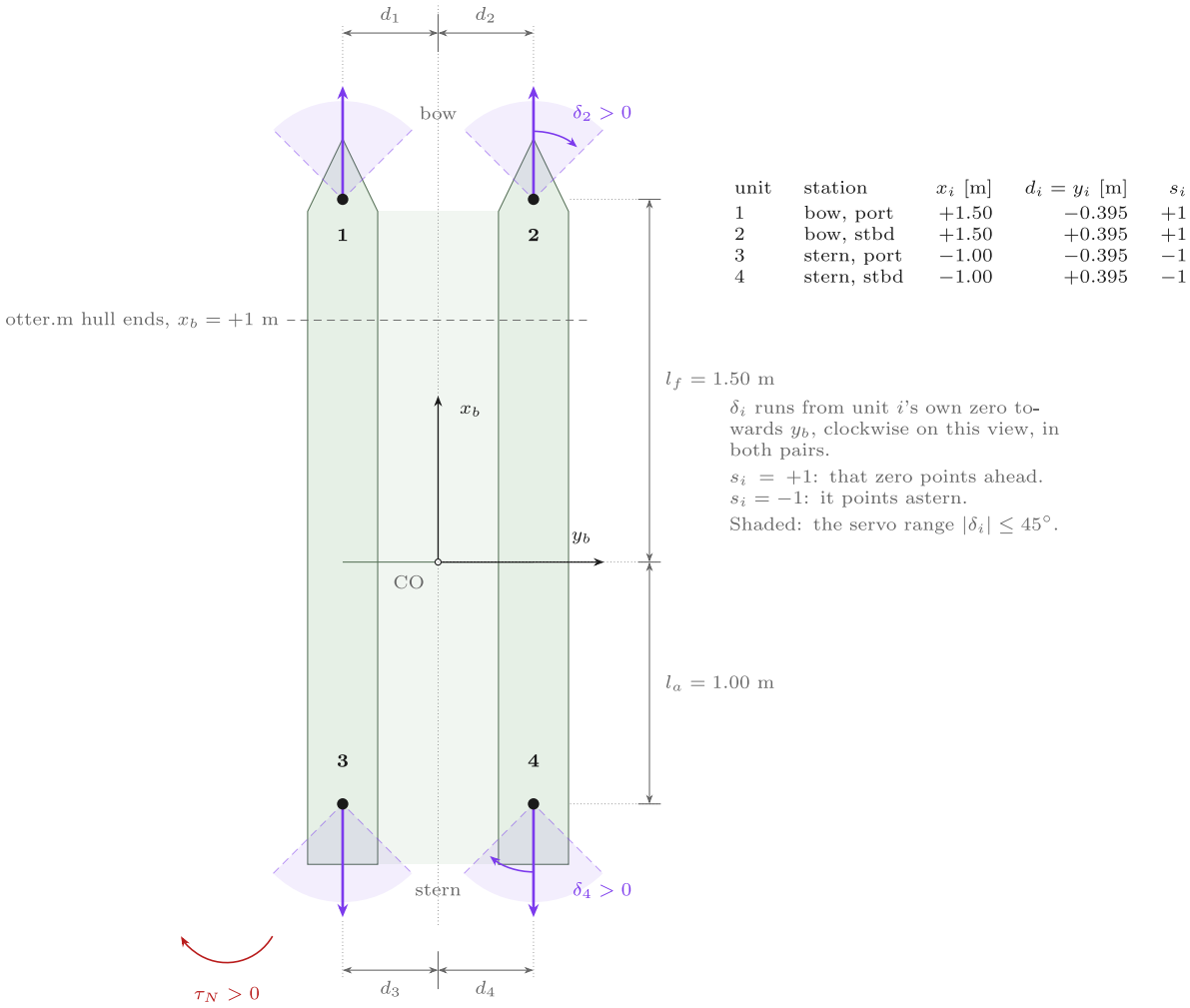
The sign is deliberately kept out of $\mathbf{f}$ and placed in B, so that $\mathbf{f}$ holds only what the hardware reports.

Which direction is called the zero of a servo is a **convention**, not a property of the hull. Calling the other direction zero at all four units gives $s = [-1 \quad -1 \quad +1 \quad +1]$ and therefore $-B$, and a source written that way is not wrong — it is a different labelling of the same four thrusters. What is not free is **consistency**: B, the servo zeros drawn in the figures, the commanded δ_i and every measured number of §7-3 must come from one choice. This course fixes that choice once, in the `mount` field of `otter_config.m`. The motion model writes B out from that field, `_tools/otter_B.m` builds the same matrix from the column rule, and `verify_otter_tilt4.m` compares the two every time it runs, so the matrix and the convention cannot drift apart.

With the column rule applied to each component separately, the force on the hull is linear in those eight numbers:

$$\boldsymbol{\tau} = \begin{bmatrix} X \\ Y \\ N \end{bmatrix} = B\mathbf{f}, \qquad \mathbf{f} = \begin{bmatrix} T_1\cos\delta_1 \\ T_1\sin\delta_1 \\ \vdots \\ T_4\cos\delta_4 \\ T_4\sin\delta_4 \end{bmatrix}, \qquad \mathbf{b}_i^c = s_i \begin{bmatrix} 1 \\ 0 \\ -y_i \end{bmatrix}, \quad \mathbf{b}_i^s = s_i \begin{bmatrix} 0 \\ 1 \\ x_i \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 0 & 1 & 0 & -1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 1 & 0 & -1 & 0 & -1 \\ d_1 & l_f & d_2 & l_f & d_3 & l_a & d_4 & l_a \end{bmatrix}$$



unit	station	x_i [m]	$d_i = y_i$ [m]	s_i
1	bow, port	+1.50	−0.395	+1
2	bow, stbd	+1.50	+0.395	+1
3	stern, port	−1.00	−0.395	−1
4	stern, stbd	−1.00	+0.395	−1

δ_i runs from unit i's own zero towards y_b, clockwise on this view, in both pairs.
$s_i = +1$: that zero points ahead.
$s_i = -1$: it points astern.
Shaded: the servo range $|\delta_i| \le 45°$.

The first two rows carry only the mounting sign, because a force adds to X and Y whatever point it acts at. The third row is where the geometry enters, and **every entry in it is a moment arm**: a length, in metres, and never anything

else. Two arms exist on this hull, one for each kind of column. Each is named once, with its sign already inside the name, so that row 3 of the matrix is six names and nothing else — this is exactly how `otter_tilt4.m` writes it.

Column of B	It multiplies	Its entry in row 3	What that length measures
$\mathbf{b}_i^c$, the $\cos$ column	$T_i \cos\delta_i$, the part of unit i's thrust along x_b	$d_i \equiv -s_i y_i$	**transverse** arm: centreline to unit i's pontoon, $\lvert d_i \rvert = 0.395$ m, signed by which way that thrust turns the bow
$\mathbf{b}_i^s$, the $\sin$ column	$T_i \sin\delta_i$, the part of the same thrust along y_b	$l_f \equiv s_1 x_1$, $l_a \equiv s_3 x_3$	**longitudinal** arm: midships CO to the thrust station, $l_f = 1.50$ m forward, $l_a = 1.00$ m aft

A moment arm is always measured **perpendicular to the force it multiplies**. A thrust along x_b can only turn the hull by acting off the centreline, so its arm is the transverse half-separation d_i; a thrust along y_b can only turn the hull by acting away from midships, so its arm is the longitudinal station l_f or l_a. This is why the two arms of one unit are different lengths although they belong to the same thruster.

Two readings of row 3 follow directly, and both are measured in §7-3.

- the four $\sin$ entries are **all positive** — at the bow $s_i = +1$ and $x_i = +l_f$, at the stern $s_i = -1$ and $x_i = -l_a$, and the two negatives cancel. A positive δ at any of the four units therefore turns the hull the same way, which is why S3 tilts all four alike and gets a pure yaw
- the four $\cos$ entries alternate in sign across the diagonal of the hull: $d_1 = +d$, $d_2 = -d$, $d_3 = -d$, $d_4 = +d$. Port-bow and starboard-stern thrust turn the bow to starboard; the other diagonal turns it to port

All four pontoons sit at the same half-separation, so the four transverse arms share one magnitude, $d = 0.395$ m, and differ only in sign. Row 3 is then the eight numbers the code holds:

$$[d_1 \quad l_f \quad d_2 \quad l_f \quad d_3 \quad l_a \quad d_4 \quad l_a] = [0.395 \quad 1.500 \quad -0.395 \quad 1.500 \quad -0.395 \quad 1.000 \quad 0.395 \quad 1.000]\ \mathrm{m}$$

Symbol	Quantity	Value · source
x_i	longitudinal station of unit i	$+1.50, +1.50, -1.00, -1.00$ m — `otter_config.m`, field `pos`
y_i	transverse station of unit i	$-0.395, +0.395, -0.395, +0.395$ m — same
d_i	transverse moment arm of unit i, $-s_i y_i$	$+0.395, -0.395, -0.395, +0.395$ m — the signed form of y_i
s_i	mounting sign: which way the zero points	$+1, +1, -1, -1$ — `otter_config.m`, field `mount`
δ_i	tilt angle, from unit i's own zero towards y_b	commanded, clipped to $\pm 45°$
T_i	thrust magnitude of unit i	$k_{pos} n_i \lvert n_i \rvert$ for $n_i \geq 0$, $k_{neg} n_i \lvert n_i \rvert$ otherwise
$\mathbf{b}_i^c, \mathbf{b}_i^s$	the two columns unit i owns	the $\cos$ column and the $\sin$ column
B	allocation matrix	3×8, rank 3 — written out in `otter_tilt4.m` and checked against `_tools/otter_B.m`
l_f, l_a	longitudinal moment arms, CO to the forward and aft stations	1.50 m, 1.00 m — `otter_config.m`, fields `l_f`, `l_a`

Derivation, one line at a time.

1. The force–moment triple produced by a planar force $\mathbf{g} = [\,g_x \;\; g_y\,]^\mathsf{T}$ applied at (x, y) in body axes is $[\,g_x,\; g_y,\; x g_y - y g_x\,]^\mathsf{T}$. This is Week 3's column rule, and it is the only definition of B this course uses.
2. For a **fixed** thruster, $\mathbf{e}_i$ is a constant of the hull. The single unknown is T_i, the rule returns one column, and four fixed thrusters give $B \in \mathbb{R}^{3\times 4}$.

3. For a **tilting** thruster, $\mathbf{e}_i = \mathbf{e}_i(\delta_i)$. The map $(T_i, \delta_i) \mapsto T_i\mathbf{e}_i$ is polar-to-Cartesian, invertible wherever $T_i > 0$, so the unit carries two independent numbers rather than one.
4. Applying line 1 to the two components of $s_i T_i[\cos\delta_i \quad \sin\delta_i]^\mathsf{T}$ separately gives the columns $s_i[1, 0, -y_i]^\mathsf{T}$ and $s_i[0, 1, x_i]^\mathsf{T}$. Four units give $B \in \mathbb{R}^{3\times 8}$, written above with its third row in symbols.
5. B has rank 3: rows 1 and 2 are independent by inspection, and row 3 is not a combination of them because the x_i and y_i are not all zero. Its null space has dimension $8 - 3 = 5$.
6. Five null-space dimensions are five independent ways to change the eight numbers without changing the force. Choosing among them is allocation — the minimum-norm choice is $\mathbf{f} = B^\dagger\boldsymbol{\tau}$ with $B^\dagger = B^\mathsf{T}(BB^\mathsf{T})^{-1}$ — and it is next stage's subject, not this one's.
7. Multiplying out $B\mathbf{f}$ gives the scalar form the motion model evaluates:

$$X = \sum_{i=1}^{4} s_i T_i \cos\delta_i, \qquad Y = \sum_{i=1}^{4} s_i T_i \sin\delta_i, \qquad N = \sum_{i=1}^{4} s_i\left(x_i T_i \sin\delta_i - y_i T_i \cos\delta_i\right)$$

8. Writing $f_{i,x} = T_i\cos\delta_i$ and $f_{i,y} = T_i\sin\delta_i$ and substituting the four stations turns the moment sum into eight separate terms, each one a force times its own arm:

$$N \;=\; \left(d_1 f_{1,x} + d_2 f_{2,x} + d_3 f_{3,x} + d_4 f_{4,x}\right) \;+\; l_f\left(f_{1,y} + f_{2,y}\right) \;+\; l_a\left(f_{3,y} + f_{4,y}\right)$$

Every sum is a plain sum, because each d_i, l_f and l_a already carries its own sign: the alternation of the diagonal lives in $d_2 = d_3 = -0.395$ m, not in the operators. The first bracket collects the transverse arms, the other two the longitudinal ones. This is the form to read when a commanded manoeuvre comes out wrong: each term names one thruster, one component of its thrust, and the one length that turns it into yaw.

9. Two consequences follow at $\delta = 0$, and Experiment 7-1 measures both. Every sine dies, so $Y = 0$ and $N = -\sum s_i y_i T_i$: the differential-thrust yaw of `otter.m`, with four stations in place of two. And because $s_3 = s_4 = -1$, the aft pair alone reproduces `otter.m` with $\boldsymbol{\tau}$ reversed.

Experiment 7-1 · The new model contains the old one (15 min)

What it measures. Lines 1, 7 and 9 of the derivation: that the column rule and the scalar form agree, and that $\delta = 0$ returns `otter.m` — forward with $\boldsymbol{\tau}$ as it stands, aft with $\boldsymbol{\tau}$ reversed.

The model. None. `verify_otter_tilt4` calls `otter_tilt4` and `otter` directly, over 200 random states, so the check is on the equations and not on any wiring.

Opening and running.

```
verify_otter_tilt4          % 네 가지 항등식 / four identities
```

Expected output:

```
verify_otter_tilt4
  stations   x = [+1.50 +1.50 -1.00 -1.00] m
             y = [-0.395 +0.395 -0.395 +0.395] m
  tilt limit +-45 deg each
  B          3 x 8, rank 3, null space 5

(a) aft mounting sign flips tau exactly :  3.55e-15
(b) fore pair, delta 0 == otter.m       :  0.00e+00
(c) tau                == otter_B * f   :  7.19e-14
(d) saturation clamps n AND delta       :  0.00e+00

thrust at +-60 rad/s   +39.888 N ahead, -23.202 N astern  (k_neg/k_pos = 0.5817)
cancelling astern speed   -78.670 rad/s   (not -60)
limits                   -101.737 .. +103.931 rad/s

PASS  (tol 1e-09, 200 random states)
```

Reading the output against the derivation.

Where to look	What is there	Which line predicts it
`(b)`	exactly zero, not merely small	line 9 — at $\delta = 0$ the forward pair evaluates the arithmetic of `otter.m`, in the same order
`(a)`	3.55×10^{-15}, one rounding step	line 9 — $s_3 = s_4 = -1$, so the aft pair gives `otter.m` with every component of $\boldsymbol{\tau}$ reversed
`(c)`	7.19×10^{-14} over 200 random states	line 7 — the scalar form and $B\mathbf{f}$ are one expression in two floating-point orders
`(d)`	zero	the model clips **both** n and δ, so a command outside the envelope is received at the envelope
`rank 3, null space 5`	printed, not asserted	lines 5 and 6

What the output says

- The four-unit model is a strict extension: nothing in it changes what the first eleven weeks measured.

What to try	What to watch
`verify_otter_tilt4` twice in a row	only `(c)` changes, in its last digits — the random states feed the $B\mathbf{f}$ comparison alone

> ✎ **In class**
>
> - **Purpose** — fix two ideas: a tilting unit is two unknowns, and the mounting sign lives in B.
> - **Point to** — the third row of B: each pair contributes $s_i(-y_i)$ then $s_i(+x_i)$, and that is the whole of the column rule.
> - **Ask** — "Eight unknowns and three equations. How many solutions?" Five dimensions of them, which is why an allocator is a choice and not a calculation.
> - **Take away** — the forward pair at $\delta = 0$ **is** `otter`, to the last bit; the aft pair is `otter` running backwards.

7-2. What had to change inside the motion model

What is observed. `_tools/otter_tilt4.m` is MSS `otter.m` copied and changed in four places, each marked in the file with `% ---- CHANGED (i of 4)`. The four marks sit in the same four places as Week 6's `otter_bow.m`.

- the rigid-body and added-mass matrices, the Coriolis terms, the restoring forces, the linear and cross-flow damping, the nonlinear yaw damping $N_h = N_r(1 + 10|r|)r$ and the kinematics are character-for-character identical
- the hull is the same hull, and the file has to show that

The result. Four changes, and nothing else.

	`otter.m`	`otter_tilt4.m`	Why
1 · input check	`length(n) ~= 2`	`length(n) ~= 4` and `length(delta) ~= 4`, plus an eighth argument `quad`	the configuration arrives as data, so the stations and tilt limits are not hard-coded in the plant
2 · thruster data	l_1, l_2	`px`, `py`, `d_max`, `s` read out of `quad`, then the six arms named and B written out	with four stations a pair of signed arms is not enough, and the mounting sign has to reach the columns
3 · thrust	a two-pass loop, n saturated	no loop: n **and** δ clipped one line each, then $T = k\,n\|n\|$	derivation line 3 below — the servo is part of the vessel, not of the controller
4 · the force vector	`tau = [T1+T2 0 0 0 0 -l1*T1-l2*T2]'`	`tau3 = B_thr * f`	derivation lines 4 to 6 — one matrix product in place of a literal

Derivation, one line at a time.

1. The propellers are unchanged. $k_{pos} = 0.02216/2$ and $k_{neg} = 0.01289/2$ stay at the values `otter.m` sets, because the same propeller is fitted four times rather than twice.
2. The shaft limits are therefore also unchanged: $n_{max} = +103.931$ rad/s and $n_{min} = -101.737$ rad/s, derived in `otter.m` from the maximum thrust each way.
3. The tilt servo belongs inside the plant. A servo that refuses $70°$ refuses it whatever asked, so the clip must be in the motion model; a clip placed only in the controller would let a mis-wired command produce a force the vessel cannot make.
4. `otter.m` writes its $\boldsymbol{\tau}$ as a literal six-row vector because two fixed thrusters reach only two of the six rows. Here every unit reaches rows 1, 2 and 6 at once, so the literal is replaced by the product $\boldsymbol{\tau}_3 = B\mathbf{f}$ of §7-1.
5. The split is deliberate. B is built once, from geometry and mounting alone, and holds every sign; $\mathbf{f}$ is rebuilt each step and holds only T_i and δ_i. Nothing in the force code can disagree with the figure, because the figure is B.
6. Rows 3, 4 and 5 stay zero. No thruster on this vessel acts in heave, roll or pitch, and the trim moment that does act in pitch is applied elsewhere in the file, exactly as in `otter.m`.

The second change, which is the column rule and the mounting sign written as code:

```
% ---- CHANGED (2 of 4) ---------------------------------------------------
px     = quad(1,:);                    % x stations (m)
py     = quad(2,:);                    % y stations (m)
d_max  = quad(3,:);                    % tilt limits, both ways (rad)
s      = quad(4,:);                    % mounting signs, +1 forward, -1 aft

% 3 행의 여섯 칸은 전부 **모멘트 팔**이다. 열 규칙(3주차)에 자리와 거치 부호를
% 넣으면 cos 열은 -s_i y_i, sin 열은 s_i x_i 이므로, 길이에 이름을 붙여 둔다.
% Row 3 is nothing but moment arms. Putting the station and the mounting sign
% into the Week 3 column rule gives -s_i y_i for a cos column and s_i x_i for a
% sin column, so each of those lengths is given a name here.
l_f = s(1)*px(1);                      % +1.500 m  forward pair, along the hull
l_a = s(3)*px(3);                      % +1.000 m  aft pair  (s = -1, x = -1 m)
d1  = -s(1)*py(1);                     % +0.395 m  unit 1, across the beam
d2  = -s(2)*py(2);                     % -0.395 m  unit 2, the other side
d3  = -s(3)*py(3);                     % -0.395 m  unit 3
d4  = -s(4)*py(4);                     % +0.395 m  unit 4

% 유효행렬을 **여기서** 쓴다 — 강의 §7-1 의 행렬 그대로다. 이름이 B_thr 인
% 것은 otter.m 의 B 가 이미 **선폭**이라서다 — 그 줄은 고치지 않는다.
% 1·2 행은 거치 부호만, 3 행은 위에서 이름 붙인 팔만 들고 있다.
% The effectiveness matrix, written out here exactly as the matrix of 7-1. It
% is named B_thr because B is already the beam in otter.m, and that line is not
% touched. Rows 1 and 2 hold only the mounting sign; row 3 holds only the arms
% named above.
B_thr = [ s(1)   0    s(2)   0    s(3)   0    s(4)   0
           0    s(1)   0    s(2)   0    s(3)   0    s(4)
          d1    l_f   d2    l_f   d3    l_a   d4    l_a ];
% -------------------------------------------------------------------------
```

The third change, which puts both hardware limits in front of the thrust law:

```
% ---- CHANGED (3 of 4) ---------------------------------------------------
n     = min(max(n(:),      n_min),      n_max);        % shaft speed (rad/s)
delta = min(max(delta(:), -d_max(:)),   d_max(:));     % tilt servo  (rad)

% 추력은 n|n| 이고, 계수가 전진과 후진에서 다르다 — 부호가 계수를 고른다.
% The thrust is k n|n|, and k differs ahead and astern, so the sign of n picks
% which coefficient applies.
k_thr  = k_pos * (n > 0) + k_neg * (n <= 0);
Thrust = k_thr .* n .* abs(n);         % 4 x 1, N
% -------------------------------------------------------------------------
```

The fourth change, which is the matrix product itself:

```
% ---- CHANGED (4 of 4) ------------------------------------------------
% Control forces and moments — written as a matrix product
% otter.m:  tau = [T1+T2  0  0 0 0  -l1*T1 - l2*T2]'      <- sway is zero
% here:     tau_3 = B f.  Each tilting thruster contributes TWO columns of B
%           and two entries of f, so the 3-by-8 B is constant and the whole
%           dependence on the commands sits in f.
%
%     f = [ T_1 cos d_1 ; T_1 sin d_1 ; ... ; T_4 cos d_4 ; T_4 sin d_4 ]
%
% 쌍마다 0 도가 다른 것은 B_thr 안에 있다 — 위에서 거치 부호 s = [1 1 -1 -1]
% 을 열에 실어 두었다. 그래서 여기에는 부호가 하나도 없다. 한 줄이 한 열에
% 대응하므로, 어느 줄이 어느 열과 곱해지는지 세지 않고 읽을 수 있다.
% The per-pair zero lives inside B_thr: the mounting sign s = [1 1 -1 -1] was
% carried into its columns above, so no sign appears here. One line per column,
% so which entry meets which column is read off rather than counted.
f = [ Thrust(1)*cos(delta(1))
      Thrust(1)*sin(delta(1))
      Thrust(2)*cos(delta(2))
      Thrust(2)*sin(delta(2))
      Thrust(3)*cos(delta(3))
      Thrust(3)*sin(delta(3))
      Thrust(4)*cos(delta(4))
      Thrust(4)*sin(delta(4)) ];

tau3 = B_thr * f;                       % [X ; Y ; N]

tau = [tau3(1)
       tau3(2)
       0
       0
       0
       tau3(3)];
% --------------------------------------------------------------------------
```

In the code	In the derivation	Note
`d1 = -s(1)*py(1)` and its three mates	$d_i = -s_i y_i$	the transverse arm of §7-1, sign included
`l_f = s(1)*px(1)` , `l_a = s(3)*px(3)`	$l_f = s_1 x_1$, $l_a = s_3 x_3$	the longitudinal arms — one name each, both positive
rows 1 and 2 of `B_thr` , holding only `s(i)`	$\mathbf{b}_i^c$ and $\mathbf{b}_i^s$, first two entries	the only place the mounting sign acts on a force
row 3 of `B_thr` , holding only the six names	row 3 of the matrix of §7-1	the two column kinds alternate: arm, station, arm, station
the eight lines of `f`	$\mathbf{f}$ of §7-1	the cosine and the sine of each unit, interleaved in that order
`tau3 = B_thr * f`	$\boldsymbol{\tau} = B\mathbf{f}$	no sign and no station appears here: both are already in B
rows 3, 4, 5 left at zero	line 6	heave, roll and pitch carry no thruster

Change 3 keeps `otter.m` 's shaft-speed limit and its k_{pos} / k_{neg} split, and adds the servo limit — which §7-4 measures as the third asymmetry. The branch that `otter.m` writes as an `if` inside a loop is written here as the mask `k_pos*(n > 0) + k_neg*(n <= 0)` , which is the same two coefficients chosen by the same test.

> **The sway row is no longer structurally zero**
>
> Week 6 made the same point about `otter_bow.m` . A motion model whose τ_y is identically zero describes an under-actuated vessel, however a sway force is added to it afterwards. The four-unit model earns a non-zero second row from the geometry itself, which is why the eight numbers of §7-3 can be given directly to the plant with no allocator between.

7-3. The force eight numbers make

What is observed. Experiment 7-3 commands eight sets of eight numbers and compares two independent computations of $\boldsymbol{\tau}$: the product $B\mathbf{f}$ of §7-1, and the force measured back out of `otter_tilt4` from the acceleration it produces. The worst disagreement over the eight scenarios is 2.8×10^{-14} N or N·m.

- the eight scenarios isolate one effect each, and are reused by §7-4, §7-5 and §7-6
- S8 asks for $70°$, and the table records what the model received

The result. The model is driven open loop: eight constants on the canvas, a saturation stage, the plant, and the log.

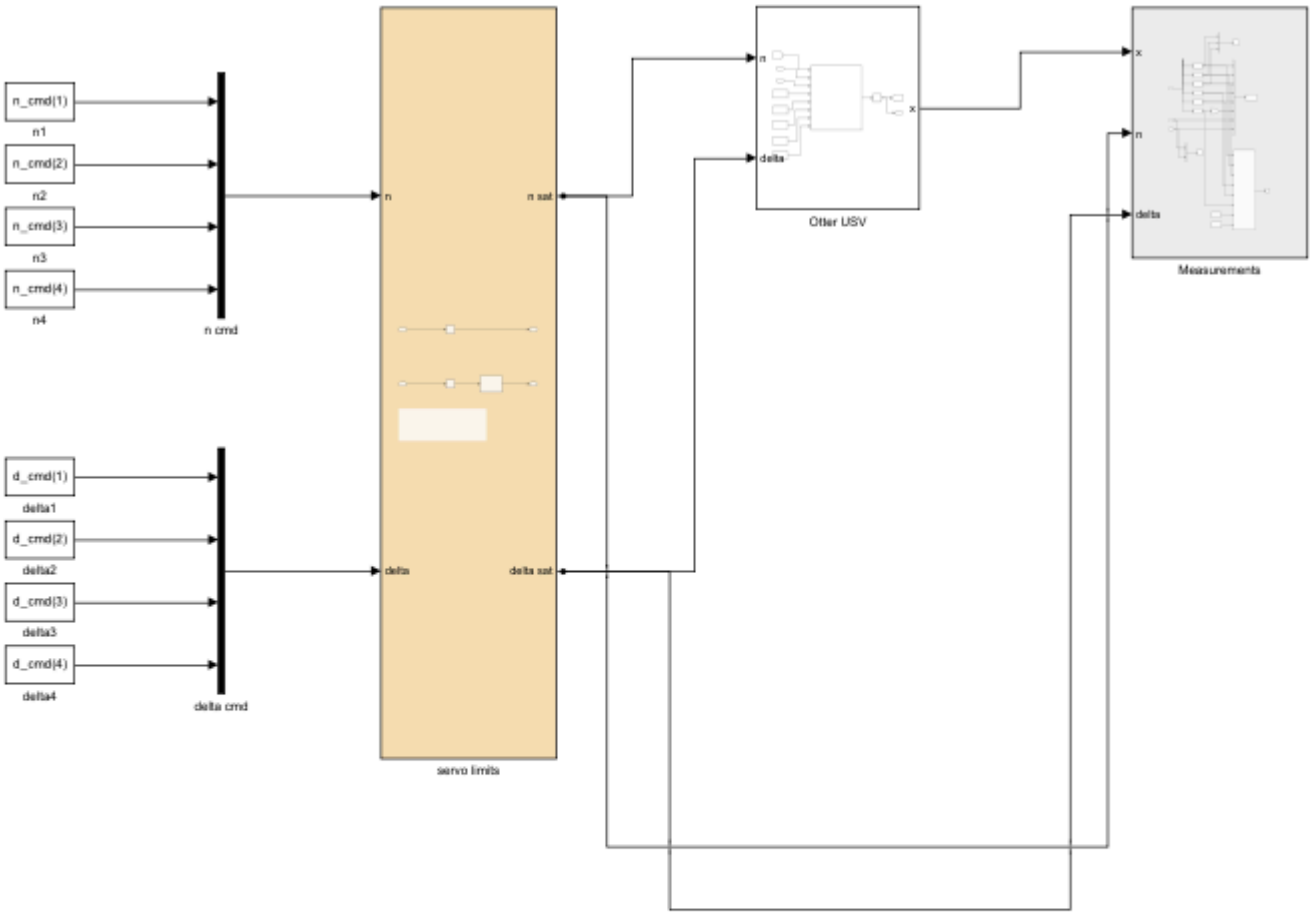
$$\boldsymbol{\tau}_{\text{measured}} \;=\; M\left(\dot{\mathbf{x}}_{\text{on}} - \dot{\mathbf{x}}_{\text{off}}\right)_{1:6}$$

Symbol	Quantity	Value · source
M	6×6 mass matrix, rigid body plus added mass	from `otter_tilt4` at the same state
$\dot{\mathbf{x}}_{\text{on}}$	acceleration with the commanded (n, δ)	`otter_tilt4(x, n, delta, …)`
$\dot{\mathbf{x}}_{\text{off}}$	acceleration with $n = 0$	the same call with the thrust removed
residual	$\max\lvert B\mathbf{f} - [X\ Y\ N]^{\mathsf{T}}\rvert$	asserted below 10^{-12} inside the script

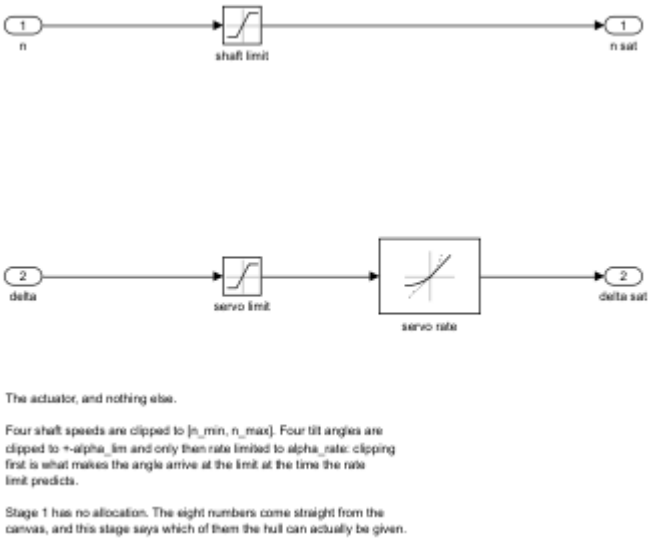
Derivation, one line at a time.

1. The six-row acceleration of the vessel is $\dot{\boldsymbol{\nu}} = M^{-1}(\boldsymbol{\tau} - \mathbf{d}(\boldsymbol{\nu}, \mathbf{x}))$, where $\mathbf{d}$ collects damping, Coriolis and restoring terms. Those terms depend on the state but not on the thrust.

2. Evaluating the same state twice, once with the thrust and once with $\boldsymbol{n = 0}$, therefore differences $\mathbf{d}$ away: $\dot{\boldsymbol{\nu}}_{\text{on}} - \dot{\boldsymbol{\nu}}_{\text{off}} = M^{-1}\boldsymbol{\tau}$.
3. Multiplying back by M recovers $\boldsymbol{\tau}$ exactly, not approximately, because acceleration is affine in the thrust. This is the same argument Week 6 used to show its superposition had been exact.
4. The comparison is run **at rest**, $\mathbf{x} = \mathbf{0}$, so that no state-dependent term can hide a sign error by cancelling it.
5. The tilt command is clipped before it reaches the column rule, so the table prints the clipped angle. A case that asks for more than $45°$ must therefore produce the row of the case that asked for exactly $45°$.



Eight Constant blocks carry the eight numbers, two Mux bars gather them into vectors, the tan subsystem applies the physical envelope, the green block is `otter_tilt4`, and the grey block logs `[u v r N E psi]` followed by the post-servo (n, δ) tapped from the branches below.



The upper lane clips the shaft speed to $[-101.737, +103.931]$ rad/s. The lower lane clips the angle to $\pm 45°$ and only then rate-limits it to $90°$/s. The two lanes never meet, because shaft speed and tilt angle are independent commands.

> **★ The angle is clipped before it is rate-limited, not after**
>
> Rate-limiting first would let the limiter chase a $70°$ target through the legal range and sit at $45°$ only on arrival, so the recorded angle would depend on how long the run lasted. Clipping first makes the target legal, and the rate limiter then describes only how fast the servo gets there.

Experiment 7-3 · Eight commands, and the force each one makes (20 min)

What it measures. Lines 3 and 5: that the force measured out of the plant is the force the column rule predicts, and that the servo clip is visible in the table.

The model. `W07_openloop.slx`. Between runs only the eight constants change; the wiring does not.

Opening and running. The runner takes a selection, so one scenario can be re-examined without redrawing the other seven.

```
W07_0_setup
open_system('W07_openloop')    % Run — 궤적과 상태량이 뜬다 / the track appears
W07_V_openloop                 % 여덟 전부 / all eight, and the three tables
W07_V_openloop('S5')           % 하나만 / one of them, its own sheet redrawn
W07_V_openloop({'S5','S6'})    % 둘을 견준다 / two, to be compared
```

Expected output, the first of three tables printed by that script:

```
(1) hand formula against the motion model, at rest

    tau from the hand formula    X = sum si Ti cos di
                                 Y = sum si Ti sin di
                                 N = sum si (xi sin di - yi cos di) Ti
                                 s = [+1 +1 -1 -1], the mounting signs
    tau measured out of otter_tilt4's acceleration, same state

  case  n [rad/s] after clip       delta [deg] clipped       X [N]    Y [N]  N [N m]    resid
  ------------------------------------------------------------------------------------------
  S1    [ 60.0  60.0 -60.0 -60.0]  [   0    0    0    0]   126.180    0.000    0.000  0.0e+00
  S2    [-60.0 -60.0  60.0  60.0]  [   0    0    0    0]  -126.180   -0.000    0.000  1.4e-14
  S3    [ 60.0  60.0  60.0  60.0]  [  45   45   45   45]     0.000    0.000  141.025  2.8e-14
  S4    [ 60.0  60.0  60.0  60.0]  [ -45  -45  -45  -45]     0.000    0.000 -141.025  2.8e-14
  S5    [ 60.0  60.0  60.0  60.0]  [  45   45  -45  -45]     0.000  112.820   28.205  1.4e-14
  S6    [ 60.0  60.0  73.5  73.5]  [  45   45  -45  -45]   -28.205  141.025    0.000  2.2e-14
  S7    [ 60.0  60.0 -60.0 -60.0]  [  45   45    0    0]   102.814   56.410   84.615  2.8e-14
  S8    [ 60.0  60.0  60.0  60.0]  [  45   45  -45  -45]     0.000  112.820   28.205  1.4e-14
  ------------------------------------------------------------------------------------------
  worst residual 2.8e-14 N or N m over the 8 scenarios
```

Reading the table against the derivation.

Where to look	What is there	Which line predicts it
the `resid` column	at most 2.8×10^{-14}	line 3 — the two computations are the same expression, so only rounding separates them
S1, $X = 126.180$ N	$2(39.888 + 23.202)$	§7-1 line 9 at $\delta = 0$: the bow pair ahead, and the stern pair ahead too, because $s_i = -1$ reverses a negative shaft speed
S2, $X = -126.180$ N	the same magnitude, mirrored	the mounting sign swaps which pair runs astern, so the asymmetric thrust curve cancels out of the total
S3, $X = Y = 0$ with $N = 141.025$ N·m	$2(l_f + l_a)T \sin 45°$	§7-1 line 7 — the four force vectors sum to zero and only their moments survive
S5, $N = 28.205$ N·m although a pure sway was asked	$2(l_f - l_a)T \sin 45°$	§7-4 — the stations are not symmetric about CO
S6, the commanded 73.5 rad/s	$60\sqrt{1.5}$	§7-4 — the stern speed that cancels that moment, at the price of -28.205 N of surge
S8 identical to S5	the `delta` column reads 45, although 70 was asked	line 5, and §7-2

What the table says

- The eight numbers reach the hull as the column rule says they will, and nothing in the wiring alters them.

What to try	What to watch
`W07_V_openloop('S1')`	the S1 sheet alone: surge only, and the vessel of Weeks 1 to 6 is back
`W07_V_openloop({'S5','S6'})`	two sheets and a two-curve overlay: the leftover moment, and the same crab with that moment cancelled

> ✎ **In class**
>
> - **Purpose** — separate "the equations are right" from "the vessel does what was wanted".
> - **Point to** — the S1 and S2 rows. Both reach 126.180 N although the thrust curve is far from symmetric, because the mounting sign decides which pair runs astern.
> - **Ask** — "S8 asked for $70°$. Which column shows that it did not get it?" The `delta` column, which prints what the plant received.
> - **Take away** — a residual of 10^{-14} says the arithmetic is right; it says nothing about whether the force was the right force.

7-4. Three asymmetries that break the symmetric guess

What is observed. Three asymmetries run through the eight rows of table (1), and each one is a property of the vessel rather than of the command.

- the thrust curve: $+60$ rad/s gives $+39.888$ N, -60 rad/s gives only -23.202 N
- the stations: $l_f = 1.50$ m forward against $l_a = 1.00$ m aft
- the servo: $70°$ asked, $45°$ given

The result. Each has a closed form, and each predicts a number in the table.

$$\underbrace{n_{rev} = -n\sqrt{\frac{k_{pos}}{k_{neg}}} = -78.670\ \text{rad/s},}_{\text{thrust curve}} \qquad \underbrace{N = \bar{x}\,Y, \quad \bar{x} = \frac{l_f - l_a}{2} = 0.25\ \text{m}}_{\text{stations}}$$

Symbol	Quantity	Value · source
k_{pos}	thrust coefficient, ahead	$0.02216/2 = 0.011080$ — `otter.m`, unchanged
k_{neg}	thrust coefficient, astern	$0.01289/2 = 0.006445$ — same
k_{neg}/k_{pos}	the asymmetry ratio	0.5817 — printed by `W07_0_setup`
n_{rev}	astern speed matching $+60$ rad/s	-78.670 rad/s — printed by `verify_otter_tilt4`
$\bar{x}$	longitudinal centroid of the four stations	0.25 m
N	residual yaw of a four-unit symmetric crab	$+28.205$ N·m — table (1), row S5

Derivation, one line at a time.

1. The thrust of one propeller is the signed square $T = kn|n|$, with $k = k_{pos}$ for $n \geq 0$ and $k = k_{neg}$ for $n < 0$. Both coefficients are positive, and $k_{neg} < k_{pos}$ because a propeller designed to push ahead is less efficient in reverse.
2. Running one unit at $+n$ and another at $-n$ therefore does not cancel: $T(+n) + T(-n) = (k_{pos} - k_{neg})n^2$.
3. Row S1 runs the bow pair at $+60$ rad/s and the stern pair at -60. Each stern unit returns -23.202 N, which its mounting sign $s_i = -1$ turns into $+23.202$ N of ahead force, so the four push ahead in unequal shares — 39.888 N forward against 23.202 N aft, a ratio of 1.72. Row S2 swaps which pair runs astern, so $|X|$ is the same

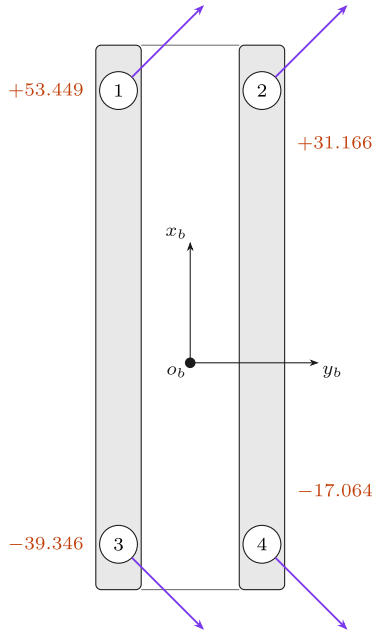
126.180 N in both. The thrust curve does not break the fore–aft symmetry of the total; it breaks **the split between the pairs**, and that split decides each pair's moment as soon as the units tilt.

4. Setting $k_{pos}n^2 + k_{neg}n_{rev}|n_{rev}| = 0$ with $n_{rev} < 0$ gives the astern speed at which a unit matches a forward one: $n_{rev} = -n\sqrt{k_{pos}/k_{neg}} = -60 \times 1.3112 = -78.670$ rad/s. `verify_otter_tilt4` prints it as `cancelling astern speed`.
5. The second asymmetry needs no thrust curve at all. Take row S5 — the bow pair at $\delta = +45°$, the stern pair at $-45°$, all four at the same thrust T. With the mounting signs every unit makes the **same** sway force $f_y = T\sin 45° = 28.205$ N, while the surge forces alternate: $+f_x$ forward, $-f_x$ aft. Then

$$N = \sum_i (x_i f_y - y_i f_{ix}) = f_y \sum_i x_i - \sum_i y_i f_{ix}$$

6. The second sum is zero: inside each pair the two units sit at $\pm y$ with the same f_{ix}, so their two contributions cancel. The first sum is not zero: $\sum x_i = 2l_f - 2l_a = 1.0$ m, so $N = 2(l_f - l_a)f_y$.
7. Dividing by $Y = 4f_y$ puts it in the form that explains itself: $N = \bar{x}Y$ with $\bar{x} = \frac{1}{4}\sum x_i = (l_f - l_a)/2 = 0.25$ m. Four equal sway forces are one sway force acting at the centroid of the four stations, and that centroid is 0.25 m ahead of o_b. With $Y = 112.820$ N this is $N = 112.820 \times 0.25 = 28.205$ N·m, the S5 and S8 reading.

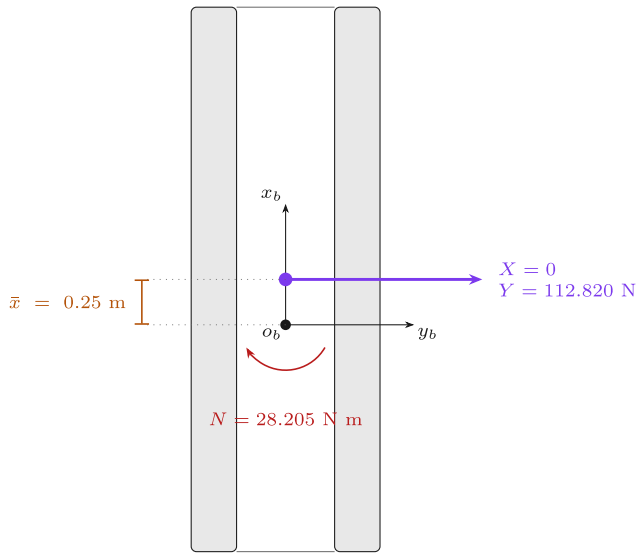
(a) four equal sway forces, four unequal arms



Every unit is given $n_i = 60$ rad/s, the bow pair at $\delta_i = +45°$ and the stern pair at $-45°$. With s_i the four *sway* forces come out equal, $f_{iy} = 28.205$ N, and the surge forces cancel fore against aft.

$x_i f_{iy} - y_i f_{ix}$ [N m] is printed beside each unit. The f_{ix} terms cancel inside each pair, at $\pm y$, and what is left is $N = f_y \sum x_i = 2\,(l_f - l_a)\,f_y$

(b) the same four forces, moved to one point



Four equal sway forces are one sway force at the centroid of the four stations:

$$\bar{x} = \tfrac{1}{4}\textstyle\sum x_i = \dfrac{l_f - l_a}{2} = 0.25 \text{ m}$$

That point is not o_b, so a sway command given symmetrically to four thrusters pulls 0.25 m ahead of the origin and yaws: $N = Y\,\bar{x} = 112.820 \times 0.25 = 28.205$ N m

8. The condition that removes it follows from the same sum with the bow pair at T_f and the stern pair at T_a, all at $|\delta| = 45°$: $\sum x_i T_i = 3.0T_f - 2.0T_a = 0$, hence $T_a = 1.5T_f$. With $T_f = 39.888$ N this needs $T_a = 59.832$ N, that is $n_a = \sqrt{59.832/k_{pos}} = 60\sqrt{1.5} = 73.485$ rad/s. Row S6 runs it and reads $N = 0.000$ N·m — at the price of $X = 2\cos 45°\,(T_f - T_a) = -28.205$ N. The moment did not vanish; it became a surge of the same number.
9. The third asymmetry is the servo, and it is not a sign or a ratio but a limit. δ is clipped before the column rule sees it, so the force is computed from $45°$ whatever was asked. A command of $70°$ and a command of $45°$ are the same command to this vessel.

10. All three together are the argument for an allocator. The eight numbers a person would guess from symmetry are not the eight numbers that produce the wanted force, and solving for them by hand — as line 8 did, for one condition — does not scale to three conditions and a changing state.

> **⚠ No equal-thrust sway inside $\pm 45°$ is free of yaw**
>
> A symmetric crab leaves 28.205 N·m, and no common shaft speed removes it, because the asymmetry is in the geometry, not in the command. Line 8 buys the moment back with a surge of exactly -28.205 N. Unequal thrusts can do better — Assignment requirement 2 asks for such a set — but four equal ones never can.

Experiment 7-4 · The three asymmetries, measured (20 min)

What it measures. Lines 3, 4, 7, 8 and 9, each against a row of table (1).

The model. The same `W07_openloop.slx`. `W07_tab_track` prints the three asymmetries below table (3), from the same eight runs.

Opening and running.

```
W07_V_openloop                % 여덟 줄 전부 / all eight rows
W07_V_openloop({'S5','S6'})   % 두 번째 비대칭만 / the second asymmetry alone
```

Expected output, the block printed below table (3):

```
      the three asymmetries this week turns on
        thrust curve   +60 rad/s gives +39.888 N, -60 rad/s gives -23.202 N, so
                       the pair running forward carries 1.72 times the pair
                       running astern -- and yet S1 and S2 reach the SAME
                       +126.180 N, because the mirrored mounting swaps which
                       pair is the one running astern
        the stations   l_f = 1.50 m forward, l_a = 1.00 m aft, so S5 crabs with
                        +28.205 N m of yaw left over, and S6 cancels that moment
                       at n = 73.485 rad/s only by letting  -28.205 N of surge
                       appear: no EQUAL-thrust sway inside +-45 deg is yaw-free
        the servo      S8 asks for 70 deg and is given 45 deg, so its row of
                       table (1) is S5's row exactly
```

Reading the output against the derivation.

Where to look	What is there	Which line predicts it
`1.72 times`	the $39.888/23.202$ split inside S1	line 3 — k_{pos}/k_{neg}
`the SAME +126.180 N`	the S1 and S2 X columns	line 3 — the mounting signs swap which pair runs astern
`+28.205 N m of yaw`	the S5 and S8 N columns	line 7 — $\bar{x}Y$ with $\bar{x} = 0.25$ m
`-28.205 N of surge`	the S6 X column, with $N = 0.000$	line 8 — the moment traded for a surge
`given 45 deg`	the S8 `delta` column	line 9

What to try	What to watch
`W07_V_openloop('S6')` alone	$N = 0.000$ N·m against S5's $+28.205$, and X going from 0 to -28.205 N
`d_deg = [90 90 -90 -90]'` in `W07_0_setup`, expecting a pure sway	the table returns S5's row. The servo refuses the extra $45°$ — line 9, not a sway

> ✎ **In class**
>
> - **Purpose** — show that the obstacle to commanding this vessel is the vessel, not the arithmetic.
> - **Point to** — cell (b) of the figure: one arrow, offset 0.25 m, and the yaw follows from the offset alone.
> - **Ask** — "Which of the three would disappear if the hull were symmetric fore and aft?" Only the second. The thrust curve and the servo belong to the hardware.
> - **Take away** — $\bar{x} = (l_f - l_a)/2 = 0.25$ m, and $N = \bar{x}Y$.

7-5. The track those forces produce

What is observed. Experiment 7-5 runs each of the eight cases for 30 s from rest and reads four quantities at the end. S1 runs straight at 1.627 m/s; S5 crabs at $74.28°$ while turning; S8 lies exactly on S5.

- the readings are taken at $t = 30$ s, by when all eight have settled
- the crab angle is $\beta = \operatorname{atan2}(v, u)$ and the speed is $U = \sqrt{u^2 + v^2}$
- turn radius is $U/|r|$, reported as `straight` when $|r| < 10^{-4}$ rad/s

The result. A force held constant in body axes produces a steady body-frame velocity, and a steady velocity with a steady yaw rate is a circle.

$$U = \sqrt{u^2 + v^2}, \qquad \beta = \operatorname{atan2}(v, u), \qquad R = \frac{U}{|r|}$$

Symbol	Quantity	Value · source
U	total speed over the water	read at $t = 30$ s
β	crab angle, body x to the velocity	positive to starboard
R	radius of the steady turn	$U/\lvert r\rvert$; infinite when $r = 0$
T_{final}	run length for these readings	30 s, set inside `W07_tab_reint`

Derivation, one line at a time.

1. With (n, δ) held constant, $\boldsymbol{\tau}$ is constant in body axes, so the body-frame equation $M\dot{\boldsymbol{\nu}} + \mathbf{d}(\boldsymbol{\nu}) = \boldsymbol{\tau}$ is autonomous and settles on an equilibrium $\boldsymbol{\nu}^\star$ where damping balances thrust.
2. That equilibrium is reached in a few seconds for surge and sway, and in rather longer for yaw, because the nonlinear yaw damping $N_h = N_r(1 + 10|r|)r$ stiffens as r grows.
3. At equilibrium the body-frame velocity is fixed but the heading is not: $\dot{\psi} = r \neq 0$ rotates the velocity in the Earth frame, which traces a circle of radius $U/|r|$.
4. The crab angle β is a property of that equilibrium, not of the command. Row S7's force stands at $\operatorname{atan2}(56.410, 102.814) = 28.74°$ in body axes, yet the hull crabs at only $5.89°$, because sway damping on this hull is far larger than surge damping.
5. A case with $r = 0$ runs straight in the direction β, and three rows do: S1 at $\beta = 0$, S2 at $180°$, and S6 at $114.0°$ — a heading held while the hull travels sideways and slightly astern.
6. S8 must trace S5's circle exactly, since §7-4 line 9 showed the two commands reach the plant identically. The overlay draws S8 dashed so the coincidence is visible rather than hidden by overdrawing.

Experiment 7-5 · Eight tracks from eight commands (20 min)

What it measures. Lines 3 to 6: that each command settles, that the settled state is a circle or a straight line, and that S8 and S5 coincide.

The model. The same `W07_openloop.slx`, run eight times at $T_{final} = 30$ s. Each run gets **its own sheet** — `img/W07_result_S1.png` … `_S8.png` — because eight curves in one axes erase each other and the question of this

section is whether each scenario did what was written down for it.

Opening and running.

```
W07_V_openloop                % 여덟 장 전부 / all eight sheets
W07_V_openloop('S5')          % 한 장만 다시 / that one sheet again
```

Expected output, the third table:

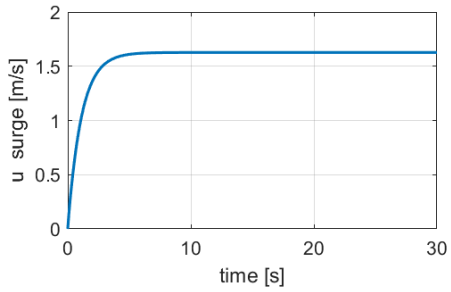
```
  (3) the motion those forces produce, scenario by scenario

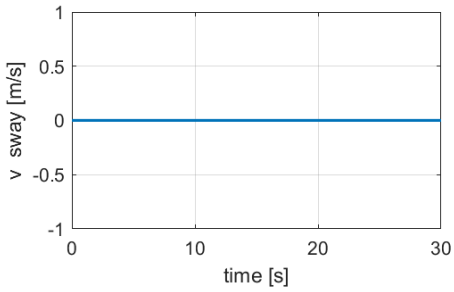
     every value is read at t = 30 s, by when all 8 have settled

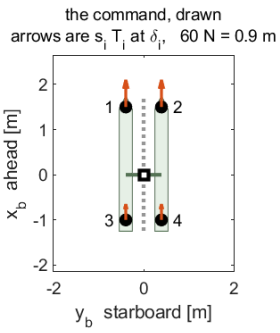
  case  command                      U [m/s]  crab [deg]  r [deg/s]  radius [m]
  ------------------------------------------------------------------------------
  S1    surge ahead                    1.627        0.00      0.000    straight
  S2    surge astern                   1.627      180.00      0.000    straight
  S3    yaw to starboard               0.030      -21.14     28.397        0.06
  S4    yaw to port                    0.030       21.14    -28.397        0.06
  S5    sway to starboard              0.747       74.28      7.589        5.64
  S6    sway, moment cancelled         0.895      113.99     -0.000    straight
  S7    turn while making way          1.464        5.89     20.239        4.15
  S8    tilt beyond the servo limit    0.747       74.28      7.589        5.64
  ------------------------------------------------------------------------------
```

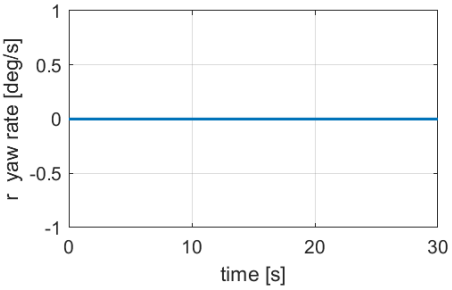
Each sheet carries the six states on the left, **the command drawn as four force arrows** at the top right, and the track with hull and heading below it. The arrow panel is what makes a wrong command distinguishable from a wrongly followed one.

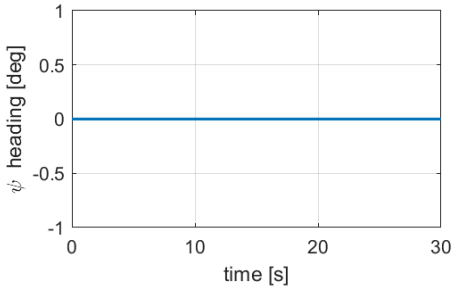
S1 surge ahead
n = [60.0 60.0 -60.0 -60.0] rad/s, δ = [0 0 0 0] deg, τ = [126.180 N, 0.000 N, 0.000 N m]
expected: runs straight ahead, no heading change: the hull settles at a steady u

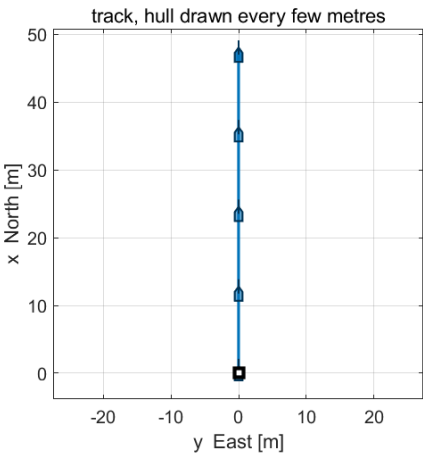


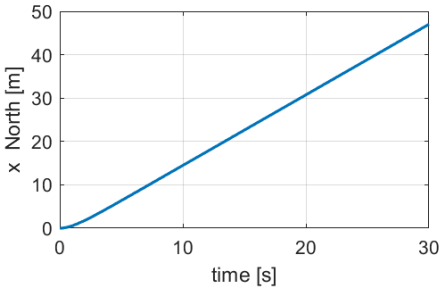


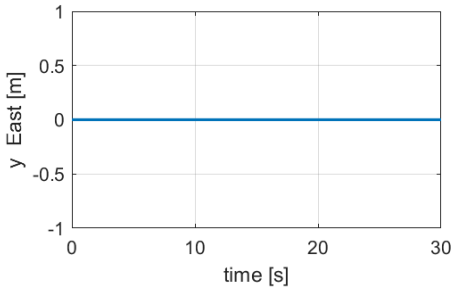




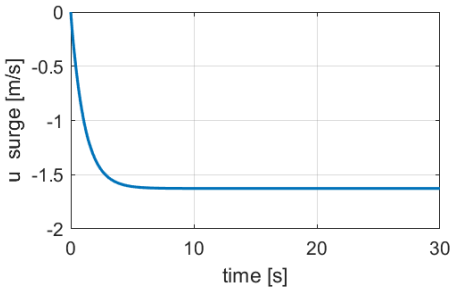


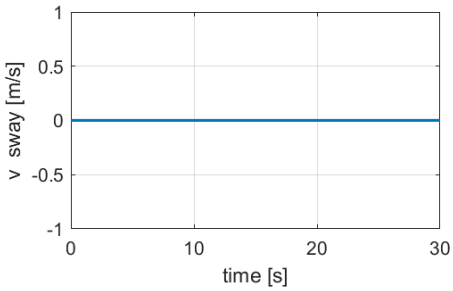


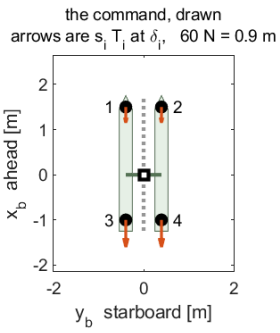


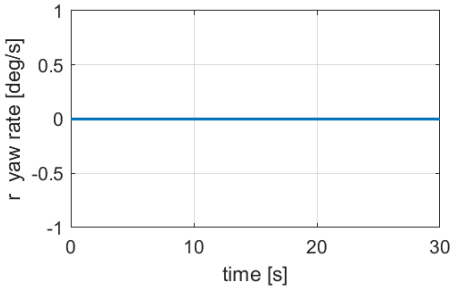


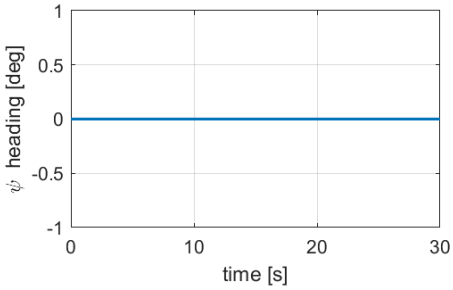
S2 surge astern
n = [-60.0 -60.0 60.0 60.0] rad/s, δ = [0 0 0 0] deg, τ = [-126.180 N, -0.000 N, 0.000 N m]
expected: runs straight astern with the SAME force magnitude as S1, mirrored

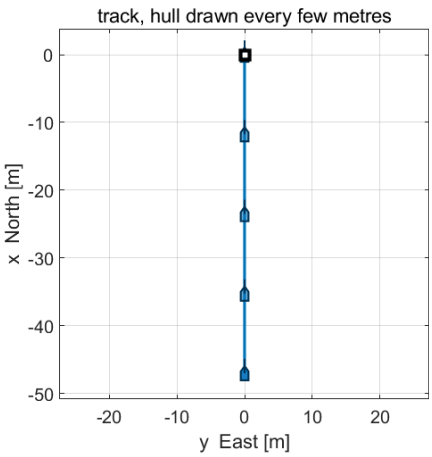


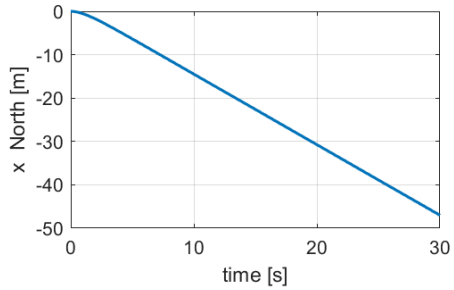


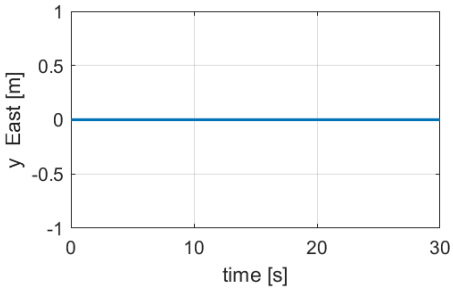




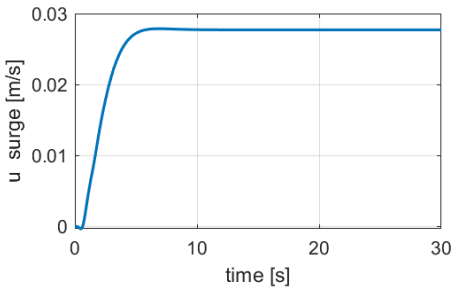


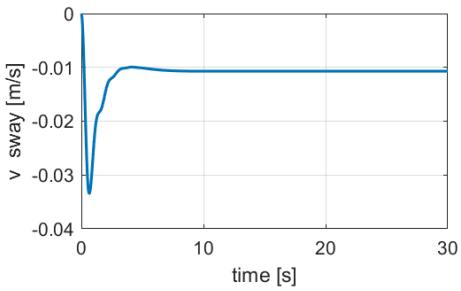


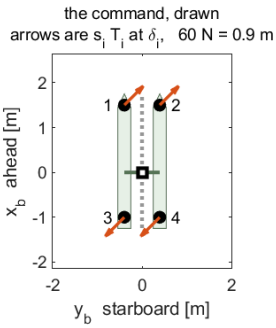


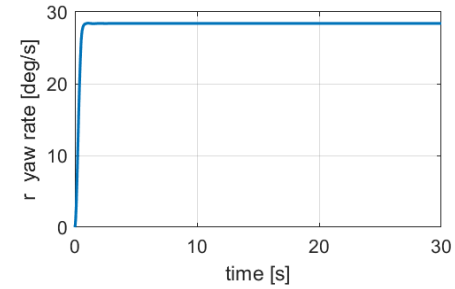


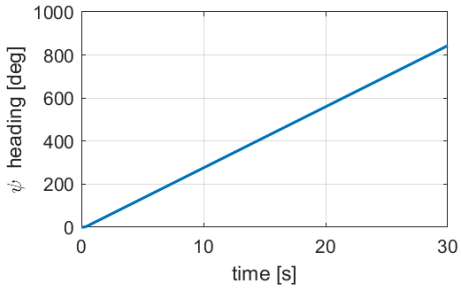
S3 yaw to starboard
n = [60.0 60.0 60.0 60.0] rad/s, δ = [45 45 45 45] deg, τ = [0.000 N, 0.000 N, 141.025 N m]
expected: turns on the spot to starboard: no net force at all, only a moment

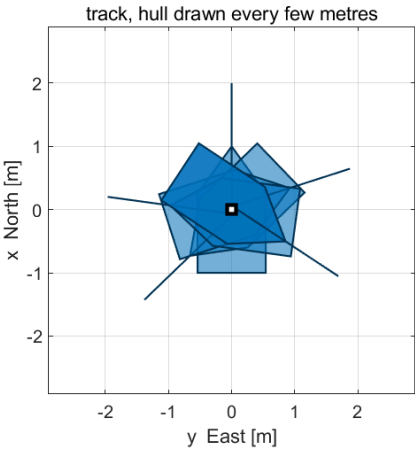


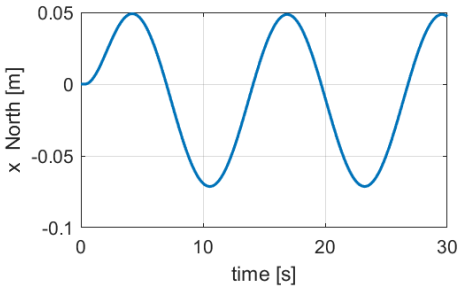


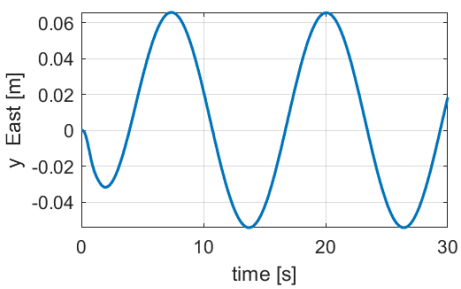




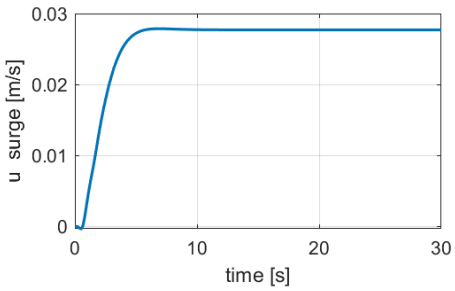


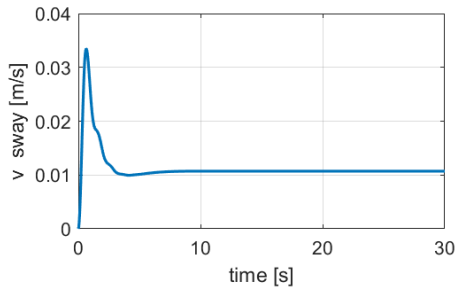


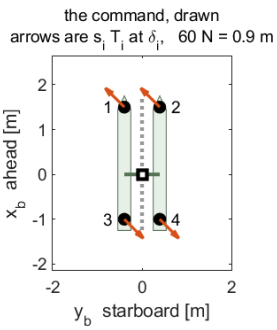


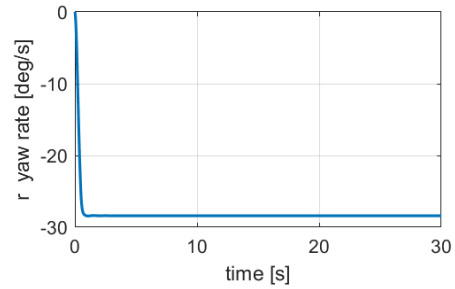


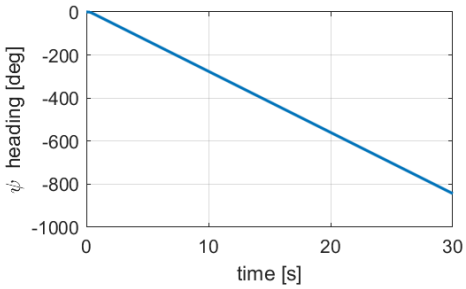
S4 yaw to port
n = [60.0 60.0 60.0 60.0] rad/s, δ = [-45 -45 -45 -45] deg, τ = [0.000 N, 0.000 N, -141.025 N m]
expected: turns on the spot to port, the exact mirror of S3

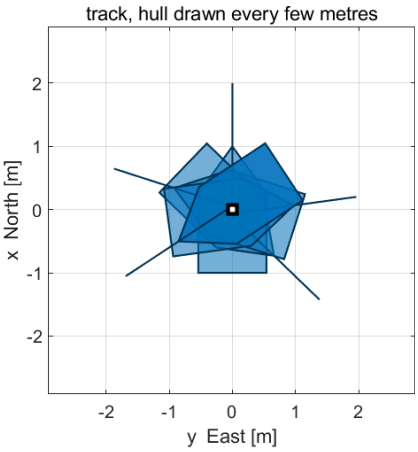


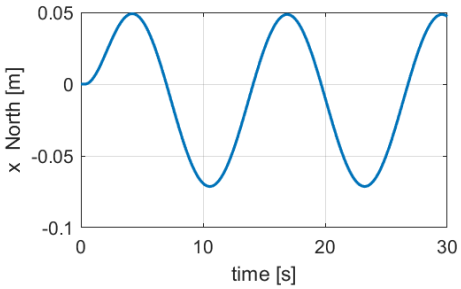


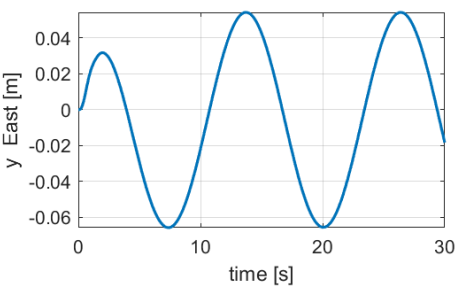




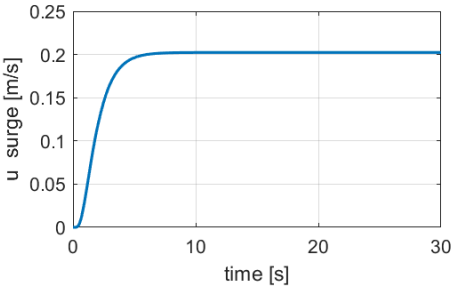


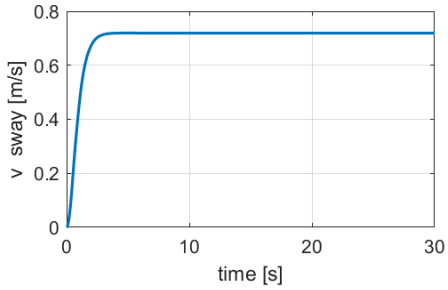


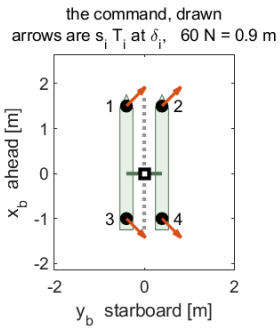


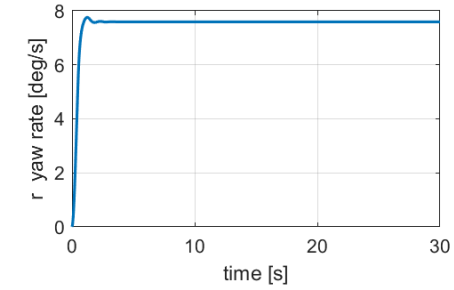


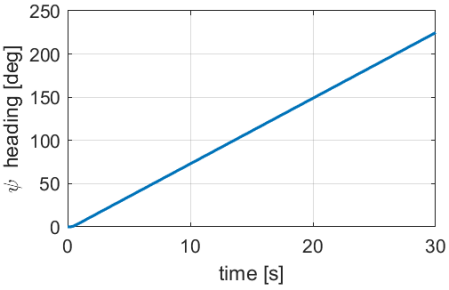
S5 sway to starboard
n = [60.0 60.0 60.0 60.0] rad/s, δ = [45 45 -45 -45] deg, τ = [0.000 N, 112.820 N, 28.205 N m]
expected: crabs to starboard with zero surge force, but keeps turning: $l_f > l_a$

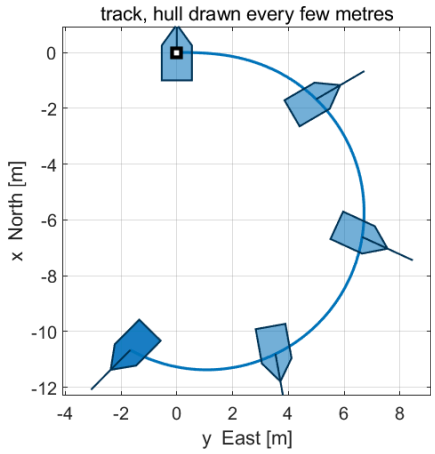


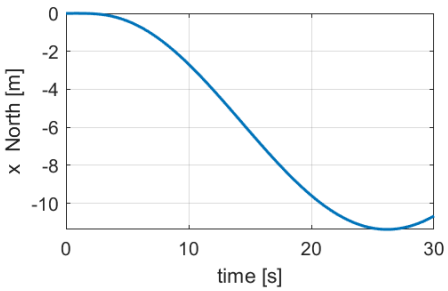


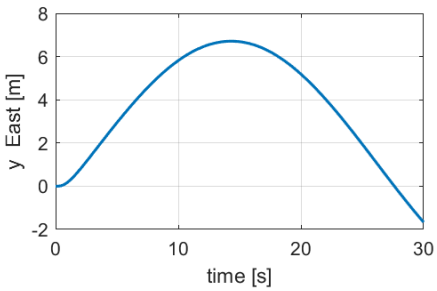




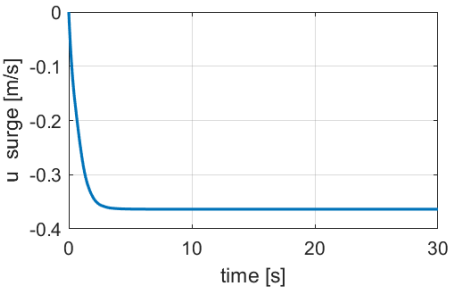


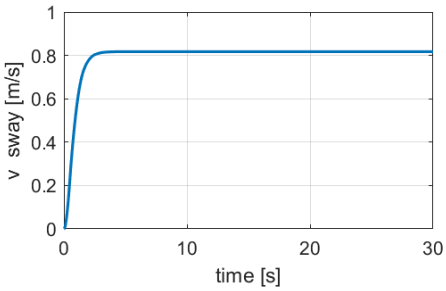


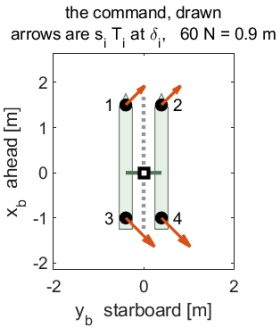


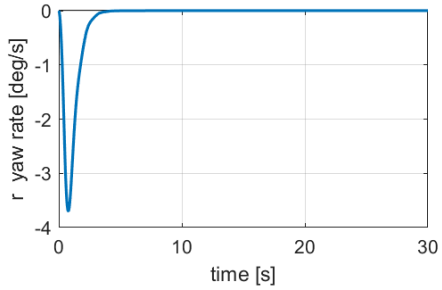


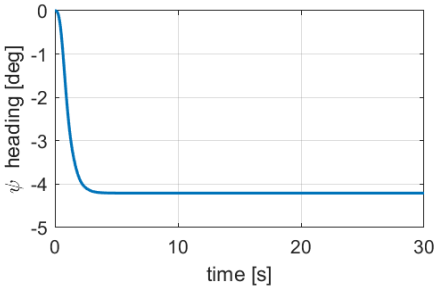
S6 sway, moment cancelled
n = [60.0 60.0 73.5 73.5] rad/s, δ = [45 45 -45 -45] deg, τ = [-28.205 N, 141.025 N, 0.000 N m]
expected: crabs to starboard on a steady heading, but now drifts ASTERN

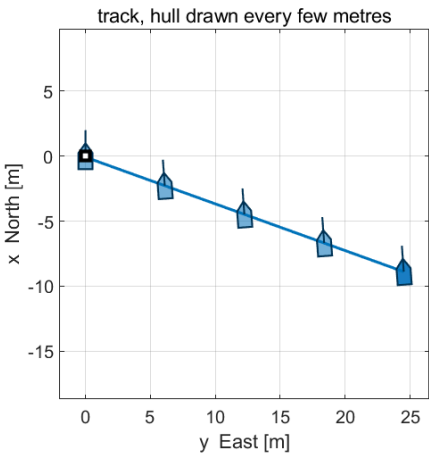


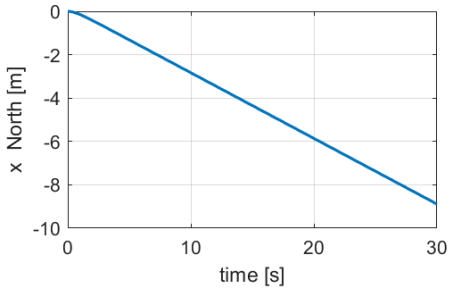


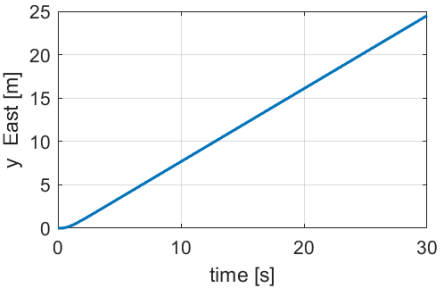




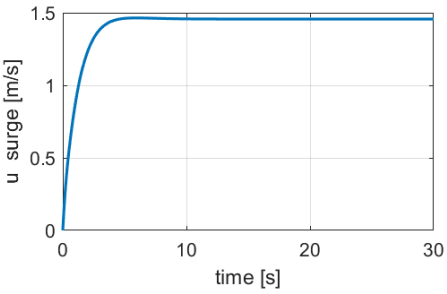


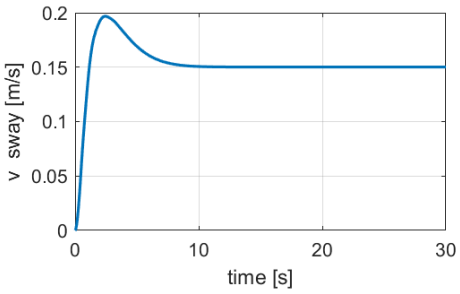


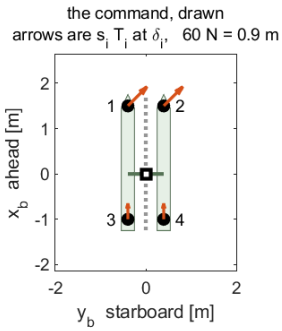


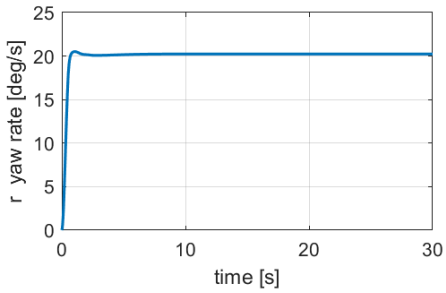


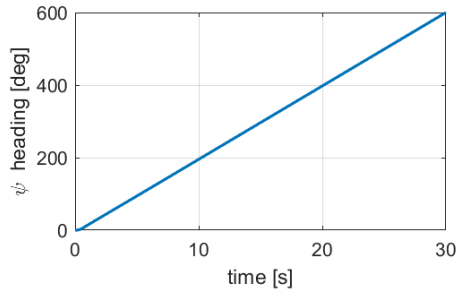
S7 turn while making way
n = [60.0 60.0 -60.0 -60.0] rad/s, δ = [45 45 0 0] deg, τ = [102.814 N, 56.410 N, 84.615 N m]
expected: surge, sway and yaw at once: a turning circle, the most ship-like row

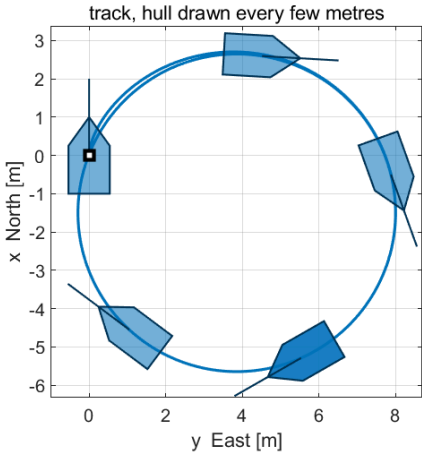


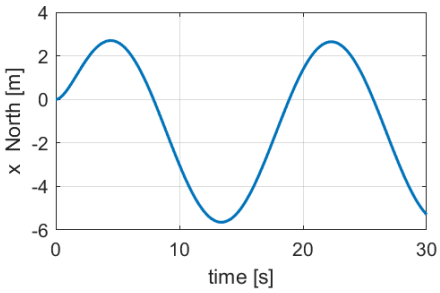


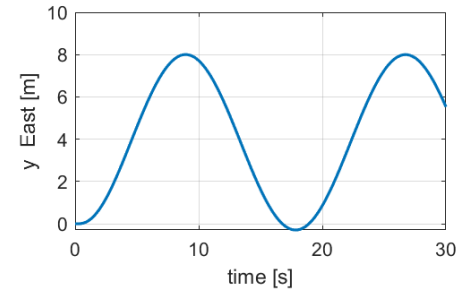




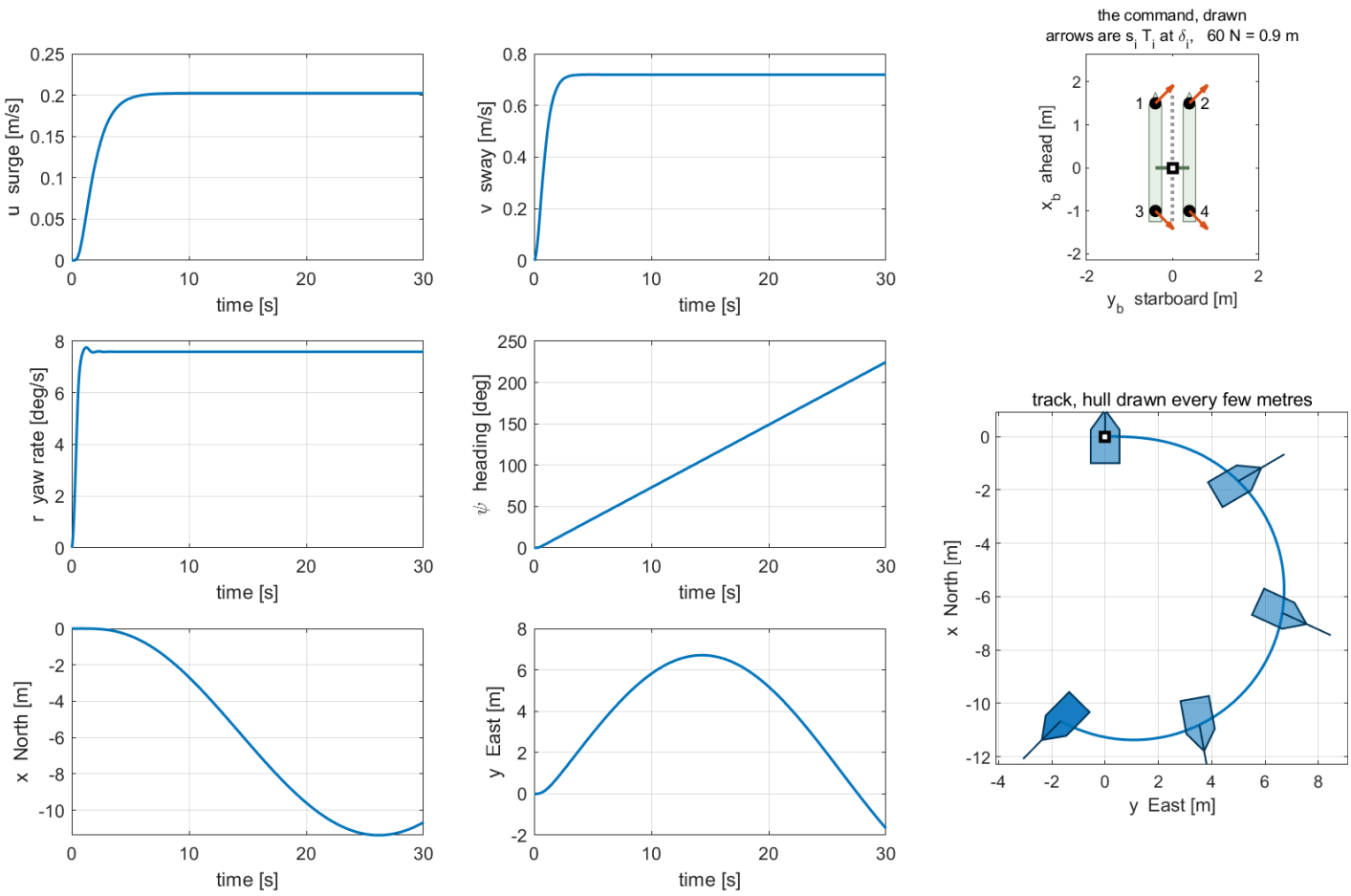




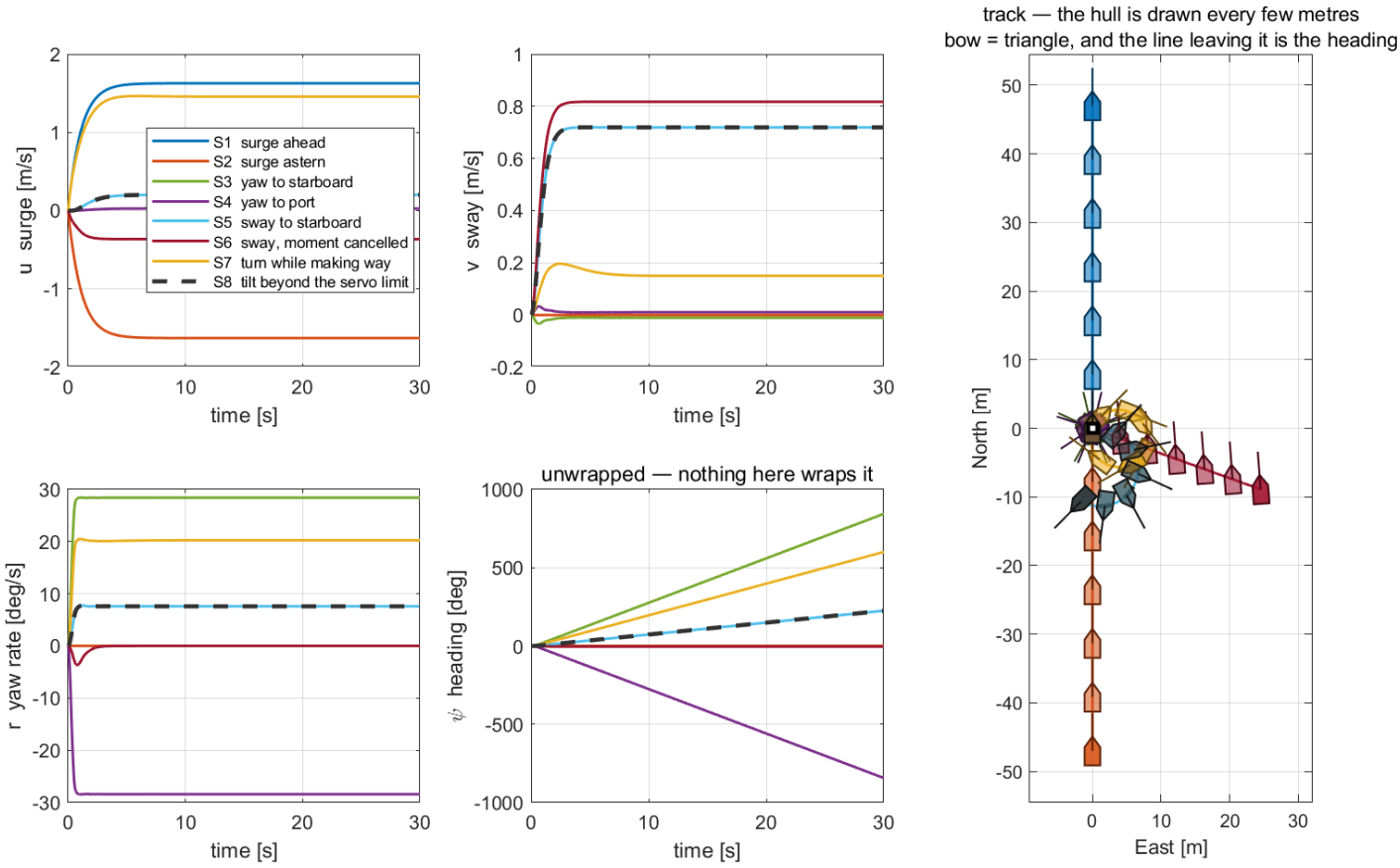




S8 tilt beyond the servo limit
n = [60.0 60.0 60.0 60.0] rad/s, δ = [70 70 -70 -70] deg, τ = [0.000 N, 112.820 N, 28.205 N m]
expected: identical to S5 in every number: the servo gave 45 deg, not 70



The overlay below is kept for comparison only. S8 is drawn dashed over S5, and the two curves coincide.



What to try	What to watch
`T_final = 120; animate = 1;` then Run	the hull drawn along the circle; S5 closes its 5.64 m circle in about 47 s
`W07_V_openloop({'S5','S6'})`	two sheets and no overlay file written — a partial run never overwrites the figure this section embeds

> ✎ **In class**
>
> - **Purpose** — connect a force in body axes to a track on the water, with nothing in between.
> - **Point to** — the S5 hull symbols on the track: the bow points well off the direction of travel.
> - **Ask** — "S7's force is at $28.74°$. Why does the hull crab at only $5.89°$?" Because sway is damped far harder than surge; the equilibrium is not the force direction.
> - **Take away** — an open-loop command gives a repeatable circle, and a circle is not a mission.

7-6. Whether the model is the equations

What is observed. Experiment 7-6 re-integrates each of the eight 30-s runs outside Simulink, with the same fixed-step `ode4` and the same step, driven at every step by the (n, δ) the model actually applied. The worst disagreement in position or speed over all eight runs is exactly $0.00\text{e}{+00}$.

- the driving signal is the **post-servo** pair read back from the log, not the command
- the heading column disagrees at 1.02×10^{-13} deg at worst, which is the unwrapping, not the dynamics

The result. Two different checks, and neither substitutes for the other.

Check	What it would catch	What it cannot catch
table (1) — hand formula against `otter_tilt4`	a wrong column, a wrong sign, a wrong arm	a model wired to the wrong signal, a wrong solver, a wrong step
table (2) — the model against independent re-integration	wiring, solver, step, saturation order	an error present in `otter_tilt4` itself, which both sides share

Derivation, one line at a time.

1. Table (1) compares two evaluations of the same function at one state. It therefore tests the function and nothing downstream of it.
2. A test of the model must re-run the model's own trajectory from an independent integrator. Writing the integrator by hand, in four lines of Runge–Kutta, removes Simulink from the comparison entirely.
3. The independent run must be given the same input the model received, which is **not** the command: the servo clipped it. Reading (n, δ) back out of the log, where they are tapped after the saturation stage, supplies the input actually applied.
4. The same fixed-step `ode4` at $h = 0.02$ s must be used. A variable-step solver on one side and a fixed step on the other would disagree at the step-error level and say nothing about the wiring.
5. A single-instant acceleration comparison would not do this job. The log carries 6 of the 12 states — $[u\ v\ r\ N\ E\ \psi]$ — and under sway the missing six, among them w, p, q and ϕ, are not zero. An acceleration reconstructed from the logged six would be evaluated at a state the vessel was never in.
6. Re-integration avoids that trap because it never reads the model's state: it starts from the same $\mathbf{x}_0$, which is known in full, and advances its own twelve states.
7. Agreement to $0.00\text{e}{+00}$ in position therefore says the two integrations performed bit-identical arithmetic, which is the strongest statement available: same equations, same inputs, same solver, same step.
8. The heading residual of 1.02×10^{-13} deg is not a dynamics disagreement. The log stores ψ in degrees and the comparison takes the smallest signed angle, so the residual is the cost of two conversions.

Experiment 7-6 · The model against an integrator written by hand (20 min)

What it measures. Lines 3, 4 and 7: that the wiring, the solver and the saturation order are what the equations assume.

The model. The same eight runs. The comparison lives in `W07_tab_reint`, which calls `otter_tilt4` directly inside its own Runge–Kutta loop.

Opening and running.

```
W07_V_openloop                % 블록 (2) 가 재적분을 한다 / block (2) re-integrates
```

Expected output, the second table:

```
(2) the Simulink model against the same equations, integrated outside it

    the same fixed-step ode4 and the same step 0.020 s, started from x0,
    driven at every step by the (n, delta) the model actually applied —
    the post-servo values read back from the log

  case   max |dN| [m]  max |dE| [m]  max |dpsi| [deg]  max |du| [m/s]   clip
  ------------------------------------------------------------------------------
  S1        0.00e+00      0.00e+00          0.00e+00        0.00e+00   n 0  d 0
  S2        0.00e+00      0.00e+00          0.00e+00        0.00e+00   n 0  d 0
  S3        0.00e+00      0.00e+00          1.02e-13        0.00e+00   n 0  d 1
  S4        0.00e+00      0.00e+00          1.02e-13        0.00e+00   n 0  d 1
  S5        0.00e+00      0.00e+00          2.54e-14        0.00e+00   n 0  d 1
  S6        0.00e+00      0.00e+00          0.00e+00        0.00e+00   n 0  d 1
  S7        0.00e+00      0.00e+00          1.02e-13        0.00e+00   n 0  d 1
  S8        0.00e+00      0.00e+00          2.54e-14        0.00e+00   n 0  d 1
  ------------------------------------------------------------------------------
  worst 0.0e+00 m or m/s over the 8 30 s runs
```

Reading the table against the derivation.

Where to look	What is there	Which line predicts it
`max dN` , `max dE` , `max du`	exactly zero, all eight runs	line 7
`max dpsi`	at most 1.02×10^{-13} deg	line 8 — the conversion, not the dynamics
`d 1` on S3 to S8	the tilt saturation bound during those runs	line 3 — the input re-integrated is the clipped one
`n 0` everywhere	the shaft limit never bound	$60 < 103.931$ and $73.485 < 103.931$

What to try	What to watch
`h = 0.05;` then `W07_V_openloop`	still zero. The agreement is between two integrators, so a coarser step moves both together
drive the re-integration with `n_cmd` instead of the logged `n`	S8 stops agreeing, because the command is $70°$ and the plant received $45°$ — line 3

✎ In class

- **Purpose** — separate a check of the equations from a check of the model, and show that this week has both.
- **Point to** — the `clip` column. The re-integration agrees **because** it was given the clipped input.
- **Ask** — "Why not just compare accelerations at one instant?" Because the log carries six of twelve states, and the other six are not zero under sway.
- **Take away** — zero is a stronger result than small, and it is only available when both sides run the same solver on the same input.

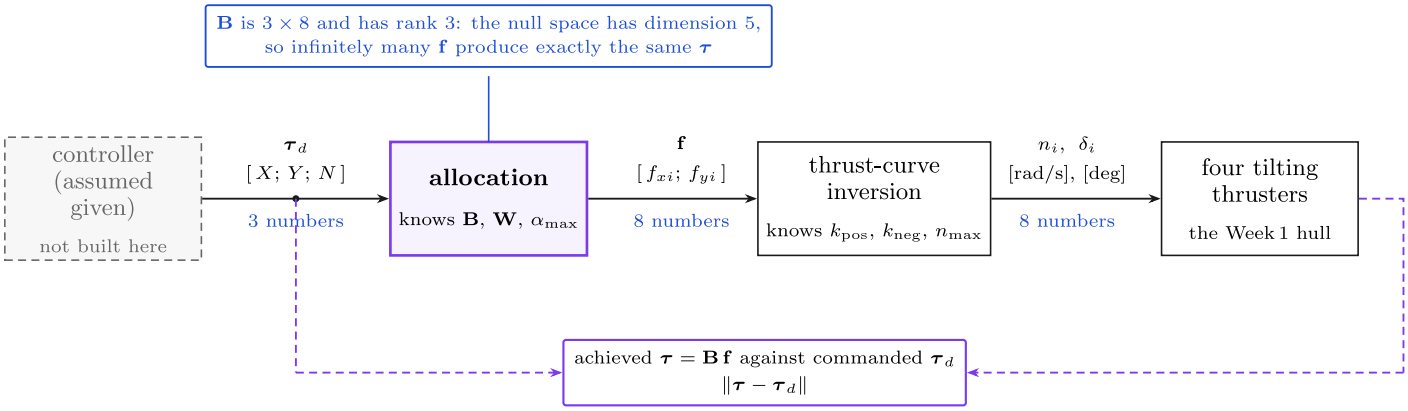
7-7. Three numbers in, eight numbers out

What is observed. Every control law of Weeks 2 to 6 ends in a demanded force: a surge force X, a sway force Y, a yaw moment N. The hull of this week does not accept those three numbers. It accepts four shaft speeds and four servo angles. The stage that stands between them is **control allocation**.

From here to the end of the week the controller is **assumed given**. No gain is tuned and no error signal appears. The demand

$$\boldsymbol{\tau}_d = \begin{bmatrix} X \\ Y \\ N \end{bmatrix}$$

arrives from outside the stage under study, exactly as a measurement arrives from outside a filter. What is studied is the map from those three numbers to the eight the hardware needs.



The result. The stage has one input, one output, and a fixed table of things it knows in advance.

Symbol	Quantity	Size and units
$\boldsymbol{\tau}_d$	demanded surge force, sway force, yaw moment	3×1, [N, N, N·m]
$\mathbf{f}$	per-unit thrust components $[f_{x,1}; f_{y,1}; \ldots; f_{x,4}; f_{y,4}]$	8×1, [N]
$\mathbf{B}$	configuration matrix of §7-1, from the Week 3 column rule	3×8, [--, --, m]
$\mathbf{W}$	weight matrix — the price put on each component	8×8, diagonal
$\alpha_{\max}$	servo limit of each unit, $45°$ for all four	4×1, [rad]
$\mathbf{n}$	shaft speeds handed to the plant	4×1, [rad/s]
$\boldsymbol{\delta}$	servo angles handed to the plant	4×1, [rad]
$\boldsymbol{\tau}$	**achieved** force, $\mathbf{Bf}$, which is compared with $\boldsymbol{\tau}_d$	3×1, [N, N, N·m]

Two facts fix everything that follows.

1. §7-1 established that a tilting unit at station (x_i, y_i) contributes **two** columns to $\mathbf{B}$, one for its forward component and one for its sideways component, so that the force the four units make is linear in the eight components:

$$\mathbf{Bf} = \boldsymbol{\tau}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 0 & 1 & 0 & -1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 1 & 0 & -1 & 0 & -1 \\ d & l_f & -d & l_f & -d & l_a & d & l_a \end{bmatrix}$$

2. The equation to be solved therefore has **three rows and eight unknowns**. It is not a matrix inversion. Three equations cannot pin down eight numbers, and the deficit is exactly what the null space counts:

$$\operatorname{rank}\mathbf{B} = 3, \qquad \dim\ker\mathbf{B} = 8 - 3 = 5$$

3. Five dimensions of freedom means that for **any** reachable $\boldsymbol{\tau}_d$ there are infinitely many eight-number commands that achieve it exactly. If $\mathbf{f}^\star$ is one of them and $\mathbf{z}$ is any vector with $\mathbf{B}\mathbf{z} = \mathbf{0}$, then $\mathbf{f}^\star + \mathbf{z}$ achieves the same force.
4. Allocation is therefore not the problem of finding **a** solution. It is the problem of **choosing among** the solutions. Every method in §7-8 to §7-10 differs only in what it decides to make small, and in which limits it refuses to exceed.

> **★ What makes this a design problem and not an algebra problem**
>
> An under-determined system has no single answer, so the engineer must supply the missing statement of what a good answer is. The three statements used in this course, in the order they are introduced: **use as little thrust as possible** (§7-8), **use the cheap components before the expensive ones** (§7-9), and **never ask a servo for an angle it does not have** (§7-10).

7-8. The smallest solution: the pseudo-inverse

What is observed. Among the infinitely many commands that achieve a demanded force, one is shorter than all the others. Making that choice turns an under-determined equation into a formula with no iteration in it.

The result. The choice is written as a minimisation, solved once with Lagrange multipliers, and the answer is a constant matrix times the demand.

Derivation.

1. Choose what to make small. The sum of squares of the eight components is the natural first choice, because it is the square of the total thrust vector's length:

$$J(\mathbf{f}) = \mathbf{f}^\top\mathbf{f} = \sum_{i=1}^{4}\left(f_{x,i}^2 + f_{y,i}^2\right) = \sum_{i=1}^{4} T_i^2$$

2. State the problem. Minimise J subject to achieving the demand exactly:

$$\min_{\mathbf{f}} \ \mathbf{f}^\top\mathbf{f} \quad \text{subject to} \quad \mathbf{B}\mathbf{f} = \boldsymbol{\tau}_d$$

3. Attach the constraint with a multiplier $\boldsymbol{\lambda} \in \mathbb{R}^3$, one entry for each row of $\mathbf{B}$:

$$L(\mathbf{f}, \boldsymbol{\lambda}) = \mathbf{f}^\top\mathbf{f} + \boldsymbol{\lambda}^\top(\boldsymbol{\tau}_d - \mathbf{B}\mathbf{f})$$

4. Set the gradient in $\mathbf{f}$ to zero. This is where the eight unknowns collapse onto three:

$$\frac{\partial L}{\partial \mathbf{f}} = 2\mathbf{f} - \mathbf{B}^\top\boldsymbol{\lambda} = \mathbf{0} \quad \Longrightarrow \quad \mathbf{f} = \tfrac{1}{2}\mathbf{B}^\top\boldsymbol{\lambda}$$

5. The solution is thus constrained to lie in the three-dimensional row space of $\mathbf{B}$. Substituting it back into the constraint gives the multiplier:

$$\mathbf{B}\left(\tfrac{1}{2}\mathbf{B}^\top\boldsymbol{\lambda}\right) = \boldsymbol{\tau}_d \quad \Longrightarrow \quad \boldsymbol{\lambda} = 2\left(\mathbf{B}\mathbf{B}^\top\right)^{-1}\boldsymbol{\tau}_d$$

6. Put the multiplier back into line 4:

$$\boxed{\mathbf{f} = \mathbf{B}^\top\left(\mathbf{B}\mathbf{B}^\top\right)^{-1}\boldsymbol{\tau}_d}$$

7. The inverse exists. $\mathbf{B}\mathbf{B}^\top$ is only 3×3, and it is non-singular precisely because $\operatorname{rank}\mathbf{B} = 3$, which §7-1 verified for this geometry. The matrix $\mathbf{B}^\top(\mathbf{B}\mathbf{B}^\top)^{-1}$ is the Moore–Penrose pseudo-inverse $\mathbf{B}^+$ of a matrix with full row

rank.

8. The residual is zero by construction, not approximately zero:

$$\mathbf{B}\mathbf{f} = \mathbf{B}\mathbf{B}^{\top}\left(\mathbf{B}\mathbf{B}^{\top}\right)^{-1}\boldsymbol{\tau}_d = \boldsymbol{\tau}_d$$

9. The solution is orthogonal to the null space. For any $\mathbf{z}$ with $\mathbf{B}\mathbf{z} = \mathbf{0}$, line 4 gives $\mathbf{f}^{\top}\mathbf{z} = \tfrac{1}{2}\boldsymbol{\lambda}^{\top}\mathbf{B}\mathbf{z} = 0$, so $\|\mathbf{f}+\mathbf{z}\|^2 = \|\mathbf{f}\|^2 + \|\mathbf{z}\|^2 \geq \|\mathbf{f}\|^2$. **Adding anything from the null space can only cost thrust.** This is the measurable prediction the derivation makes, and Experiment 7-8 tests it.

Symbol	Quantity	Value or source
$\mathbf{B}^{+}$	$\mathbf{B}^{\top}(\mathbf{B}\mathbf{B}^{\top})^{-1}$, pseudo-inverse of $\mathbf{B}$	8×3, built from `cfg.B`
$\boldsymbol{\lambda}$	Lagrange multiplier, one entry per demanded component	3×1
$\mathbf{z}$	any null-space vector, $\mathbf{B}\mathbf{z} = \mathbf{0}$	8×1, 5 free dimensions
$\mathbf{B}\mathbf{B}^{\top}$	the 3×3 matrix actually inverted	non-singular, $\operatorname{rank}\mathbf{B} = 3$

From eight components to eight commands. The plant of §7-2 takes shaft speeds and servo angles, not components, so one more stage is needed. It is pure trigonometry and the inverse of the thrust curve, and it contains no choices.

1. The sign of each unit's thrust is read off its forward component, so that a unit pushing astern is described as negative thrust at a small angle rather than positive thrust at $180°$:

$$\sigma_i = \operatorname{sign}(f_{x,i}), \qquad T_i = \sigma_i\sqrt{f_{x,i}^2 + f_{y,i}^2}$$

2. The servo angle is the direction of the component pair, measured in the frame the sign choice fixes:

$$\delta_i = \operatorname{atan2}(\sigma_i f_{y,i},\ \sigma_i f_{x,i})$$

2-a. **That σ_i inside the $\operatorname{atan2}$ is the whole angle fold.** Multiplying *both* arguments by $\sigma_i = -1$ rotates the reported direction by exactly π, and since $\sigma_i f_{x,i} \geq 0$ always holds, the result can never leave $\left[-\tfrac{\pi}{2}, +\tfrac{\pi}{2}\right]$. Line 2 is therefore identical to the two-branch form in which the raw angle is taken first and then wrapped:

One line, as written here	Two branches, written separately
$\sigma_i = \operatorname{sign}(f_{x,i})$	$\alpha_i = \operatorname{atan2}(f_{y,i}, f_{x,i})$
$T_i = \sigma_i\|(f_{x,i}, f_{y,i})\|$	if $\alpha_i > \pi/2$: $\alpha_i \leftarrow \alpha_i - \pi$
$\delta_i = \operatorname{atan2}(\sigma_i f_{y,i},\ \sigma_i f_{x,i})$	if $\alpha_i < -\pi/2$: $\alpha_i \leftarrow \alpha_i + \pi$
—	the thrust sign is then carried as $\operatorname{sign}(f_{x,i})$

Over $20\,000$ random component pairs the two agree to $4.44\text{e}{-16}$ rad with no difference in commanded shaft speed, and rebuilding $\mathbf{f}$ from (T_i, δ_i) returns the original to $1.14\text{e}{-13}$ N, so the fold loses nothing.

2-b. **Without the fold the servo limit destroys the demand rather than weakening it.** A unit asked for $\mathbf{f}_i = (-50, 0)$ N is a unit pushing straight astern:

	thrust T_i	commanded δ_i	after the $45°$ servo limit	force the unit then makes
raw $\operatorname{atan2}$, no fold	$+50.000$ N	$180.0°$	$45.0°$	$(+35.355, +35.355)$ N
folded, as in line 2	-50.000 N	$-0.0°$	$-0.0°$	$(-50.000, 0.000)$ N

The unfolded command is not merely degraded — the forward component has changed sign and a sideways component has appeared that was never demanded. Swept over a demand of 100 N at every $5°$ of heading, 216 of the 288 unit commands (75%) have a raw angle outside $\pm 45°$, and the resulting residual $\|\mathbf{B}\mathbf{f} - \boldsymbol{\tau}_d\|$ is:

	mean residual	worst residual
folded	18.999 N	68.486 N
not folded	66.560 N	106.527 N

The folded residual is not zero because the $45°$ limit is a real restriction — that is what §7-10 solves. The point of the fold is that what remains is a restriction and not a corruption.

> **⚠ $\mathrm{sign}(0)$ is a decision, not a detail**
>
> MATLAB's `sign(0)` is 0, which would set $T_i = 0$ and discard a unit whose forward component happens to be exactly zero — the case of a pure sideways demand. `W07_alloc` therefore forces $\sigma_i \leftarrow +1$ wherever $\mathrm{sign}(f_{x,i}) = 0$, which keeps $T_i = \|\mathbf{f}_i\|$ at $\delta_i = +90°$. An implementation that leaves `sign(0) = 0` in place loses one unit's thrust at exactly the demands that need it most.

3. The thrust curve of §7-1 is inverted with the branch that matches the sign of the thrust, because the two coefficients differ:

$$n_i = \mathrm{sign}(T_i)\sqrt{\frac{|T_i|}{k_i}}, \qquad k_i = \begin{cases} k_{\mathrm{pos}} = 0.011080 & T_i \geq 0 \\ k_{\mathrm{neg}} = 0.006445 & T_i < 0 \end{cases}$$

4. The thrust cap needs **no constraint inside the allocation**. Because $\mathbf{B}$ is linear, scaling the whole command by one scalar ρ scales the achieved force by the same scalar and **leaves its direction unchanged**:

$$\rho = \min\left(1,\ \min_i \frac{T_{\max,i}}{|T_i|}\right), \qquad \mathbf{B}(\rho\,\mathbf{f}) = \rho\,\boldsymbol{\tau}_d$$

5. So a shaft limit weakens the demand but does not bend it, and one line after the solve is enough. The servo limit of §7-10 is **not** of this kind: clipping an angle changes the direction of $\boldsymbol{\tau}$, which is why it has to be handled inside the allocation and not after it.

Experiment 7-8 · The smallest of infinitely many (15 min)

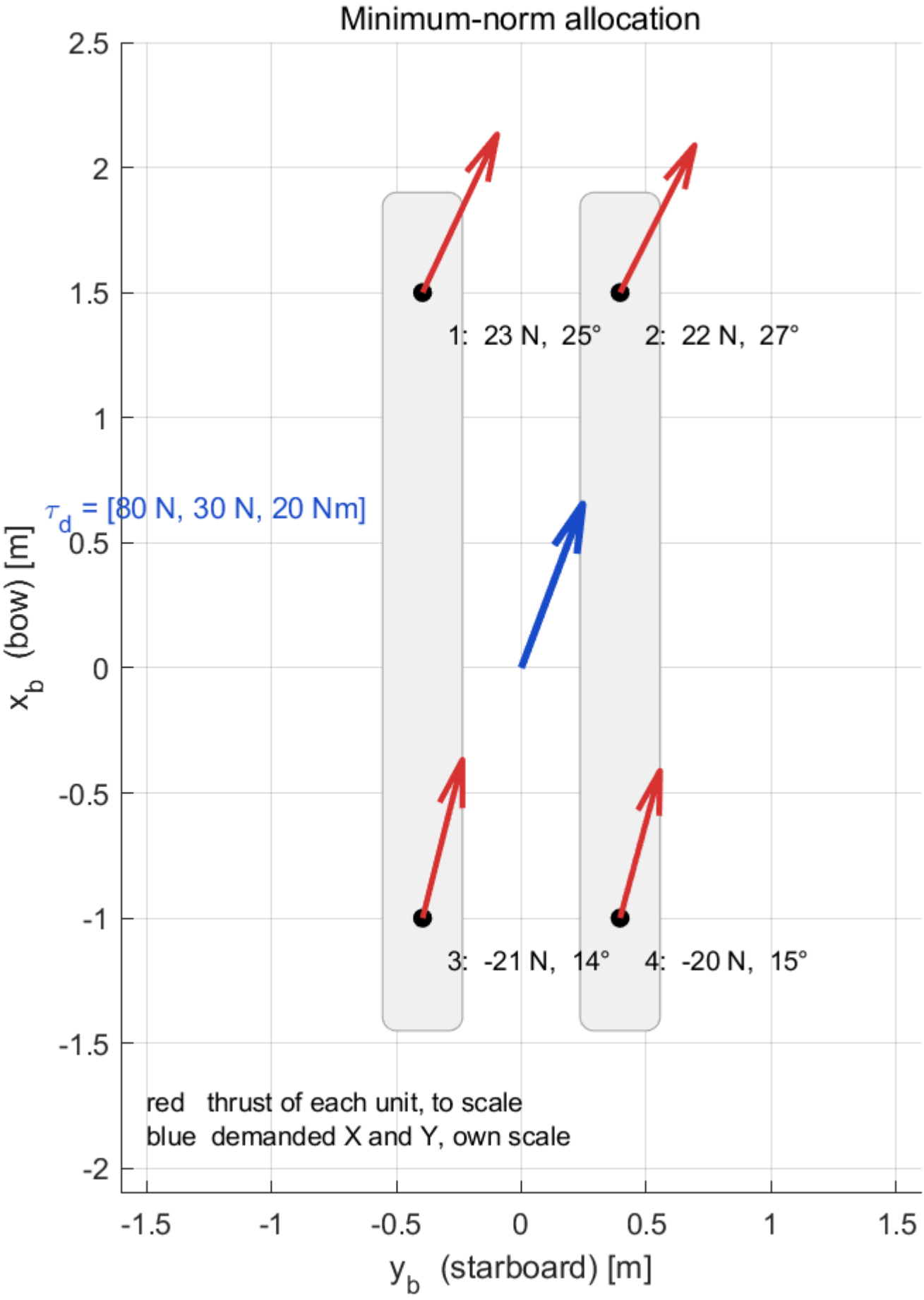
```
W07_CA(8)
```

```
=== 7-8  the allocation problem and its minimum-norm solution ===
B is 3x8,  rank 3,  null space dimension 5
demand  tau_d = [80 N, 30 N, 20 N m]

(1) two solutions that give the same tau
    minimum norm      ||f|| =   42.99 N   ||B f - tau|| = 0.00e+00
    null space added          58.72 N               7.94e-15
    -> the same force costs the thrusters 37 % more

(2) minimum norm  f = B^T (B B^T)^{-1} tau
    unit      f_x [N]   f_y [N]
     1         20.718     9.773
     2         19.282     9.773
     3        -20.718    -5.227
     4        -19.282    -5.227

(3) hardware command
    unit     T [N]   delta [deg]    n [rad/s]   |delta|<=45?
     1      22.908       25.25        45.47        true
     2      21.617       26.88        44.17        true
     3     -21.367       14.16       -57.58        true
     4     -19.978       15.17       -55.68        true
    rho = 1.0000   (1 means the thrust limit was not reached)
    tau achieved = [80.000 30.000 20.000]   distance from the demand 0.00e+00
```



Where to look	What is there	Which line predicts it
the two residuals in block (1)	$0.00e{+}00$ and $7.94e{-}15$	line 8 — both solutions achieve the demand exactly

Where to look	What is there	Which line predicts it
42.99 N against 58.72 N	the null-space solution costs 37% more thrust for the same force	line 9 — adding $\mathbf{z}$ can only add length
f_y equal within each pair, 9.773 and -5.227	the sideways components split by pair, not by unit	§7-7 line 1 — the f_y columns carry both Y and the long moment arms
$\lvert f_{x,1}\rvert = \lvert f_{x,3}\rvert = 20.718$	the forward components are antisymmetric about the beam	the $\pm d$ entries in row 3 of $\mathbf{B}$
signs of n_3 and n_4	negative, and larger in magnitude than n_1, n_2	$k_{\text{neg}} < k_{\text{pos}}$, so reverse thrust needs more shaft speed
$\rho = 1.0000$	no shaft limit was reached, so the demand was met in full	line 4 — ρ is the only thing that can shrink it

What to try	What to watch
raise the demand to `tau_d = [300; 100; 60]`	ρ drops below 1 and the achieved force shrinks **along the same direction** — line 4
set `tau_d = [0; 0; 20]`	all four f_x vanish; a pure moment is carried on the long f_y arms alone
compare `B'*((B*B')\tau)` with `pinv(B)*tau`	the same numbers. `pinv` forms a singular-value decomposition, which is unnecessary when $\mathbf{B}$ has full row rank

✎ **In class**

- **Purpose** — show that allocation is a choice, and that one specific choice has a closed form.
- **Point to** — the 37% figure. It is the price of a solution that is correct but not minimal.
- **Ask** — "Why is $\mathbf{B}\mathbf{B}^{\top}$ only 3×3 when there are eight unknowns?" Because line 4 restricts the answer to three directions before the inverse is ever taken.
- **Take away** — a formula, not an iteration: $\mathbf{f} = \mathbf{B}^{+}\boldsymbol{\tau}_d$.

7-9. The weight matrix, and how it is tuned

What is observed. The minimum-norm solution treats all eight components as equally desirable. They are not. Sideways thrust means a tilted servo, and a hull may prefer to keep its servos near zero, or to spare one tired thruster, or to load the bow pair rather than the stern pair. The weight matrix is where such a preference is written down.

⚠ **Two matrices are both called $\mathbf{W}$ in allocation work, and they weight different things**

One of them prices the **force error** $\mathbf{B}\mathbf{f} - \boldsymbol{\tau}_d$, which is how a solver is told that a newton-metre of lost yaw matters more than a newton of lost surge. This section prices the **effort** $\mathbf{f}$ itself, which is how it is told that one component or one unit is dearer than another. Both appear together in §7-10, where the error weight is written $\mathbf{Q}$ to keep them apart.

The result. Attaching a price to each component changes the closed form in one place, and the resulting matrix is still constant. Measuring the sweep then shows something that the formula alone does not: **the price list cannot override what the demand itself has already fixed.**

Derivation.

1. Replace the plain sum of squares with a **weighted** sum, where $w_j > 0$ is the price of component j:

$$J(\mathbf{f}) = \mathbf{f}^\top \mathbf{W} \mathbf{f} = \sum_{j=1}^{8} w_j f_j^2, \qquad \mathbf{W} = \mathrm{diag}(w_1, \ldots, w_8)$$

2. Repeating §7-8 lines 3 to 6 with this cost changes two things and nothing else. The gradient becomes $2\mathbf{W}\mathbf{f} - \mathbf{B}^\top \boldsymbol{\lambda} = \mathbf{0}$, so $\mathbf{f} = \frac{1}{2}\mathbf{W}^{-1}\mathbf{B}^\top \boldsymbol{\lambda}$, and the constraint then gives:

$$\boxed{\mathbf{f} = \mathbf{W}^{-1}\mathbf{B}^\top \left(\mathbf{B}\mathbf{W}^{-1}\mathbf{B}^\top\right)^{-1} \boldsymbol{\tau}_d}$$

3. Setting $\mathbf{W} = \mathbf{I}$ recovers §7-8 exactly, so the pseudo-inverse is the special case in which everything costs the same.
4. **Only ratios matter.** Replacing $\mathbf{W}$ by $c\mathbf{W}$ for any $c > 0$ leaves the formula unchanged, because c cancels between $\mathbf{W}^{-1}$ and $(\mathbf{B}\mathbf{W}^{-1}\mathbf{B}^\top)^{-1}$. A weight has no unit and no absolute size; $\mathrm{diag}(1, 10)$ and $\mathrm{diag}(100, 1000)$ are the same price list.
5. **What a price list can change is who pays, not how much is owed.** This is the part the formula hides, and the geometry of $\mathbf{B}$ decides it. Row 2 of $\mathbf{B}$ is $[0\ 1\ 0\ 1\ 0\ -1\ 0\ -1]$, so the sway force is produced by the f_y components **and by nothing else**:

$$Y = f_{y,1} + f_{y,2} - f_{y,3} - f_{y,4} \quad \Longrightarrow \quad \sum_i |f_{y,i}| \ \geq\ |Y|$$

6. The demand therefore **pins** the total sideways thrust from below. Pricing f_y at a hundred times f_x cannot reduce $\sum_i |f_{y,i}|$ below $|Y|$, and in this geometry the weighted solution sits exactly on that bound.
7. What a high f_y price does instead is move the **yaw moment** off the f_y columns. Row 3 of $\mathbf{B}$ shows two kinds of moment arm: the f_y components act on the long arms $l_f = 1.5$ m and $l_a = 1.0$ m, the f_x components only on the half-beam $d = 0.395$ m. Taking the moment away from the long arms and onto the short one needs a **larger** differential in f_x.
8. So the prediction is counter-intuitive and sharp: raising the price of the sideways component leaves $\sum |f_{y,i}|$ unchanged, makes the forward components lopsided, and therefore makes the worst servo angle **larger**, not smaller. Experiment 7-9 sweeps the price over three decades and measures all three quantities.
9. The preference a diagonal $\mathbf{W}$ **can** express is between **units**. Pricing the aft pair above the bow pair moves the thrust share forward, because the four units are genuinely interchangeable in a way that f_x and f_y are not.

Symbol	Quantity	Value or source
$\mathbf{W}$	price list, diagonal and positive	8×8, `P.W`
w	price put on every f_y in the sweep	$1 \ldots 1000$, dimensionless
l_f, l_a	moment arms of the f_y components	1.5 m, 1.0 m — §7-1
d	moment arm of the f_x components	0.395 m — half beam, §7-1
$\mathbf{K}_W$	$\mathbf{W}^{-1}\mathbf{B}^\top(\mathbf{B}\mathbf{W}^{-1}\mathbf{B}^\top)^{-1}$	8×3, constant — see §7-11

★ How a weight matrix is tuned

The tuning rule follows from lines 4, 6 and 9, and it is short.

- **Set the ratios, not the magnitudes.** Start from $\mathbf{W} = \mathbf{I}$ and raise one entry at a time; a common factor does nothing (line 4).
- **Price what is genuinely redundant.** Two units that can do each other's work respond to a price. A component that is the only source of a demanded axis does not (line 6).
- **Each change must be measured, not assumed.** The sweep below moves the worst servo angle the opposite way from the intuition behind it.
- **A hard limit is not a price.** A servo that must never pass $45°$ belongs in a constraint (§7-10), not in $\mathbf{W}$. A large weight discourages; only a constraint forbids.

Experiment 7-9 · What the price list can and cannot do (20 min)

```
W07_CA(9)
```

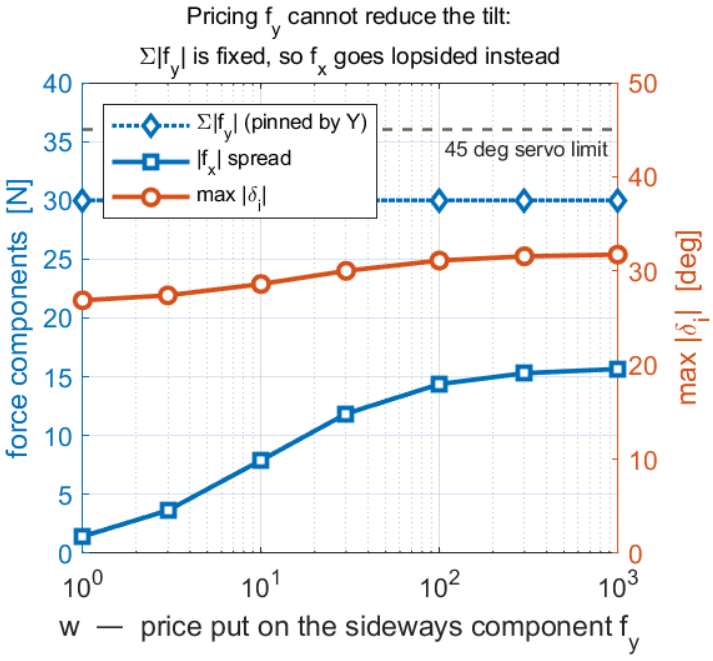
```
=== 7-9  the weight matrix W ===
demand tau_d = [80 N, 30 N, 20 N m],  W = diag([1 w 1 w 1 w 1 w])
Y comes from the f_y columns alone (row 2 of B = [0 1 0 1 0 -1 0 -1]) -> sum|f_y| is pinned

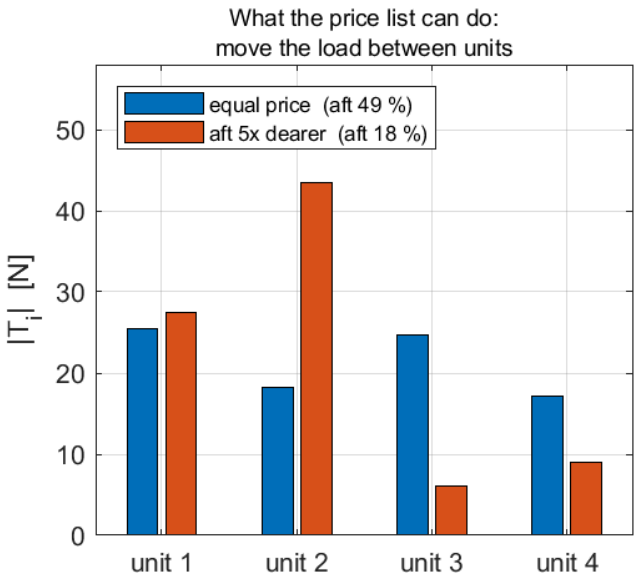
       w   sum|f_y| [N]   f_x spread [N]   max|delta| [deg]   ||f|| [N]   ||B f - tau||
       1         30.00             1.44              26.88        42.99         0.0e+00
       3         30.00             3.65              27.40        43.05         7.1e-15
      10         30.00             7.91              28.60        43.52         7.1e-15
      30         30.00            11.86              30.01        44.35         7.1e-15
     100         30.00            14.38              31.10        45.08         0.0e+00
     300         30.00            15.31              31.55        45.38         0.0e+00
    1000         30.00            15.67              31.73        45.50         3.6e-15

distance between the W and 1000*W solutions : 2.81e-15 N   (only ratios matter)

what the price list can change: who carries the share
  same price fore and aft : |T| = [ 25.50  18.28  24.75  17.22] N   aft pair 48.9 %
  aft priced 5x dearer    : |T| = [ 27.44  43.39   6.02   9.02] N   aft pair 17.5 %

what the price list cannot change: what the demand has already fixed
  sum|f_y| : 30.00 N and 30.00 N  (Y = 30 N fixes it)
```





Where to look	What is there	Which line predicts it
the $\Sigma\lvert f_y\rvert$ column	30.00 N at every price, equal to the demanded Y	line 6 — the bound $\sum\lvert f_{y,i}\rvert \geq \lvert Y\rvert$, met with equality
the spread column	$1.44 \rightarrow 15.67$ N as the price rises	line 7 — the moment moves onto the 0.395 m arm
the worst-angle column	$26.88° \rightarrow 31.73°$, rising	line 8 — the counter-intuitive prediction, confirmed
the dashed line on the left panel	the $45°$ servo limit, never reached by this demand	§7-10 uses a demand that does reach it
$\mathbf{W}$ against $1000\mathbf{W}$, $2.81\text{e}{-15}$ N	the same solution to solver precision	line 4 — a common factor cancels
the right panel, 48.9% against 17.5%	the load does move between units	line 9 — units are interchangeable, components are not
the last column of the table	residual at machine precision throughout	the constraint $\mathbf{Bf} = \boldsymbol{\tau}_d$ is still exact

What to try	What to watch
`P.W = diag([1 1 1 1 1 1 1 1])` in the sweep	the first row of the table reappears: $\mathbf{W} = \mathbf{I}$ is §7-8 — line 3
price only unit 1, `diag([5 5 1 1 1 1 1 1])`	unit 1's thrust falls and the other three absorb it; $\Sigma\lvert f_y\rvert$ does not move
set the demand to `[80; 0; 20]` and sweep again	with $Y = 0$ the bound of line 6 is slack, and now the price **does** reduce the sideways thrust

> ✎ **In class**
>
> - **Purpose** — give the weight matrix a meaning that survives measurement, and show how to tune it.
> - **Point to** — the $\Sigma|f_y|$ column. Seven different price lists, one number.
> - **Ask** — "A reference implementation prices f_y a thousand times above f_x to keep the servos near zero. What does the table say it achieves?" A worse worst angle, on this geometry.
> - **Take away** — a weight redistributes among genuine alternatives. What one axis alone can supply is not an alternative.

7-10. The servo-angle limit, and the shape of the feasible set

What is observed. The demand $\boldsymbol{\tau}_d = [15\ \mathrm{N};\ 70\ \mathrm{N};\ 10\ \mathrm{N{\cdot}m}]$ pushes mostly sideways. The weighted minimum norm answers it with a worst servo angle of $85.68°$. No servo on this hull turns past $45°$. The closed form has produced a command the hardware cannot execute, and the question of what to do instead is the question of this section.

The result. The limit is written as a constraint, the constrained problem is solved by a quadratic program, and the shape of the constraint set decides how many quadratic programs are needed. The honest answer for this hull is sixteen, and the reason is worth more than the number.

Derivation.

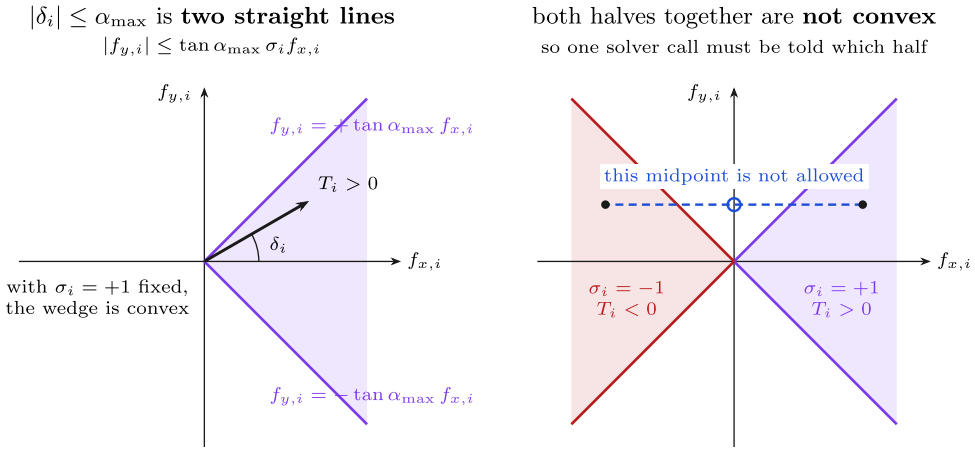
1. State the limit on the quantity the hardware actually limits:

$$|\delta_i| \le \alpha_{\max}, \qquad \delta_i = \operatorname{atan2}(\sigma_i f_{y,i},\ \sigma_i f_{x,i}), \qquad \alpha_{\max} = 45°$$

2. As written this is nonlinear, which is why a reference implementation reaches for a general nonlinear solver. It need not be. Taking the tangent of both sides, with the per-unit thrust sign σ_i **held fixed**, turns it into two linear inequalities:

$$|f_{y,i}| \le \tan(\alpha_{\max})\,\sigma_i f_{x,i} \quad \Longleftrightarrow \quad \begin{cases} f_{y,i} - \tan(\alpha_{\max})\,\sigma_i f_{x,i} \le 0 \\ -f_{y,i} - \tan(\alpha_{\max})\,\sigma_i f_{x,i} \le 0 \end{cases}$$

3. Two inequalities per unit, eight rows in all, assembled into $\mathbf{A}(\boldsymbol{\sigma})\mathbf{f} \le \mathbf{0}$. For $\alpha_{\max} = 45°$ the tangent is 1 and the two boundary lines are simply $f_{y,i} = \pm f_{x,i}$.



one unit has 2 halves, so four units have $2^4 = 16$ convex pieces. Guessing one piece is fast but can be wrong, or infeasible when the demand would in fact fit in another piece; solving all sixteen returns the true optimum.

4. **The set is convex only after the sign is fixed.** The figure's right panel draws the proof: take a point in the forward wedge and a point in the reverse wedge, and the straight segment between them passes through the sideways region that both wedges exclude. A convex solver cannot represent the union; it can only be told which half to work in.

5. The union of the two half-cones, over four units, is covered **exactly** by $2^4 = 16$ convex subsets, one for each sign combination $\boldsymbol{\sigma} \in \{+1, -1\}^4$. This is a complete description, not an approximation — every feasible command lies in at least one of them.
6. **The angle limit is fixed by the hardware; the force is what the controller asked for.** Requiring $\mathbf{Bf} = \boldsymbol{\tau}_d$ while also enforcing the angle limit can be infeasible, and an infeasible problem returns nothing at all, which is useless to a controller running at 50 Hz. The angle limit cannot be relaxed, so the demand is moved into the cost, where it can be approached rather than met:

$$\min_{\mathbf{f}} \ \left\| \mathbf{Q}\left(\mathbf{Bf} - \boldsymbol{\tau}_d\right) \right\|^2 + \varepsilon \left\| \mathbf{W}^{1/2}\mathbf{f} \right\|^2 \quad \text{subject to} \quad \mathbf{A}(\boldsymbol{\sigma})\,\mathbf{f} \le \mathbf{0}$$

7. This is the **weighted least-squares** form of control allocation. The first term asks for the demanded force, the second keeps the command small among the many that achieve the first equally well, and $\varepsilon \ll 1$ states which of the two matters. Because the feasible set always contains $\mathbf{f} = \mathbf{0}$, a solution always exists.
8. Both terms are quadratic in $\mathbf{f}$ and the constraints are linear, so each of the sixteen subsets is a convex **quadratic program** — the class of problem for which `quadprog` and `lsqlin` are written, and for which a global optimum is guaranteed in finitely many steps.
9. Three ways to use this, and what each costs:

$$\textbf{clip} \longrightarrow \textbf{one sign combination} \longrightarrow \textbf{all sixteen sign combinations}$$

- **Clip.** Solve unconstrained, then fold each angle back onto the limit. Always returns something, never exceeds the limit, but the clipped command achieves a different force than the one demanded, and nothing controls the error.
- **One sign combination.** Take $\boldsymbol{\sigma}$ from the weighted minimum-norm solution and solve that single quadratic program. Fast, and correct whenever that combination admits a solution. Line 4 is the reason it may not, and line 10 is the measurement.
- **All sixteen sign combinations.** Solve all sixteen and keep the cheapest feasible one. The result is the true optimum over the exact feasible set, at sixteen times the cost.

10. The choice of sign combination is not a technicality. For the demand of this section the weighted minimum norm gives $\boldsymbol{\sigma} = [+1\ +1\ -1\ -1]$, and **that combination has no feasible point**: `quadprog` exits -2. The best of the sixteen is $\boldsymbol{\sigma} = [-1\ -1\ -1\ -1]$ at cost 1.3950, which achieves the demand to 0.020 N. The demand was reachable all along, under a sign combination the closed form never produced.
11. The thrust cap is still handled by the scalar ρ of §7-8 line 4 after the solve, and for the same reason: it scales $\boldsymbol{\tau}$ without turning it.

Symbol	Quantity	Value or source
$\boldsymbol{\sigma}$	per-unit thrust sign, one of 16 combinations	± 1, `info.sigma`
$\mathbf{A}(\boldsymbol{\sigma})$	the eight angle-limit rows of line 3	8×8, rebuilt for each combination
$\mathbf{Q}$	relative importance of X, Y, N	3×3, $\mathbf{I}$ here
ε	weight of the regularising term	10^{-4}, `P.eps_reg`
$\tan \alpha_{\max}$	slope of each boundary line	1.0 at $45°$
cost	the objective value of line 6 at the optimum	`info.cost`

★ Which MathWorks solver, and why

All three functions below are in Optimization Toolbox, and the course uses version 24.2 on R2024b.

Function	What it is given	Documentation
`lsqlin`	a residual $\|\mathbf{Cf}-\mathbf{d}\|^2$ and linear constraints	the form of line 6, written directly
`quadprog`	an explicit $\frac{1}{2}\mathbf{f}^\top\mathbf{Hf}+\mathbf{c}^\top\mathbf{f}$ and linear constraints	used here with $\mathbf{Bf}=\boldsymbol{\tau}_d$ as a hard equality
`fmincon`	a general nonlinear cost and nonlinear constraints	what line 1 needs if line 2 is not noticed
`optimoptions`	solver settings, `'Display','off'` among them	built once and reused, §7-11

`lsqlin` and `quadprog` solve the **same** problem here; `lsqlin` is preferred because line 6 is already a least-squares residual and needs no squaring by hand. `fmincon` is the honest comparison point: it is given the nonlinear constraint of line 1 rather than the linear one of line 2, and §7-11 measures what that costs. See also the toolbox's own quadratic programming overview.

Experiment 7-10 · Five answers to one impossible command (25 min)

```
W07_CA(10)
```

```
=== 7-10  the servo-angle limit ===
a demand that exceeds it,  tau = [15 N, 70 N, 10 N m]
  (the sideways force is larger than the forward one -> the angle grows)

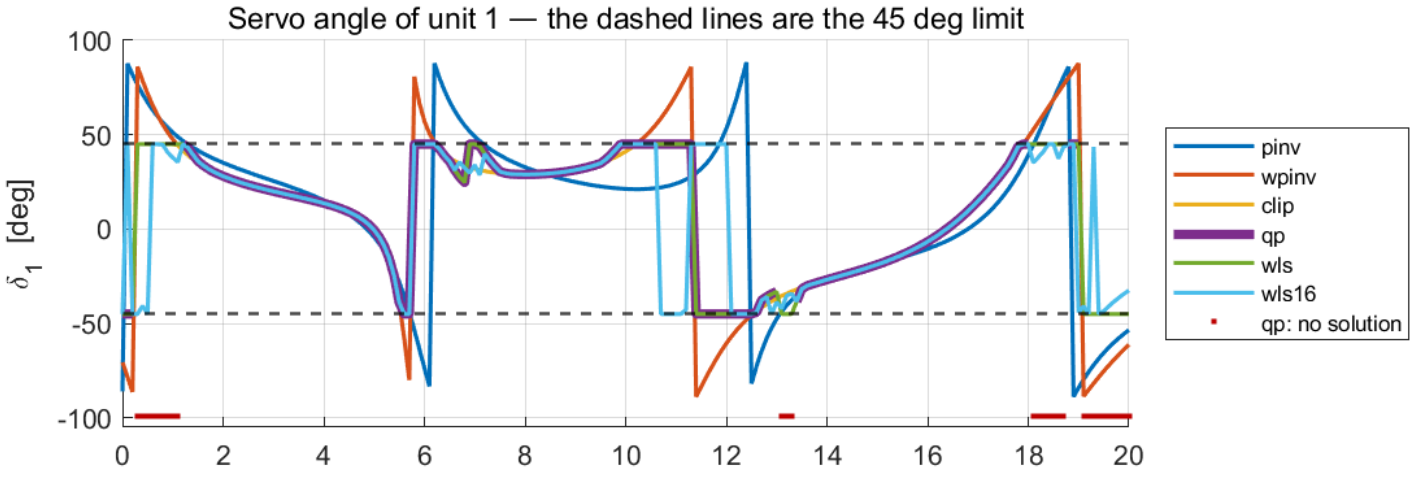
  method       max|delta| [deg]   ||B f - tau|| [N]   ||f|| [N]   QPs   exit
  wpinv               85.68              0.000        36.14      0      1
  clip                45.00             40.728        36.14      0      1
  qp                  83.78              0.000        36.01      1     -2
  wls                 45.00             38.891        30.05      1      1
  wls16               45.00              0.020        53.17     16      1

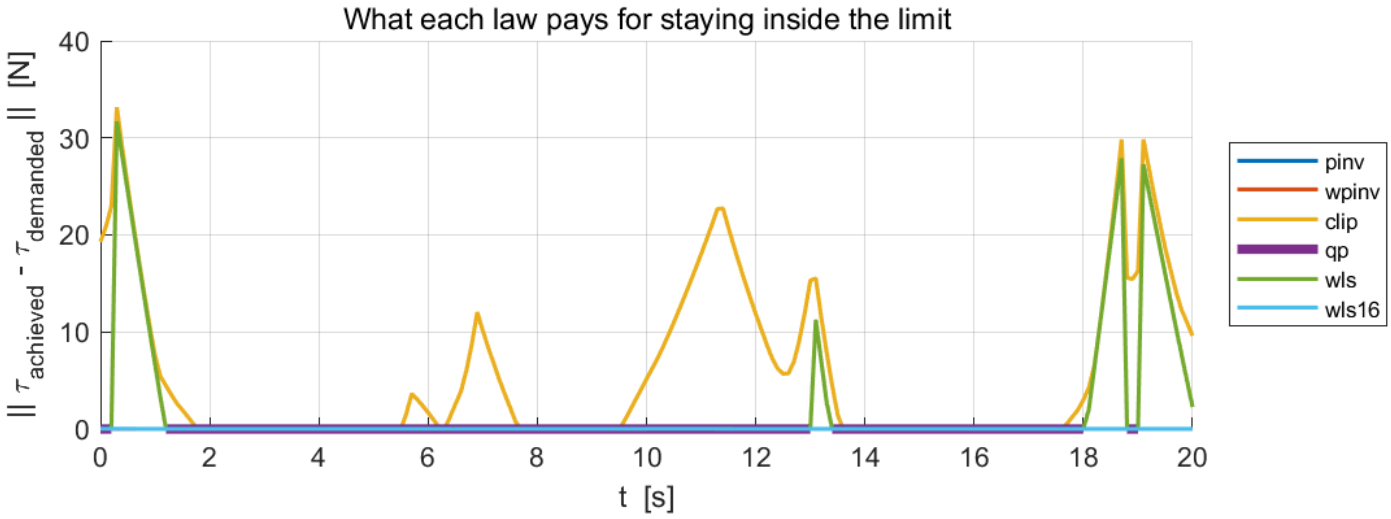
sign combination from the closed form  sigma = [ 1  1 -1 -1]  -> quadprog exit -2
best of the sixteen combinations       sigma = [-1 -1 -1 -1]  -> cost 1.3950
  -> the demand is feasible, but not with the sign combination the
     weighted minimum norm produced

the demand  tau = [100 sin0.5t ; 60 cos0.3t ; 30 sin0.7t],  t = 0..20 s, 201 points

  method     solved    past limit   max|delta|   mean |tau err|   max |tau err|   rho<1
           [% of pts]    [% of pts]       [deg]             [N]             [N] [% of pts]
  pinv         100.0          43.8        89.61           0.000           0.000        0.0
  wpinv        100.0          50.7        89.38           0.000           0.000        0.0
  clip         100.0           0.0        45.00           5.570          33.158        0.0
  qp            85.6           0.0        45.00           0.000           0.000        0.0
  wls          100.0           0.0        45.00           2.135          31.688        0.0
  wls16        100.0           0.0        45.00           0.010           0.018        0.0

  (the limit and error statistics are taken only where a solution was
   found -- qp fails to find one at some of the points)
```





Where to look	What is there	Which line predicts it
`wpinv` at $85.68°$	the closed form has no angle limit in it	§7-9 has no constraint in it
`clip` at $45.00°$ and 40.728 N	inside the limit, but the force is no longer the demanded one	line 9, first bullet
`qp` exit -2	the chosen sign combination admits **no** feasible point	line 4 — a convex solver sees one subset only
`wls16` at 0.020 N	the demand was reachable under a different sign combination	line 10 — the closed form never produced it
`wls` at 38.891 N against `wls16` at 0.020 N	same solver, same cost function, sixteen combinations instead of one	line 9, second and third bullets
the `solved` column, `qp` at 85.6%	one in seven instants of the trajectory has no answer	line 5 — the union is not covered by any single subset
`pinv` and `wpinv`, past limit 43.8% and 50.7%	half the trajectory asks for angles the servo cannot reach	the angle limit is set by the hardware, not by preference
the upper panel, flat tops at $\pm 45°$	every constrained law holds at the limit rather than passing it	line 3 — the constraint is active there
`wls16` mean error 0.010 N	not exactly zero	the ϵ term of line 6, not the angle limit

What to try	What to watch
`P.eps_reg = 1e-8`	the `wls16` residual falls further; the command becomes less well conditioned in exchange
`P.Q = diag([1 1 5])`	yaw is now bought before sway, and the force error moves from N into Y
`P.alpha_max = deg2rad(90)`	every method agrees, because the angle limit no longer binds and line 1 is vacuous
in `W07_alloc`, make `'wls16'` try only combinations $0..7$	the combination of line 10 is excluded and the residual jumps — the enumeration is not padding

> ✎ **In class**
>
> - **Purpose** — show that a hard limit changes the problem's **shape**, not just its numbers.
> - **Point to** — `qp` exit -2 beside `wls16` at 0.020 N. Same limit, same demand, opposite verdicts.
> - **Ask** — "Why not simply put a large weight on f_y instead of a constraint?" §7-9 line 8 measured that: the worst angle gets worse.
> - **Take away** — the angle limit is fixed by the hardware, the force demand is what the controller asked for, and the sign combination is a discrete choice that no convex solver will make.

7-11. What runs at 50 Hz

What is observed. A controller sampling at 50 Hz has 20 ms between demands. Allocation is one stage inside that budget, sharing it with the control law, the filters and the logging. A method that is correct and too slow has not solved the problem.

The result. The closed forms are two to three orders of magnitude faster than any solver, because the only matrix inversion in them does not depend on the demand and can be done once, before the loop starts.

Derivation.

1. In the weighted closed form of §7-9 line 2, every factor except $\boldsymbol{\tau}_d$ is built from $\mathbf{B}$ and $\mathbf{W}$, both of which are fixed by the hull and the design. Group them:

$$\mathbf{K}_W = \mathbf{W}^{-1}\mathbf{B}^{\top}\left(\mathbf{B}\mathbf{W}^{-1}\mathbf{B}^{\top}\right)^{-1} \in \mathbb{R}^{8\times 3}, \qquad \mathbf{f} = \mathbf{K}_W\,\boldsymbol{\tau}_d$$

2. $\mathbf{K}_W$ is computed **once**, offline. What the loop executes is one 8×3 matrix times a 3×1 vector: 24 multiplications and 16 additions, with no division, no factorisation and no iteration.
3. A solver cannot be precomputed in this way, because which constraints are active depends on $\boldsymbol{\tau}_d$ and therefore changes every step. An active-set method walks from one candidate set to another; the number of steps is bounded but not fixed, so the **worst** case, not the mean, is what a real-time budget must accommodate.
4. The sixteen-combination method of §7-10 multiplies that cost by sixteen. Whether this is affordable is a measurement, not an opinion, and Experiment 7-11 makes it.
5. The nonlinear form of §7-10 line 1 is the slowest of all, because `fmincon` must estimate gradients of an $\operatorname{atan2}$ constraint by finite differences at every iteration. Noticing that line 2 makes the same constraint linear is worth more than any amount of solver tuning.

Symbol	Quantity	Value or source
$\mathbf{K}_W$	precomputed allocation matrix	8×3, built once
h_c	controller period at 50 Hz	20 ms $= 20\,000$ µs
QPs	quadratic programs solved per demand	0, 1 or 16

Experiment 7-11 · The 50 Hz budget, measured (15 min)

```
W07_CA(11)
```

```
=== 7-11  the time one solve takes ===
101 demands allocated one after another.  the 50 Hz budget is 20000 us

  method       QPs   mean [us]   max [us]   x pinv   budget used [%]
  pinv           0        13.1      227.6        1              0.07
  wpinv          0         7.7       19.8        1              0.04
  clip           0         7.9       16.7        1              0.04
  qp             1       436.5     1879.0       33              2.18
  wls            1       179.6     1075.9       14              0.90
  wls16         16      2558.5     2845.0      195             12.79
  fmincon        1      5315.7    40035.1      406             26.58

  with one 8x3 matrix prepared in advance : 0.94 us  (24 multiplies)
  distance between Kw*tau and wpinv       : 5.62e-15 N
```

Where to look	What is there	Which line predicts it
`wpinv` at 7.7 µs mean	a closed form, with the inverse still formed inside the call	§7-9 line 2
the precomputed matrix at 0.94 µs	eight times faster again, for the same answer	line 2 — 24 multiplications and nothing else
the difference $5.62\text{e}{-15}$ N	precomputing changes the cost, not the result	line 1 — the grouping is algebraic
`qp` and `wls`, mean against max	$436.5 \to 1879.0$ µs and $179.6 \to 1075.9$ µs	line 3 — the active set is demand-dependent
`wls16` at 12.79% of the budget	sixteen combinations fit, with room for the rest of the loop	line 4 — affordable, and measured to be so
`fmincon` worst case $40\,035$ µs	**twice** the $20\,000$ µs deadline, on its own	line 5 — finite differences on an $\operatorname{atan2}$
the `x pinv` column	1, 1, 1, 33, 14, 195, 406	the three orders of magnitude between a formula and a solver
`pinv` at 13.1 µs mean but 227.6 µs max	the first call of any method pays for the library being loaded	the `max` column is a tail, not a typical cost

> **⚠ A timing is a measurement of one machine**
>
> These figures were taken on the laboratory machine with MATLAB R2024b and nothing else running, and they move by tens of per cent between runs. What does **not** move is the ordering and the orders of magnitude: a closed form is microseconds, one quadratic program is hundreds of microseconds, sixteen are milliseconds, and a nonlinear solver on the same problem overruns a 50 Hz budget. A deployed allocator must be timed on its own target, under load, and judged on its worst case.

What to try	What to watch
raise the demand count to 1001 points	the mean settles; the maximum does not, because it is a tail
time `Kw*tau` against `wpinv` inside a tight loop	the gap is the cost of forming $(\mathbf{B}\mathbf{W}^{-1}\mathbf{B}^{\top})^{-1}$ every step
add `'Algorithm','active-set'` to the `quadprog` options	the mean moves; the conclusion of line 3 does not

> **✎ In class**
>
> - **Purpose** — make real-time feasibility a measured quantity rather than an assertion.
> - **Point to** — the two numbers 0.94 µs and $40\,035$ µs. Both solve a version of the same problem.
> - **Ask** — "Which column matters for a 50 Hz loop, mean or max?" The max, every time; a deadline missed once is a deadline missed.
> - **Take away** — the fastest allocator and the most correct allocator are not the same one, and the choice is now a numbered table rather than a preference.

7-12. The second limit, and what the hull actually receives

What is observed. Section 7-10 solved the servo-angle limit and left the thrust cap to a single scalar ρ applied after the solve. Section 7-11 timed every law. Neither checked the answer against the hull. When the 201-point demand of §7-10 is allocated by `wls16` — which honours the $45°$ angle limit at every instant — and the eight components are turned into four shaft speeds, unit 3 is asked for -115 rad/s against a limit of -101.74 rad/s. The motion model clips it, at 41.8% of the sweep, and the force the hull receives is then not the force the allocator reported.

The result. The shaft-speed limit is not a second kind of constraint. It is the thrust cap seen through the thrust curve, and in each unit's own axes the two limits together bound a circular **sector**: the angle is the servo limit, the radius is the shaft-speed limit. Replacing the arc of that sector by M inscribed chords adds M linear rows per unit to the sixteen sign combinations of §7-10 and makes the feasible set exact in both limits. Measured out of the motion model, the mean force error over the same sweep falls from 7.353 N to 0.012 N, for 17.89% of the 50 Hz budget instead of 12.80%.

Derivation.

1. Write the whole chain once, because each section so far has looked at a single link of it:

$$\boldsymbol{\tau}_d \xrightarrow{\text{allocation}} \mathbf{f} \in \mathbb{R}^8 \xrightarrow{\text{per unit}} (T_i,\ \delta_i) \xrightarrow{\text{thrust curve}} (n_i,\ \delta_i) \xrightarrow{\text{hardware}} \boldsymbol{\tau}_{\text{act}}$$

2. The two middle arrows are §7-8 lines 1 to 4, repeated here because this section inverts them:

$$T_i = \sigma_i \sqrt{f_{x,i}^2 + f_{y,i}^2}, \qquad \delta_i = \operatorname{atan2}(\sigma_i f_{y,i},\ \sigma_i f_{x,i}), \qquad n_i = \operatorname{sign}(T_i)\sqrt{\frac{|T_i|}{k_i}}$$

with $k_i = k_{\text{pos}}$ when $T_i \geq 0$ and $k_i = k_{\text{neg}}$ otherwise.

3. Impose the shaft-speed limit on the last of those. The square root is monotone, so the limit passes straight through it:

$$|n_i| \le n_{\max} \quad \Longleftrightarrow \quad |T_i| \le k_i\, n_{\max}^2$$

The rpm saturation **is** a thrust cap. It is not a separate kind of limit; it is the same limit seen through the curve of line 2.

4. The two caps are not equal, because the propeller is not symmetric. Astern uses a different coefficient from ahead:

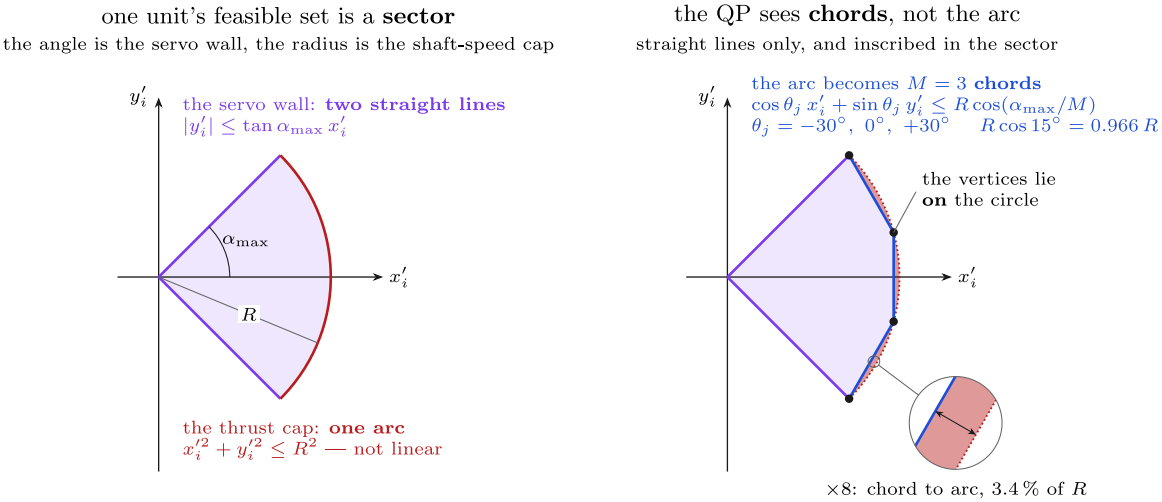
$$T^+ = k_{\text{pos}}\, n_{\max}^2 = 0.011080 \times 103.93^2 = 119.682 \text{ N}, \qquad T^- = k_{\text{neg}}\, n_{\min}^2 = 0.006445 \times 101.74^2 = 66.708 \text{ N}$$

The astern cap is 56% of the ahead cap. For a pure surge demand, units 3 and 4 push astern against the smaller of the two, so **the reverse cap is the one that binds** even though the ahead cap is the number usually quoted. At $\delta_i = 0$ the whole hull can therefore produce at most $X = 2(119.68) + 2(66.71) = 372.78$ N.

5. Put both limits in the same picture. In the unit axes $x_i' = \sigma_i f_{x,i}$ and $y_i' = \sigma_i f_{y,i}$ of §7-10 line 2, the servo limit and the thrust cap read

$$|y_i'| \le \tan(\alpha_{\max})\, x_i' \qquad \text{and} \qquad x_i'^2 + y_i'^2 \le R_i^2, \qquad R_i = \begin{cases} T^+ & \sigma_i = +1 \\ T^- & \sigma_i = -1 \end{cases}$$

Two straight lines and one arc: the feasible set of a single unit is a circular **sector**.



In unit axes $x_i' = \sigma_i f_{x,i}$ and $y_i' = \sigma_i f_{y,i}$, so one picture serves a unit pushing ahead and one pushing astern. Only R differs: ahead $R = k_{\text{pos}} n_{\max}^2 = 119.68$ N, astern $R = k_{\text{neg}} n_{\min}^2 = 66.71$ N, which is 56 % of it, so the astern cap is the wall that binds. `W12_alloc` builds exactly these rows in `cone_cap`: two for the wall and M for the cap, for each of the four units.

6. A quadratic program accepts linear rows only, and the arc is quadratic. Cut it with M chords, the angle divided evenly:

$$\cos\theta_j\, x_i' + \sin\theta_j\, y_i' \le R_i \cos\!\left(\frac{\alpha_{\max}}{M}\right), \qquad \theta_j = -\alpha_{\max} + \left(j - \tfrac{1}{2}\right)\frac{2\alpha_{\max}}{M}, \qquad j = 1, \dots, M$$

For $\alpha_{\max} = 45°$ and $M = 3$ the normals stand at $-30°$, $0°$ and $+30°$, and the right-hand side is $R_i \cos 15° = 0.9659\, R_i$.

7. **The polygon is inscribed, so it can never break the cap.** Two neighbouring chords are $2\alpha_{\max}/M = 30°$ apart and each stands off by $R\cos 15°$, so they meet at $R\cos 15°/\cos 15° = R$ — exactly on the circle. Only the chord midpoints sit at $0.9659\, R$, which is 3.41% short. The approximation is therefore conservative everywhere and worst at three isolated angles, and raising M shrinks that figure as $1 - \cos(\alpha_{\max}/M)$ at the cost of M rows per unit.

8. Assembling line 5 and line 6 for all four units gives $4(2+M)=20$ rows for $M=3$, against the 8 rows of §7-10. The sixteen sign combinations are unchanged — $\boldsymbol{\sigma}$ still decides which half-cone and, now, which of the two cap heights applies.
9. **Why the scalar ρ of §7-10 line 11 is not enough.** It multiplies the whole of $\mathbf{f}$ by one number, so $\mathbf{B}(\rho\mathbf{f})=\rho\boldsymbol{\tau}$: the direction of the demand survives and its magnitude does not, including in the components no unit was straining. One saturated unit therefore drags all three others down with it. The chord rows do the opposite — they tell the solver which unit is at its cap, and the optimum moves load onto the units that are not. Table (1) measures the difference on a demand of 300 N: 16.584 N of error with ρ, 0.009 N with the chords.
10. **The error must be measured out of the motion model, not out of the allocator.** `otter_tilt4` applies its own limits to $\boldsymbol{n}$ and $\boldsymbol{\delta}$ before computing anything, so an out-of-limit command produces a different hull force and nothing in the allocator's own bookkeeping records it. The actual force is recovered by calling the model twice in the same state, once with the thrusters commanded and once with $\boldsymbol{n}=\mathbf{0}$, exactly as §7-2 did:

$$\boldsymbol{\tau}_{\text{act}} = \mathbf{M}\left(\dot{\boldsymbol{\nu}}_{\text{on}} - \dot{\boldsymbol{\nu}}_{\text{off}}\right)\Big|_{\text{rows }1,2,6}$$

11. Line 10 makes a prediction that can be checked: **clipping inside the allocator and clipping inside the hardware are the same operation.** A law that ignores the servo limit entirely and a law that enforces it by clipping must therefore deliver the *same* force, differing only in whether the allocator knows what it produced. Table (2) measures 9.049 N mean and 31.452 N maximum for `wpinv` and for `clip` alike — identical to three decimals. The value of enforcing a limit inside the allocation is not that it changes the clipping; it is that it lets the remaining freedom be spent before the clipping happens.
12. Every link of line 1 has now been derived, so the whole allocator can be written down in one place. Lines 1 to 11 of the listing are the optimisation of §7-10 and §7-12; lines 12 to 18 are the per-unit conversion of §7-8, including the angle fold of §7-8 item 2-a; lines 19 to 21 are the hardware, which acts whether or not the allocator anticipated it.

```
Algorithm 1   Force-to-input allocation under the servo and shaft-speed limits

Require: B (3x8), tau_d (3x1), W (8x8), Q (3x3), eps,
         alpha_max, k_pos, k_neg, n_max, n_min, M
Ensure:  n (4x1) [rad/s], delta (4x1) [rad], tau_act (3x1) [N, N, N m]

 1: R_pos <- k_pos * n_max^2                    // ahead thrust cap        (line 4)
 2: R_neg <- k_neg * n_min^2                     // astern thrust cap       (line 4)
 3: J_best <- inf
 4: for each sigma in {+1,-1}^4 do               // 16 sign combinations  (7-10 l.5)
 5:     A <- [] ; b <- []
 6:     for i = 1..4 do
 7:         R_i <- R_pos if sigma_i = +1 else R_neg
 8:         append  +-y'_i - tan(alpha_max) * x'_i <= 0        // servo limit, 2 rows
 9:         append  cos(th_j) x'_i + sin(th_j) y'_i <= R_i cos(alpha_max/M),
                   th_j = -alpha_max + (j-1/2) * 2 alpha_max/M,  j = 1..M
10:     f, J, exit <- argmin ||Q (B f - tau_d)||^2 + eps ||W f||^2  s.t. A f <= b
11:     if exit > 0 and J < J_best then  f_best <- f ;  J_best <- J
12: for i = 1..4 do                              // one unit at a time      (7-8)
13:     sigma_i <- sign(f_x,i) ;  if sigma_i = 0 then sigma_i <- +1
14:     T_i     <- sigma_i * hypot(f_x,i, f_y,i)
15:     delta_i <- atan2(sigma_i * f_y,i, sigma_i * f_x,i)   // the angle fold, 7-8 2-a
16:     k_i     <- k_pos if T_i >= 0 else k_neg
17:     n_i     <- sign(T_i) * sqrt(|T_i| / k_i)             // invert T = k n|n|
18: n     <- clip(n, n_min, n_max)               // the hardware acts here, not before
19: delta <- clip(delta, -alpha_max, +alpha_max)
20: tau_act <- M_rb ( nu_dot|on - nu_dot|off ) rows 1,2,6    // measured, not reported
```

In the pseudo-code	In the derivation	Note
lines 1 to 2	line 4	the two caps differ because $k_{\text{pos}} \neq k_{\text{neg}}$; the astern one binds
line 4	§7-10 line 5	the enumeration is exact, not a heuristic — skipping it is what `qp` does, and §7-10 measured the cost
lines 7 to 9	lines 5 to 7	$2 + M$ rows per unit: two for the angle, M for the cap
line 10	§7-10 lines 6 to 8	the angle limit is a constraint, the force demand is in the cost, so a solution always exists
line 11	§7-10 line 10	`exit > 0` is what makes the number a measurement (`standing-orders.md` §15-76)
line 13	§7-8 warning	`sign(0) = 0` would discard a unit asked for pure sway
line 15	§7-8 item 2-a	one line in place of the two angle-wrap branches, and algebraically the same map
lines 18 to 19	line 11	these two lines run inside `otter_tilt4` whether or not lines 4 to 11 respected the limits
line 20	line 10	the allocator's own residual is not the hull's; this is the one that counts

Dropping lines 4 to 9 and solving $\min \|\mathbf{W}\mathbf{f}\|$ subject to $\mathbf{B}\mathbf{f} = \boldsymbol{\tau}_d$ recovers §7-9's closed form; the remaining lines are unchanged. That is the sense in which the five laws of §7-10 are one algorithm with constraints added, not five

algorithms.

Symbol	Quantity	Value or source
k_{pos}, k_{neg}	thrust coefficients, ahead and astern	0.011080, 0.006445, `otter_config`
$n_{\max}$, $n_{\min}$	shaft-speed limits	$+103.93$, -101.74 rad/s
T^{+}, T^{-}	the two thrust caps of line 4	119.682 N, 66.708 N, `P.T_pos` and `P.T_neg`
R_i	cap height for unit i in its own axes	T^{+} or T^{-} by σ_i
M	chords replacing the arc	3, `P.n_chord`
θ_j	normal direction of chord j	$-30°$, $0°$, $+30°$ at $M = 3$
τ_{act}	force the motion model produced	line 10, `W07_chain`

> **Choosing M**
>
> $M = 1$ is a single flat cut across the sector and gives up 29% of the radius at the corners; $M = 3$ gives up 3.41%; $M = 6$ gives up 0.86%. Each step adds four rows to a problem already solved sixteen times, and §7-11's measurement says what that costs. $M = 3$ is used here because the thrust cap is itself a fitted quantity — k_{pos} and k_{neg} are regression coefficients, not exact numbers — and an approximation three times finer than the data it approximates buys nothing.

Experiment 7-12 · Following one demand all the way to the hull (25 min)

```
W07_CA(12)
```

```
=== 7-12  two limits: the servo angle and the shaft speed ===

  the shaft speed limits are n_max =  +103.93 and n_min =  -101.74 rad/s
  through T = k n|n| those become thrust limits
      ahead   T <= k_pos n_max^2 = 0.011 * 103.93^2 =  119.682 N
      astern  T <= k_neg n_min^2 = 0.006 * 101.74^2 =   66.708 N
  the astern limit is 56% of the ahead limit, and that is what binds
  at delta = 0 the hull can therefore make at most
      X = 2 * 119.68 + 2 * 66.71 = 372.78 N

  (1) one demand followed through the chain,  tau = [300 N, 0 N, 0 N m]

      pure surge, so units 1-2 push forward and units 3-4 must push astern
      the astern cap is the smaller one, and that is what binds

    law           T fwd  T astern    worst n  n clip  d clip    |tau err|
                    [N]       [N]    [rad/s] [units] [units]          [N]
    ------------------------------------------------------------------------
    wpinv         75.00    -75.00     -107.9       2       0       16.584
    clip          75.00    -75.00     -107.9       2       0       16.584
    wls16         75.00    -75.00     -107.9       2       0       16.588
    wls16cap      85.56    -64.43     -100.0       0       0        0.009
    ------------------------------------------------------------------------
    the caps are +119.68 N ahead and  -66.71 N astern

  (2) tau = [320 sin0.5t ; 60 cos0.3t ; 30 sin0.7t],  t = 0..20 s, 201 points

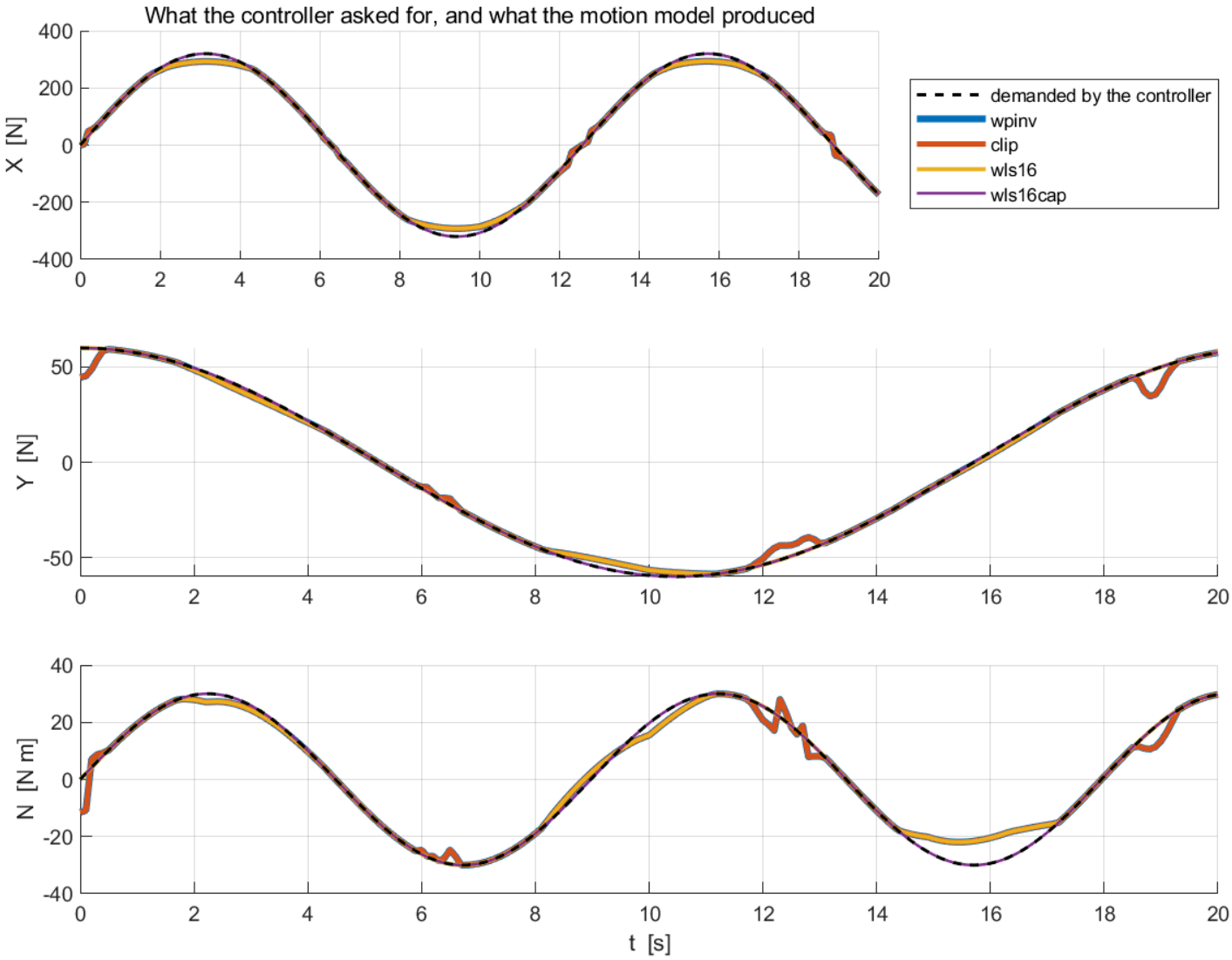
    law          max|d|  servo lim    rpm lim   mean |err|    max |err|  no sol
                  [deg]   [% of t]   [% of t]          [N]          [N]   [pts]
    ---------------------------------------------------------------------------
    wpinv         89.76       15.9       41.8        9.049       31.452       0
    clip          45.00        0.0       41.8        9.049       31.452       0
    wls16         45.00        0.0       41.8        7.353       28.361       0
    wls16cap      45.00        0.0        0.0        0.012        0.018       0
    ---------------------------------------------------------------------------
    the error is measured out of the motion model, not out of the allocator
    every statistic is taken over the points whose exitflag is positive

  (3) allocation time over the same 201 demands,  50 Hz budget = 20000 us

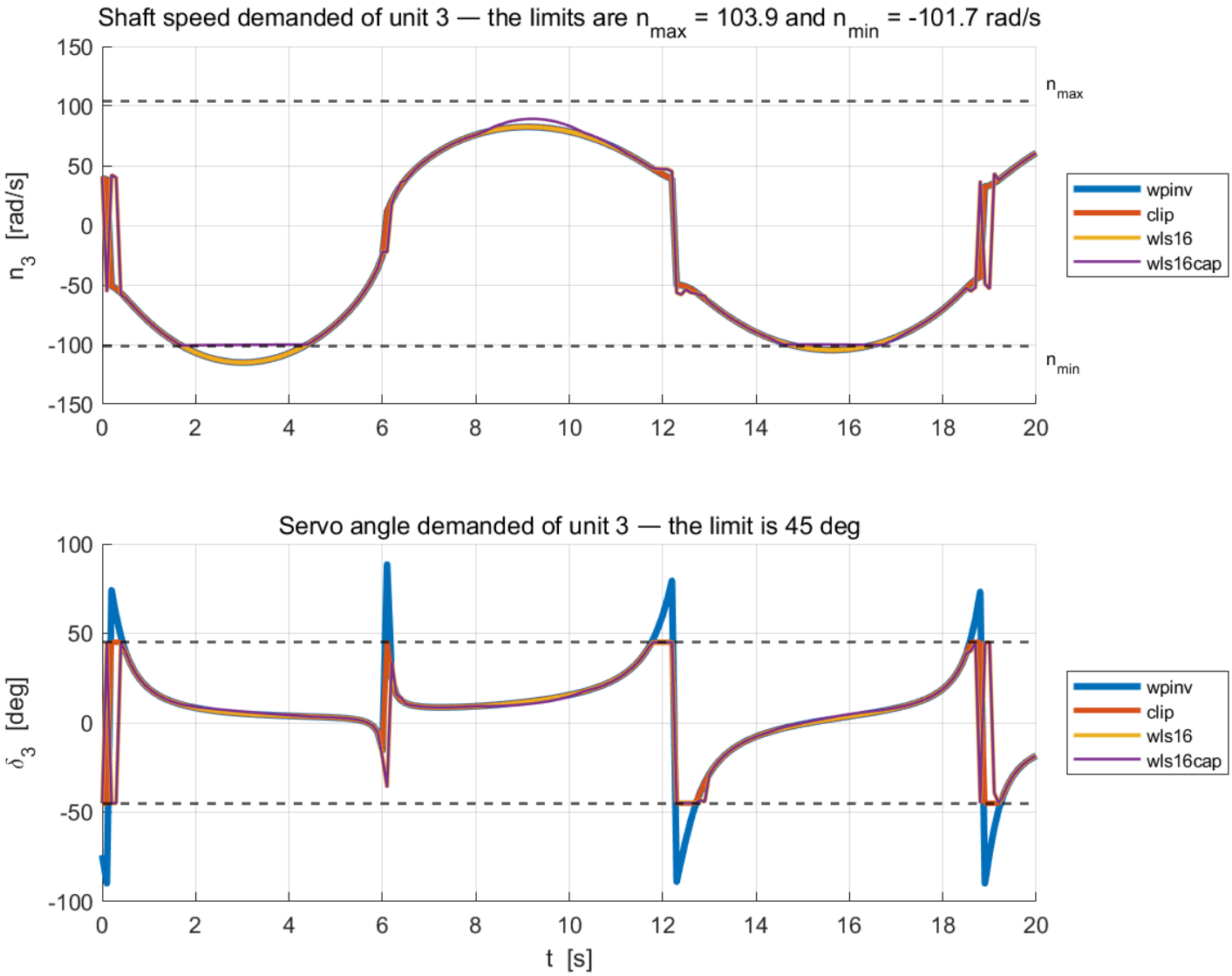
    law           QPs   mean [us]    max [us]  budget used [%]
    ----------------------------------------------------------
    wpinv           0         6.5       112.0             0.03
    clip            0         5.5        24.6             0.03
    wls16          16      2560.0      4495.9            12.80
    wls16cap       16      3577.7      4222.9            17.89
    ----------------------------------------------------------
    the cap adds 3 rows per unit, so wls16cap is wls16 plus that
```

Table (1) is the whole of lines 1 to 9 on one demand. The first three laws split 300 N into ± 75 N per unit and are correct to the column rule, yet two of the four units are driven past the shaft-speed limit, because 75 N astern exceeds the 66.708 N of line 4. Only `wls16cap` knows this before the command leaves the allocator: it shifts 10.56 N of the load from the astern pair onto the ahead pair, where the cap is nearly twice as high, and the demand is then met to 0.009 N.

The `no sol` column of table (2) counts the demands for which the solver returned a non-positive exit flag. It reads 0 for all four laws, so each statistic in the row is taken over all 201 points. That column is not decoration: `quadprog` returns its last iterate as an ordinary number even when the constraints cannot be met, and a mean taken without reading the flag would quietly average that number in. Where the column is not zero the figures below break at those instants rather than drawing through them.



Where to look	What is there	Which line predicts it
the purple `wls16cap` trace, all three panels	lies on the dashed demand everywhere	lines 6 to 8 — the feasible set is now exact in both limits
the orange `clip` trace around $t = 12$ s	departs from the demand in X, Y and N at once	line 9 — ρ scales the whole vector, so one saturated unit moves all three components
the blue halo under the orange trace	`wpinv` and `clip` coincide to the width of the line	line 11 — the plant clips where `clip` clips
the yellow `wls16` trace, $8 < t < 11$ s	inside the orange but still off the dashed line	line 3 — the servo limit is solved, the thrust cap is not
all four laws near $t = 0$ and $t = 20$ s	the largest departures, in N especially	line 4 — the demand turns through the astern cap there



Where to look	What is there	Which line predicts it
upper panel, $2 < t < 4.5$ s	three laws reach -115 rad/s, below the -101.74 dashed line	line 3 — the shaft-speed limit is the astern thrust cap, and it binds first
upper panel, the purple trace in the same window	flat along -101.74, never below	line 7 — the chords are inscribed, so the cap cannot be broken
upper panel, $8 < t < 10$ s	no law reaches $n_{\max} = 103.9$	line 4 — the ahead cap is 1.79 times the astern one
lower panel, the blue spikes at $\pm 90°$	`wpinv` commands twice the servo travel that exists	§7-10 line 1 — the closed form has no angle limit in it
lower panel, the other three traces	clipped flat at $\pm 45°$	line 11 — allocator clipping and hardware clipping meet at the same value
lower panel, where blue leaves the limit	the same instants at which X departs in the previous figure	line 9 — the direction error becomes a force error

Table (3) re-times the laws because `wls16cap` did not exist when §7-11 ran. The cap adds three rows per unit to a problem already solved sixteen times, and the measured cost of that is $2560.0 \to 3577.7$ µs in the mean: 40% more, for an error 600 times smaller. The worst case of 4222.9 µs is still under 22% of the $20\,000$ µs budget, so the exact answer is affordable at 50 Hz on this machine.

What to try	What to watch
set `P.n_chord = 1` and run again	the thrust cap is still respected, but the error rises: one flat cut gives up 29% of the radius
set `P.T_neg = P.T_pos` and run again	the `n clip` column of table (1) empties, which identifies the asymmetry of line 4 as the cause
raise the demand in `W07_tab_chain` from 320 N to 400 N	`wls16cap` begins to miss as well, because 400 N is past the 372.78 N of line 4
print `info.sigma` for `wls16` and `wls16cap` at the same instant	the two laws choose different sign patterns once the cap is in the problem

✎ In class

- **Purpose** — close the loop between the allocator's arithmetic and the force the hull receives.
- **Point to** — the `rpm lim` column of table (2): 41.8% for three laws and 0.0% for one.
- **Ask** — " `wpinv` and `clip` have exactly the same error. What did clipping buy?" Nothing, until the clipped limit is inside the optimisation.
- **Take away** — a limit enforced after the solve costs force in every component; the same limit enforced inside the solve costs milliseconds.

ℹ What this week does not do

Three things are deliberately left out, and each has a reason.

- **Rate limits on the servos.** The angle is constrained, its rate is not. A rate limit couples one instant to the next and makes allocation a problem over time rather than over one demand.
- **Singularity-free azimuth handling.** Nothing here prevents an angle from sweeping the long way round between two demands, because the sign pattern is chosen afresh each step.
- **Explicit optimal solutions computed offline.** The sixteen sign combinations of §7-10 are enumerated at run time. Pre-solving the parametric problem and storing the regions of the solution is the standard way to buy back the factor of sixteen.

7-13. The four laws inside one model

What is observed. Sections 7-8 to 7-12 put one demand at a time to an allocator in a script. A controller does not present its demand that way. The allocator sits between the control law and the hull, runs at every step of a continuous simulation, and the force it fails to deliver is integrated by the vessel rather than printed. One model holding all four laws, with a single constant choosing between them, turns the comparison into something that is run rather than read.

The result. On a pure sway demand the two closed forms solve $\mathbf{B}\mathbf{f} = \boldsymbol{\tau}_d$ **exactly** and still deliver only 28.3 N and 29.7 N of the 40 N asked, because the servo saturation that follows them is not in their arithmetic. Folding inside the allocator changes nothing the hull receives; putting the limit inside the solve recovers 39.989 N.

Derivation.

1. The model is the four-stage chain of §7-0 — command, allocation, plant, measurement — with one stage replaced. The command stage carries three Constants for X, Y, N and one Constant `law`; the plant is the same `otter_tilt4`; only the allocation stage is new.
2. The `allocate` block evaluates $\mathbf{f} = \texttt{W07_sl_alloc}(\boldsymbol{\tau}_d, \texttt{law}, \ldots)$, which maps the constant onto a law already derived earlier in the week:

`law`	Name drawn on the Scope	Method in `W07_alloc`	Derived in
1	minimum norm	`pinv`	§7-8
2	weighted minimum norm	`wpinv`	§7-9 line 2
3	weighted, folded onto the servo limit	`clip`	§7-10 line 9
4	servo limit and thrust cap as constraints	`wls16cap`	§7-12 lines 5 to 8

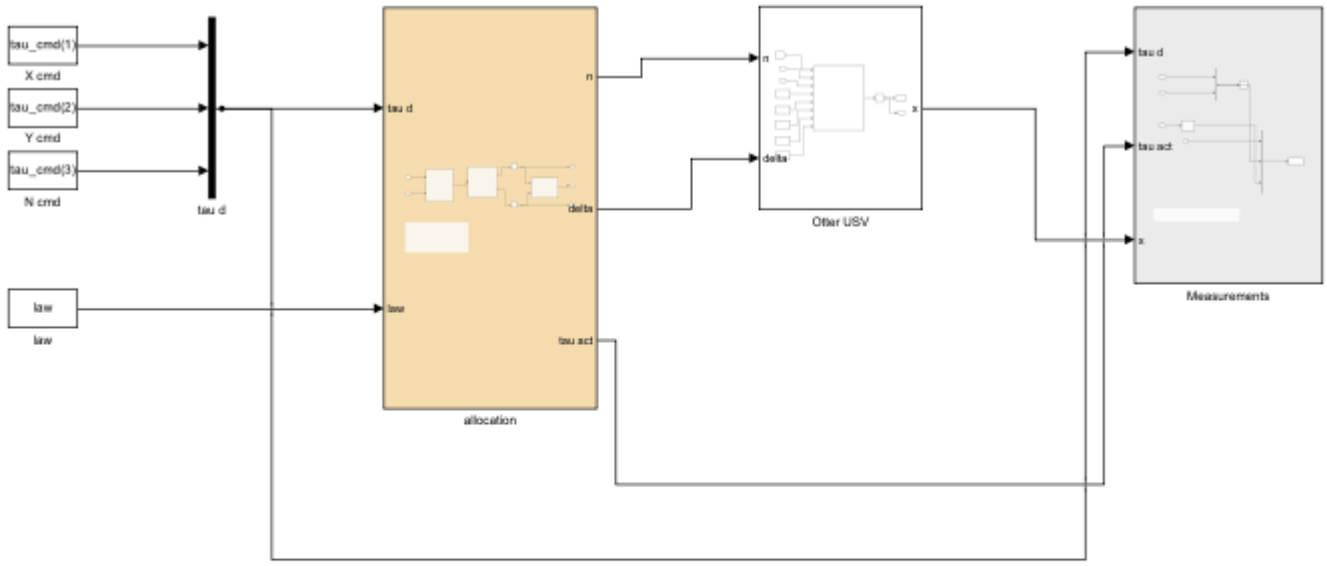
The index is taken as $k = \max(1, \min(4, \mathrm{round}(\mathtt{law})))$, so a value outside the range selects the nearest law instead of failing mid-run.

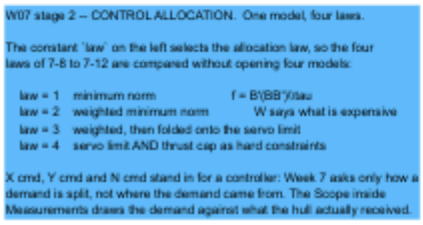
3. The `force to input` block is §7-8 lines 1 to 4 drawn as one block: the thrust sign $\sigma_i = \mathrm{sign}(f_{x,i})$, the magnitude $T_i = \sigma_i \|\mathbf{f}_i\|$, the folded angle $\delta_i = \mathrm{atan2}(\sigma_i f_{y,i},\ \sigma_i f_{x,i})$ and the shaft speed $n_i = \mathrm{sign}(T_i)\sqrt{|T_i|/k_i}$. The fold is inside this block, so every law reaches the servo with an angle already inside $[-90°, +90°]$.
4. The two Saturation blocks come **after** that, not before it: `shaft limit` clips n into $[n_{\min}, n_{\max}]$ and `servo limit` clips δ into $\pm\alpha_{\max}$. Their position in the diagram is the whole point of the section — an allocator that does not carry a limit cannot know that its answer was clipped.
5. The `force delivered` block recomputes $\boldsymbol{\tau}_{\mathrm{act}} = \mathbf{B}\,\mathbf{f}(n_{\mathrm{clipped}}, \delta_{\mathrm{clipped}})$ by §7-12 line 10 and sends it to the Scope beside $\boldsymbol{\tau}_d$. The gap is therefore read off a figure, not inferred from a residual.
6. The demand is $\boldsymbol{\tau}_d = [0,\ 40\ \mathrm{N},\ 0]^{\top}$, held constant for the whole run. A pure sway is the hardest direction for a $\pm 45°$ servo: the ideal unit force is sideways, which is $\delta_i = 90°$, exactly twice the travel that exists.
7. Before the saturation, laws 1 and 2 are both exact — $\boldsymbol{\tau}_{\mathrm{act}} = \boldsymbol{\tau}_d$ to three decimals — at raw angles of $87.2°$ and $74.0°$. The weight matrix of §7-9 moved the angle by $13°$ and no further, which is §7-9's own conclusion restated: pricing f_y redistributes load, it does not buy angle.
8. After the saturation they are no longer exact, and they are no longer equal either: clipping $87.2°$ and clipping $74.0°$ leave different eight-component vectors. The delivered sway is 28.333 N and 29.677 N, and a yaw moment of -10.526 N m and -7.675 N m appears although none was asked for.
9. Law 3 clips to $45°$ inside the allocator, so the servo block finds nothing left to clip. It therefore reaches the hull as the **same** eight components as law 2, and delivers $\boldsymbol{\tau}_{\mathrm{act}}$ identical to three decimals. What folding buys is not force but agreement: the allocator's own output is what the hull receives, so a controller reading it is not misled.
10. Law 4 delivers 39.989 N of the 40 N. The residual 0.011 N is the inscribed-chord error of §7-12 line 7, not a saturation error, and it falls as `P.n_chord` rises.

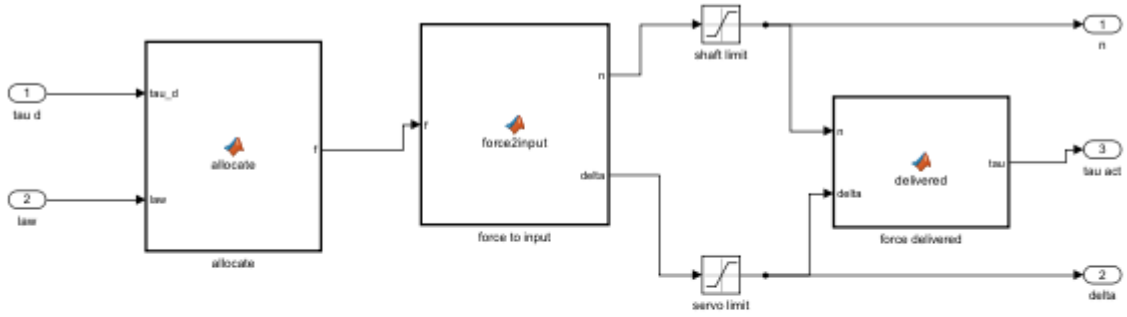
Symbol	Quantity	Value or source
`law`	Constant selecting the allocation law	1 to 4; set in `W07_0_setup`
$\boldsymbol{\tau}_d$	demanded generalised force	`tau_cmd` $= [0; 40; 0]$, N and N m
$\boldsymbol{\tau}_{\mathrm{act}}$	force recovered from the clipped inputs	$\mathbf{Bf}$, §7-12 line 10
$\alpha_{\max}$	servo travel	$45°$, `otter_config('quad_tilt')`

Experiment 7-13 · One model, four laws (20 min)

```
W07_2_build_alloc       % writes W07_alloc_chain.slx and the block-diagram PNGs
W07_0_setup             % puts every constant on the workspace
```







Allocation, and the hardware limits that follow it.

allocate one demand tau_d -> eight force components f, by the law the constant `law` names. All four laws are in W07_alloc.
force to input f -> four shaft speeds n and four tilt angles delta. The thrust sign is taken first, which is what folds every angle into [-90, +90] deg before the servo ever sees it.
shaft limit n clipped to [n_min, n_max].
servo limit delta clipped to +-alpha_lim.
force delivered the actuator model run forward from the CLIPPED values, so the Scope shows what the hull was given, not what the allocator wished for.

Changing the law means changing one number and pressing Run. `W07_plot_ca` is the model's `StopFcn`, so the table and the figure appear without a further command.

```
law = 1;  sim('W07_alloc_chain');
```

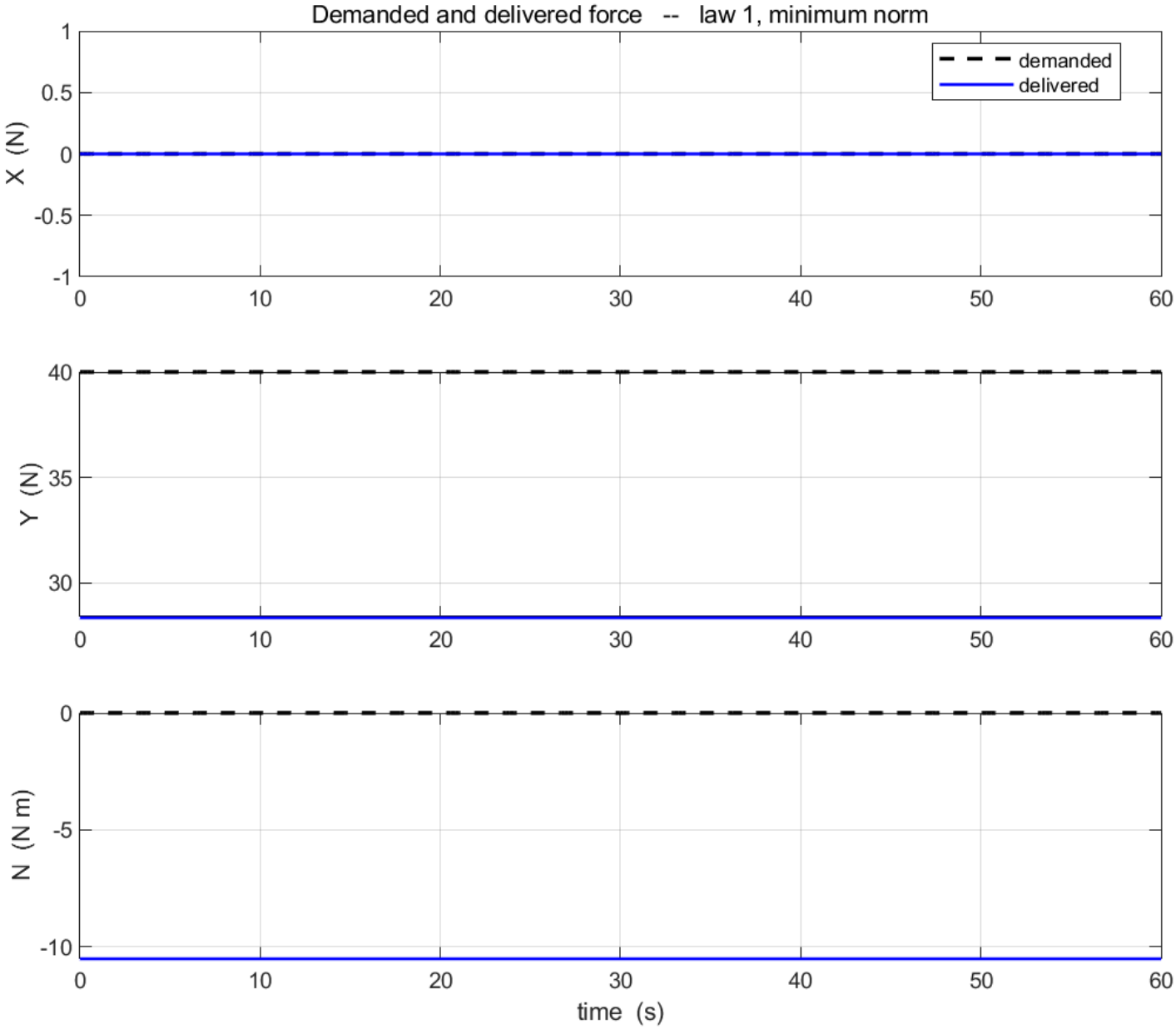
```
  law 1   minimum norm
  ------------------------------------------------------------
  component       demanded       delivered        max error
  X  (N)           0.000          0.000          0.000  at t =  0.00 s
  Y  (N)          40.000         28.333        -11.667  at t =  0.00 s
  N  (N m)         0.000        -10.526        -10.526  at t =  0.00 s
```

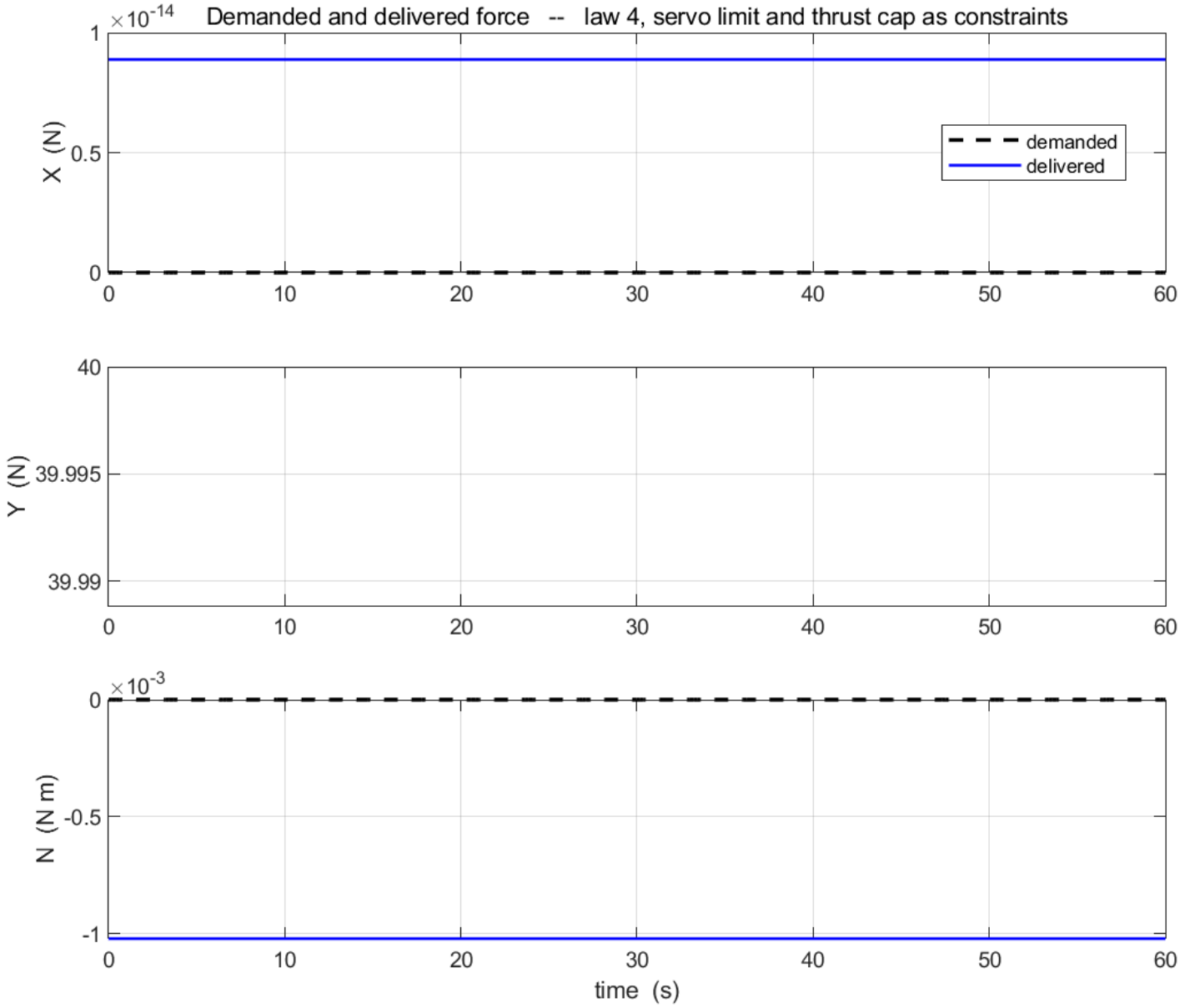
```
law = 4;  sim('W07_alloc_chain');
```

```
law 4   servo limit and thrust cap as constraints
------------------------------------------------------------
component      demanded      delivered        max error
X  (N)          0.000         0.000         0.000  at t =  0.00 s
Y  (N)         40.000        39.989        -0.011  at t =  0.00 s
N  (N m)        0.000        -0.001        -0.001  at t =  0.00 s
```

Collecting the Y and N rows of all four printed blocks, with the raw angle each law asked for before the servo block saw it:

law	Method	raw max $\lvert\delta_i\rvert$	Y delivered [N]	N delivered [N m]
1	`pinv`	$87.2°$	28.333	-10.526
2	`wpinv`	$74.0°$	29.677	-7.675
3	`clip`	$45.0°$	29.677	-7.675
4	`wls16cap`	$45.0°$	39.989	-0.001


Demanded and delivered force -- law 1, minimum norm
demanded
delivered
X (N)
Y (N)
N (N m)
time (s)



Where to look	What is there	Which line predicts it
law 1, middle panel	delivered sway flat at 28.3 N under a 40 N dashed demand	line 8 — the clip removes what the closed form assumed it could use
law 1, lower panel	-10.5 N m of yaw from a demand with none in it	line 8 — four units clipped unequally no longer cancel
laws 2 and 3, identical rows	the same 29.677 and -7.675	line 9 — folding moves where the clip happens, not what reaches the hull
laws 2 and 3, the angle column	$74.0°$ against $45.0°$	line 9 — only law 3's own output is what the hull received
law 4, all three panels	the two traces coincide	line 10 — the limit is a constraint of the solve, not a correction after it

What to try	What to watch
set `tau_cmd = [40; 0; 0]` and run all four laws	the four rows collapse onto one: a pure surge needs $\delta_i = 0$, and no law reaches a limit
print the raw angles with `f = W07_alloc(B, [0;40;0], 'wpinv', P)` and `atan2` on its pairs	the $74.0°$ of the table, before any Saturation block exists
set `law = 2.6`	law 3 runs, because the index is rounded and clamped by line 2
raise `tau_cmd` to $[0; 120; 0]$	law 4 misses as well: 120 N of sway is past what four units at $45°$ can make

> ✎ **In class**
>
> - **Purpose** — show the four laws as one model with one number changed, rather than four files.
> - **Point to** — laws 2 and 3 in the table: the same two delivered numbers, different raw angles.
> - **Ask** — "Laws 1 and 2 both solve $\mathbf{B}\mathbf{f} = \boldsymbol{\tau}_d$ exactly. Why does the hull get 28 N?" Because the servo block is downstream of the arithmetic.
> - **Take away** — the position of a saturation block in the diagram decides whether the allocator's answer is a prediction or a guess.

Part 2 · Laboratory run order

The experiments of Part 1 are worked through in order; this table is the index of what was run, for repeating the week at home.

Experiment	Model	Script	What it shows
7-0	`W07_openloop`	`W07_1_build_openloop`, `W07_0_setup`	the model written from code, and every constant
7-1	none	`verify_otter_tilt4`	four identities; $\delta = 0$ returns `otter.m`
7-3	`W07_openloop`	`W07_V_openloop` → `W07_tab_formula`	table (1) — force against the column rule
7-4	`W07_openloop`	`W07_V_openloop` → `W07_tab_track`	the three asymmetries, each against a row
7-5	`W07_openloop`	`W07_V_openloop` → `W07_plot_scenario`	the eight sheets `img/W07_result_S1.png` … `_S8.png`
7-6	`W07_openloop`	`W07_V_openloop` → `W07_tab_reint`	table (2) — the model against hand re-integration
7-8	none	`W07_CA(8)`	the pseudo-inverse, and the 37% a null-space solution costs
7-9	none	`W07_CA(9)`	the price sweep: what $\mathbf{W}$ moves and what it cannot
7-10	none	`W07_CA(10)`	five laws at the $45°$ servo limit, and the sixteen sign combinations
7-11	none	`W07_CA(11)`	the 50 Hz budget, measured for every law
7-12	none	`W07_CA(12)`	both limits, and the force the motion model produced
7-13	`W07_alloc_chain`	`W07_2_build_alloc` → `W07_0_setup`, then set `law` and Run	the same four laws inside one model, one constant apart

Selecting runs: `W07_V_openloop` for all eight, `W07_V_openloop('S5')` for one, `W07_V_openloop({'S5','S6'})` for a pair, `W07_V_openloop(1:4)` by number. A partial run writes only the sheets it ran and leaves the overlay file alone.

Sections 7-8 to 7-12 are all driven by the one script `W07_CA`, whose argument is the section number: `W07_CA(8)` prints §7-8 and draws its figure, `W07_CA(12)` prints §7-12 and draws its two. The five sections share one allocator and one hardware model, so they share one file rather than five. Everything `W07_CA` calls is named for what it does: `W07_alloc` holds the eight laws behind one `method` flag, `W07_hw` turns eight force components into four shaft speeds and four servo angles by §7-8 lines 1 to 4, `W07_chain` calls `otter_tilt4` to recover the force the hull actually received by §7-12 line 10, `W07_sweep_laws` and `W07_tab_chain` print the tables, `W07_plot_alloc`, `W07_plot_weights`, `W07_plot_limits` and `W07_plot_chain` draw the figures, and `W07_save_fig` writes every one of them to `img/`. The sections run in any order and in isolation, because the demand $\boldsymbol{\tau}_d$ is supplied by the script rather than by a controller.

Section 7-13 is where a controller's view of the same four laws is restored, and it is the only part of the allocation half with a model file. `W07_alloc_chain` is built by `W07_2_build_alloc`, filled by `W07_0_setup`, and switched between

laws by the single Constant `law`; `W07_sl_alloc` is the wrapper its MATLAB Function block calls, and `W07_plot_ca` is its `StopFcn`. Four laws therefore need one model, not four.

> **One run order, four sections**
>
> Sections 7-3 to 7-6 share the **same eight 30-s runs**: table (1) measures the force at the start of each, table (2) re-integrates the whole of each, table (3) reads the end of each, and `W07_plot_scenario` draws each. Splitting them would send the vessel out thirty-two times for eight trajectories, and the tables would then be read against runs only nominally the same. The runs happen once, in `W07_V_openloop`, and each section's block is a file of its own — `W07_cases` fixes the eight commands, `W07_tab_formula` is §7-3, `W07_tab_track` is §7-4 and §7-5, `W07_tab_reint` is §7-6. The arrow names the file to open for that section.

Summary

Week Summary

Step	What was done	How it was verified
1	B built for four tilting units from the Week 3 column rule, with the mounting signs	`verify_otter_tilt4` prints 3×8, rank 3, null space 5
2	`otter.m` changed in four marked places into `otter_tilt4.m`	identities (a) and (b) exactly zero: $\delta = 0$ reproduces `otter.m`
3	the trigonometric form checked against Bf	identity (c), 7.19×10^{-14} over 200 random states
4	an open-loop model built, eight hand constants to the plant	`W07_1_build_openloop` prints overlaps 0, diagonals 0
5	the force of eight commands measured out of the plant	table (1), worst residual 2.8×10^{-14} N or N·m
6	the mirrored mounting shown to make surge symmetric	S1 and S2 both reach $\lvert X \rvert = 126.180$ N despite $k_{pos} \neq k_{neg}$
7	the residual yaw of a symmetric sway derived	$N = \bar{x}Y = 28.205$ N·m, matching S5 and S8 exactly
8	the thrust ratio that removes that yaw derived and run	$T_a = 1.5T_f$, i.e. $n_a = 73.485$ rad/s: $N = 0.000$ N·m at $X = -28.205$ N
9	the servo limit shown to act inside the plant	S8 asks $70°$, table (1) records $45°$, and S8 lies on S5
10	the eight tracks read at $t = 30$ s, one sheet each	table (3) and `img/W07_result_S1.png` … `_S8.png`
11	the model checked against an independent integrator	table (2), $0.00\text{e}{+00}$ m and m/s over eight 30-s runs
12	allocation stated as a choice among infinitely many solutions	42.99 N against 58.72 N for the same τ: the null space costs 37%
13	the minimum-norm solution derived and inverted to hardware	residual $0.00\text{e}{+00}$, $\rho = 1$, worst angle $26.88°$
14	the weight matrix measured over three decades	$\Sigma \lvert f_y \rvert$ fixed at 30.00 N; the worst angle **rises** $26.88° \rightarrow 31.73°$
15	the servo limit shown to be two straight lines per sign	five laws compared: `wls16` meets a demand at 0.020 N where one-guess `qp` returns nothing
16	every law timed against the 50 Hz budget	1.29 µs precomputed against `fmincon` 's worst case $70\,447$ µs
17	the shaft-speed limit shown to be the thrust cap	119.682 N ahead and 66.708 N astern, the astern cap 56% of the ahead one
18	both limits put inside the solve, the force measured out of the hull	mean error $7.353 \rightarrow 0.012$ N over 201 demands, for $12.80 \rightarrow 17.89\%$ of the budget
19	the four laws put into one model, selected by one constant	28.333, 29.677, 29.677 and 39.989 N delivered of a 40 N sway demand

Progress Check

★ Minimum condition for following the next week

Theory

- [ ] Able to state the column rule and apply it to one tilting unit, obtaining both of its columns
- [ ] Able to say why B is 3×8 and what the five null-space dimensions mean for an allocator
- [ ] Able to say where the mounting sign s_i sits in B, and why the two pairs have different zeros but the same sense
- [ ] Able to name the four places `otter.m` changes, and say why the damping is not one of them
- [ ] Able to derive $N = \bar{x}Y$ with $\bar{x} = (l_f - l_a)/2$, and predict 28.205 N·m without the script
- [ ] Able to say why re-integrating the logged input is a different check from table (1), and why a single-instant acceleration comparison is neither
- [ ] Able to name the input and the output of the allocation stage, and say what it knows in advance
- [ ] Able to derive $\mathbf{f} = \mathbf{B}^{\top}(\mathbf{B}\mathbf{B}^{\top})^{-1}\boldsymbol{\tau}_d$ with a Lagrange multiplier, and say why the inverted matrix is 3×3
- [ ] Able to say why the thrust cap needs one scalar after the solve while the servo limit does not
- [ ] Able to say what a weight matrix changes, what it cannot change on this geometry, and why only ratios matter
- [ ] Able to turn $|\delta_i| \le \alpha_{\max}$ into two linear inequalities, and state what has to be fixed first
- [ ] Able to say why there are sixteen convex subsets, and what taking only the sign combination of the closed form risks
- [ ] Able to say why the force demand belongs in the cost while the servo limit belongs in the constraints
- [ ] Able to show that $|n_i| \le n_{\max}$ and $|T_i| \le k_i n_{\max}^2$ are the same constraint, and give both cap values
- [ ] Able to say why the chord polygon of §7-12 can never break the cap, and where its worst 3.41% sits
- [ ] Able to say why a scalar ρ after the solve costs force in components no unit was straining

Laboratory

- [ ] `verify_otter_tilt4` ran and printed `PASS` with identities (a), (b) and (d) exactly zero
- [ ] `W07_1_build_openloop` ran and printed overlaps 0, diagonals 0
- [ ] `W07_V_openloop` ran and wrote all three tables and the eight sheets
- [ ] `W07_V_openloop('S5')` ran alone and rewrote only that one sheet
- [ ] A ninth case was commanded by hand from `W07_0_setup`, and its X, Y and N were predicted before the run
- [ ] `W07_CA(8)` ran and printed a residual of exactly 0 and $\rho = 1$
- [ ] `W07_CA(9)` ran and printed the seven-row price sweep and the two share figures
- [ ] `W07_CA(10)` ran and printed both the single hard demand and the 201-point trajectory
- [ ] `W07_CA(11)` ran and printed the microseconds each law costs against the $20\,000$ µs budget
- [ ] `W07_CA(12)` ran and printed all three tables, with the `rpm lim` column 0.0% for `wls16cap` alone
- [ ] `W07_2_build_alloc` ran and printed overlaps 0, diagonals 0
- [ ] The four laws ran inside `W07_alloc_chain` from the single `law` constant, and the delivered sway was read off each time

Recorded observations

- [ ] The residual yaw measured for a symmetric $45°$ sway command, and the arm it comes from
- [ ] The crab angle of S7, and why it is not the $28.74°$ of its own force
- [ ] The thrust the null-space solution of §7-8 wastes, as a percentage of the minimum-norm solution
- [ ] The value of $\sum_i |f_{y,i}|$ at $w = 1$ and at $w = 1000$, and why the two are equal
- [ ] The sign pattern a weighted minimum norm pointed at for $\boldsymbol{\tau} = [15;\ 70;\ 10]$, and the one that is actually feasible

- [ ] The worst-case time `fmincon` took, and whether it fits inside one 50 Hz step
- [ ] The two thrust caps of §7-12 line 4, and which of the two bound the pure-surge demand of table (1)
- [ ] The mean force error of `clip` and of `wls16cap` on the same sweep, measured out of the motion model

---

Assignment 7

- **Due**: before the Week 8 session
- **Submit**: code, results, and a short analysis

① Requirements

Requirements 1 to 3 are three runs, each one eight numbers given by hand in `W07_0_setup` : no allocator and no controller. Requirement 4 reverses the direction of the arithmetic and lets the allocator of §7-8 to §7-10 choose the eight numbers instead, and requirement 5 follows the chosen numbers through the hardware limits to the force the hull actually received.

1. **Reproduce the yaw-free crab.** Apply §7-4 line 8 to obtain the stern shaft speed that cancels the residual yaw of a $45°$ four-unit sway command. Derive it rather than copying 73.485 rad/s: show the step from $\sum x_i T_i = 0$ to $T_a = 1.5 T_f$ and the conversion to shaft speed, including which thrust coefficient applies and why. Report the surge force the cancellation costs.
2. **Find a motion that is neither.** Choose eight numbers for which $X = 0$ **and** $N = 0$ while $Y \neq 0$ — a pure sway, with no surge and no yaw. Keep $\delta_i \in \{+45°, -45°\}$ so that only the four shaft speeds are free. State the two linear conditions on $T_1 \dots T_4$, solve them by hand, convert each T_i to a shaft speed with the correct sign-dependent coefficient, and check every value against the shaft limits before running.
3. **Break it deliberately.** Take the solution of requirement 2 and change one shaft speed by 10 %. Report what appears in X, Y and N, and say which of the three asymmetries the change exposed.
4. **Let the allocator choose, and audit it.** Take the pure-sway demand solved by hand in requirement 2 as $\boldsymbol{\tau}_d$, scale its sway component until the servo limit is exceeded, and allocate that demand with `W07_alloc` under `'wpinv'` , `'clip'` , `'wls'` and `'wls16'` in turn, with $\mathbf{W} = \mathrm{diag}([1\ 10\ 1\ 10\ 1\ 10\ 1\ 10])$. For each law report $\max_i \lvert\delta_i\rvert$, $\lVert \mathbf{B}\mathbf{f} - \boldsymbol{\tau}_d \rVert$ and the exit flag returned in `info.exit` , then feed the eight numbers of the law with the smallest force error into `W07_hw` and command the resulting n_i and δ_i to the plant. State whether the measured X, Y and N match $\mathbf{B}\mathbf{f}$ and, where a law went past the limit, say what the plant did with the command it could not execute.
5. **Find the second limit, and measure past it.** Derive both thrust caps from `otter_config('quad_tilt')` by §7-12 line 3 — not by copying 119.682 N and 66.708 N — and state which of the two binds a pure surge demand and why. Then raise a pure surge demand $\boldsymbol{\tau}_d = [X_d\ 0\ 0]^\top$ from $X_d = 100$ N in steps until `'wls16'` first asks for a shaft speed outside the limits, report that X_d against the envelope corner of §7-12 line 4, and allocate the same demand with `'wls16cap'` . For both laws report the four T_i, the worst n_i, the two clip counts and the force error **measured out of the motion model** by §7-12 line 10, using `W07_chain` . Explain the direction in which `'wls16cap'` moved the load between the two pairs, and why the scalar ρ of §7-8 line 4 could not have done the same.

② Verification — mandatory

> **★ A claim is not a result. Numbers are required, with the averaging window stated**

What to report	How
hand-predicted X, Y, N for each of the three runs	from §7-1 line 7, before running anything
measured X, Y, N	table (1) of `W07_V_openloop`, or the same arithmetic in a script written for the purpose
the residual between the two	one number per run, and it must be below 10^{-12}
U, β and r at $t = 30$ s	read at $t = 30$ s, as table (3) does; state the instant, not an average
the clip flags	whether the shaft or the servo limit bound during the run
for requirement 4, one row per law	$\max_i \lvert\delta_i\rvert$, $\lVert \mathbf{B}\mathbf{f} - \boldsymbol{\tau}_d \rVert$, `info.exit` and the number of quadratic programs solved
the achieved force of the chosen law	$\mathbf{B}\mathbf{f}$ computed from the eight components, against the plant's measured X, Y, N
for requirement 5, the two thrust caps	each one as $k_i n_{\lim}^2$ with the coefficient named, and their ratio
for requirement 5, one row per law	the four T_i, the worst n_i, the shaft and servo clip counts, and $\lVert \boldsymbol{\tau}_{\text{act}} - \boldsymbol{\tau}_d \rVert$
for requirement 5, the measurement itself	the two calls to the motion model of §7-12 line 10 shown in the submitted code, not the allocator's own residual

A run whose predicted and measured forces disagree by more than rounding is a mistake in the prediction, not in the model. Find it before reporting. A law reporting a non-positive `info.exit` has returned no solution, and its angle and error figures are not results: report the flag rather than the numbers beside it.

③ Analysis (5–10 lines)

Requirement 2 asked for a pure sway and the vessel can produce one, while requirement 1's command cannot. Explain which freedom made the difference, counting it against the five null-space dimensions of §7-1. Then compare requirement 4 against requirements 1 to 3: state which of the two directions of arithmetic has a unique answer and which does not, name what `'wls16'` searched that `'wls'` did not, and say what the hand solution would have to be redone for if the demanded force changed every 20 ms. Close with requirement 5: state which of the two limits a law can satisfy without knowing the other, and why a residual reported by the allocator is not a statement about the hull.

Grading

Criterion	Weight
The model runs and produces the requested output	25%
Verification performed and numbers reported	40%
Correctness of the analysis	25%
Readability of the code	10%

Troubleshooting

- Only faults that have actually occurred. Hypothetical entries are not included

Symptom	Cause	Fix
A tilt of $70°$ or $90°$ produces exactly the $45°$ result	the servo clip inside `otter_tilt4`, §7-2 change 3	expected. Read the `delta` column of table (1), which prints what the plant received
Four equal $\delta_i = +45°$ at four equal n_i produce no force at all, only a spot turn	the mounting signs: $s_3 = s_4 = -1$ reverses the stern pair's force, so the four cancel in surge and sway	expected, and it is row S3. A sway needs the two pairs at **opposite** δ, which is S5
A B copied from another drawing or script has **every** sign reversed from the one in §7-1	the convention is not in the matrix but in s_i: a source that lets the bow zero push astern, $s = [-1,-1,+1,+1]$, produces exactly $-B$, and its third row reads $[-d_1, -l_f, -d_2, -l_f, \dots]$ — every arm of row 3 changes sign, because each one carries an s_i	neither labelling is wrong, but the two cannot be mixed. Rebuild B from the column rule rather than importing it: the course, `_tools/otter_B.m` and every measured number of §7-3 use $s = [+1,+1,-1,-1]$, and taking B from one convention while commanding δ in the other reverses every manoeuvre
A symmetric four-unit sway command yaws at 7.589 deg/s	$\bar{x} = 0.25$ m, §7-4 line 7	expected. Apply the ratio of §7-4 line 8 to cancel it, and accept the surge it costs
`W07_result_openloop.png` does not change after a run	a run of fewer than eight scenarios deliberately does not overwrite it, so the figure §7-5 embeds always carries all eight	run `W07_V_openloop` with no argument
A Constant block shows `-C-` instead of its value	`mss_style` sets Constant width to 55 px, and a longer expression does not fit	keep the Value expression short — the model uses `d_cmd`, not `delta_cmd`
`n_cmd = 600` saturates immediately	the command is rad/s, not rpm	60 rad/s is about 573 rpm; the limit is 103.931 rad/s
`W07_V_openloop` runs but no figure appears	the script was called before `W07_0_setup`	run `W07_0_setup` first; it defines `animate`, `pace` and `h`
A quadratic program solved in one sign combination reports an angle and a force error that look reasonable, while the demand is in fact unreachable in that combination	`quadprog` and `lsqlin` return their last iterate together with a non-positive exit flag, and an iterate of an infeasible problem satisfies nothing	read `info.exit` before reading anything else, and exclude those instants from every statistic. Experiment 7-10 does this, which is why its limit and error columns are computed over the solved points only
Multiplying $\mathbf{W}$ by a constant changes nothing	$\mathbf{W}^{-1}\mathbf{B}^{\mathsf{T}}(\mathbf{B}\mathbf{W}^{-1}\mathbf{B}^{\mathsf{T}})^{-1}$ is unchanged by a scalar factor: only the ratios between the eight entries enter the solution	change the ratio, not the magnitude. Measured in Experiment 7-9: $\mathbf{W}$ and $1000\mathbf{W}$ agree to $2.81\text{e}{-15}$ N
Raising the price of the sideways components does not reduce the tilt	row 2 of $\mathbf{B}$ is $[0\,1\,0\,1\,0\,-1\,0\,-1]$, so $\sum_i \lvert f_{y,i}\rvert$ is pinned by the demanded Y and the price can only move the yaw onto the short f_x arms	a hard limit is not a price. Put $\alpha_{\max}$ in the constraints, as §7-10 does, and use $\mathbf{W}$ for what it can do — moving load between units
Two timing runs of `W07_CA(11)` disagree by tens of per cent	the solver timings include allocation, interpreter and operating-system load	expected. The ordering of the laws and the orders of magnitude between them are reproducible; a single figure

Symptom	Cause	Fix
		is not. Report the run, not a remembered number
A script run by name appears to do nothing at all	a `.slx` sharing its base name takes precedence, and MATLAB opens the model instead of running the script	never let a `.m` and a `.slx` share a name; the builders append `_run` where a collision is possible
A law that keeps every δ_i inside $45°$ still drives a shaft past $n_{\min}$	the servo limit and the thrust cap are two different limits, and §7-10 put only the first inside the solve	use `'wls16cap'`, which adds the chord rows of §7-12 line 6. The `rpm lim` column of Experiment 7-12 is the check
An allocation reports a residual of 0 while the hull produces a visibly different force	the residual was computed as $\|\mathbf{Bf} - \boldsymbol{\tau}_d\|$ inside the allocator, where the hardware limits have not yet been applied	measure out of the motion model by §7-12 line 10. `W07_chain` does this, and `W07_hw` alone does not
`wpinv` and `clip` report **identical** errors, which makes the clipping look pointless	the plant clips at the same value the allocator clips to, so the two produce the same force	expected, and it is §7-12 line 11. Clipping earns nothing on its own; it earns something only when the clipped limit is inside the optimisation

References

Primary

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, §11.2 (thrust allocation and the configuration matrix), §7.3 (manoeuvring models and damping)
- MSS toolbox, `Tools/MSS/VESSELS/otter.m` — the hull, the propeller coefficients and the shaft limits used unchanged
- MSS toolbox, `demoOtterUSVPathFollowingHeadingControl` — the block-level conventions this course's models follow
- Johansen, T. A. and Fossen, T. I. "Control allocation — A survey", *Automatica* **49**(5):1087–1103, 2013. DOI 10.1016/j.automatica.2013.01.035 — the source for the classification §7-8 to §7-10 follows: unconstrained closed form, weighted pseudo-inverse, and constrained quadratic programming
- Härkegård, O. "Efficient active set algorithms for solving constrained least squares problems in aircraft control allocation", *41st IEEE Conference on Decision and Control*, 2002, pp. 1295–1300. DOI 10.1109/CDC.2002.1184694 — the weighted least squares formulation of §7-10, with the force demand in the cost and the limits in the constraints
- Härkegård, O. *Backstepping and Control Allocation with Applications to Flight Control*, PhD thesis, Linköping University, 2003 — the longer treatment of the same formulation, including why an active-set method is preferred to a general-purpose solver when the problem is solved every step
- Härkegård, O. *QCAT — Quadratic Programming Control Allocation Toolbox*, MATLAB toolbox accompanying the thesis above. Its `wls_alloc` is the active-set implementation of the §7-10 cost, and its `vview` draws the attainable moment set that §7-12 line 4 computes one corner of. The laws of this week are written out from the equations above rather than called from the toolbox, so that each row of $\mathbf{A}$ can be traced to a sentence
- Johansen, T. A., Fossen, T. I. and Berge, S. P. "Constrained nonlinear control allocation with singularity avoidance using sequential quadratic programming", *IEEE Transactions on Control Systems Technology* **12**(1):211–216, 2004. DOI 10.1109/TCST.2003.821952 — the marine form of the sector constraint of §7-12 line 5, and the rate and singularity terms listed as left out above

Optimization Toolbox documentation

Function	What it solves here	Link
`lsqlin`	the least-squares form of §7-10 directly, without forming $\mathbf{C}^\top\mathbf{C}$	lsqlin
`quadprog`	the same problem written as a quadratic program	quadprog
`fmincon`	the nonlinear form, with $\lvert\delta_i\rvert$ kept as `atan2`	fmincon
`optimoptions`	silencing the solver and fixing its algorithm before the loop	optimoptions

- Quadratic programming, the problem class §7-10 reduces the servo limit to — overview

In this course

- Week 3 — the column rule, and the only definition of B this course uses
- **Appendix A1 — actuation and the control effectiveness matrix**: the column rule written out (§A1-2), the rank of B and what it leaves unreachable (§A1-3), the extended thrust vector of a tilting unit (§A1-4), and the attainable control set (§A1-6)
- **Week 6 §6-2 — the square allocation**: three demands, three thrusters, exactly one answer, and what leaks out when that answer is clipped at a ceiling. This week is the case the square rule cannot cover: the null space is five-dimensional, so there is something to choose, and §7-7 to §7-12 are how it is chosen
- Week 5 — saturation, and what a limit does to a command
- Week 6 — `otter_bow.m`: changing the motion model rather than adding to its input
- `_tools/otter_tilt4.m` — the four changes, each marked `% ---- CHANGED (i of 4)`
- `_tools/verify_otter_tilt4.m` — the four identities of Experiment 7-1
- `W07_simulink/W07_alloc.m` — the eight allocation laws of §7-8 to §7-12 behind one `method` flag
- `W07_simulink/W07_hw.m` — the thrust-curve inversion of §7-8 lines 1 to 5, including the scalar ρ
- `W07_simulink/W07_CA.m` — the one entry point of §7-8 to §7-12: `W07_CA(n)` runs the section whose number is given
- `W07_simulink/W07_sl_alloc.m` — the wrapper the model's MATLAB Function block calls, mapping the `law` constant of §7-13 onto four of those eight methods

---

Next Week

- **The closed loop, and what the allocator owes it.** §7-7 to §7-12 assumed the three demanded numbers arrived from outside. Closing the loop puts a controller in front of the allocator, and three of this week's measurements then become the controller's problem: the force error a constrained law pays (§7-10) is a disturbance the controller did not ask for, the worst-case solve time (§7-11) is dead time inside the loop, and the gap between the force the allocator reported and the force the hull received (§7-12) is a model error the controller must reject without ever being told of it
- The five null-space dimensions counted in §7-1 are now spent three ways — minimum norm, a price list, and sixteen convex subsets — and none of the three asked what the hull was trying to do
- What must be prepared: the three asymmetries of §7-4 stated from memory with the numbers 0.5817, 0.25 m and $45°$ attached to the right one, and the four allocation laws of §7-10 ranked by what each one gives up
- What is deliberately left open: servo rate limits, singularity-free azimuth handling, and solving the sixteen subsets offline rather than every step